HTTP fundamentals

Authors: MDN (Mozilla Developer Network)

Sources: MDN (https://developer.mozilla.org)

Table of contents

HTTP guide

- An overview of HTTP 1
- Evolution of HTTP 10
- HTTP messages 18
- A typical HTTP session 25
- HTTP request methods 33
- Connection management in HTTP1.x 34
- Protocol upgrade mechanism 39

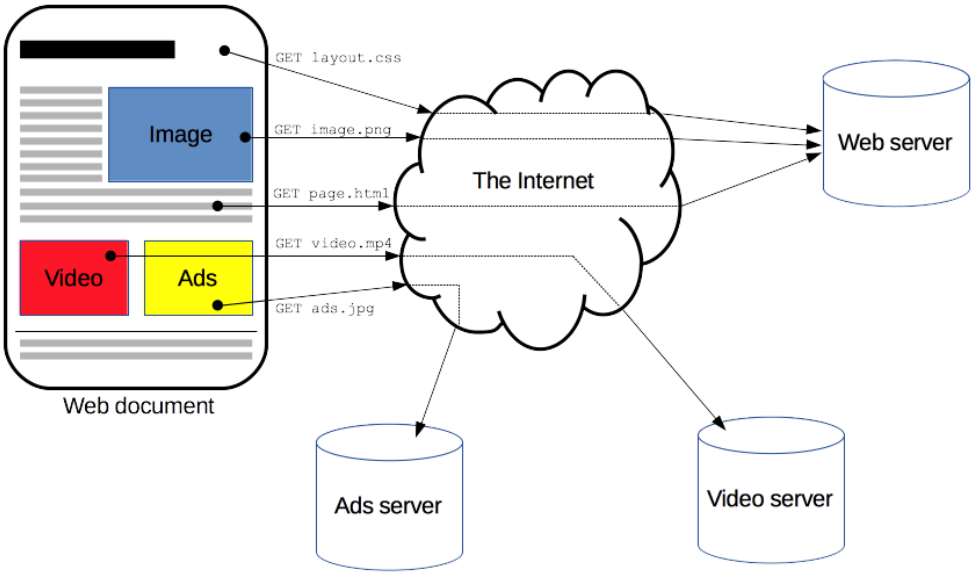
HTTP security

- Content Security Policy (CSP) 44
- HTTP Strict Transport Security (HSTS) 52
- Cookie security 121
- X-Content-Type-Options 55
- X-Frame-Options 57
- X-XSS-Protection 61

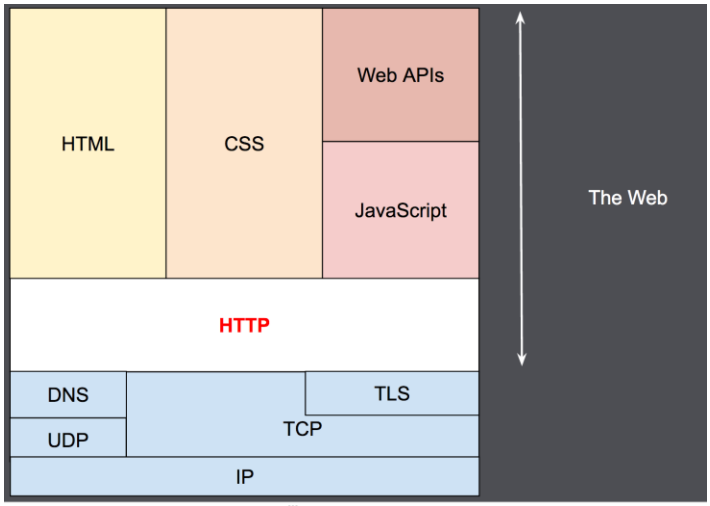
HTTP access control (CORS) 64
HTTP authentication 76
HTTP caching 82
HTTP compression 100
HTTP conditional requests 104
HTTP content negotiation 114
HTTP cookies 121
HTTP range requests 129
HTTP redirects 133
HTTP resources and specifications 141

An overview of HTTP

HTTP is a protocol for fetching resources such as HTML documents. It is the foundation of any data exchange on the Web and it is a client-server protocol, which means requests are initiated by the recipient, usually the Web browser. A complete document is reconstructed from the different sub-documents fetched, for instance, text, layout description, images, videos, scripts, and more.



Clients and servers communicate by exchanging individual messages (as opposed to a stream of data). The messages sent by the client, usually a Web browser, are called requests and the messages sent by the server as an answer are called responses.

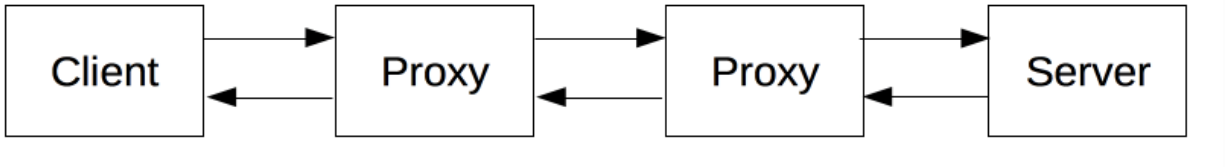


Designed in the early 1990s, HTTP is an extensible protocol that has evolved over time. It is an application layer protocol that is sent over TCP, or over a TLS-encrypted TCP connection, though any reliable transport protocol could theoretically be used. Due to its extensibility, it is used to not only fetch hypertext documents but also images and videos or to post content to servers, like with HTML form results. HTTP can also be used to fetch parts of documents to update Web pages on demand.

Components of HTTP-based systems

HTTP is a client-server protocol: requests are sent by one entity, the user-agent (or a proxy on behalf of it). The user-agent is usually a Web browser, but it can be anything, for example, a robot that crawls the Web to populate and maintain a search engine index.

Each individual request is sent to a server, which handles it and provides an answer called the response. There are numerous entities between the client and the server, collectively called proxies, which perform different operations and act as gateways or caches, for example.



In reality, there are more computers between a browser and the server handling the request: there are routers, modems, and more. Thanks to the layered design of the Web, these are hidden in the network and transport layers. HTTP is on top, at the application layer. Although important for diagnosing network problems, the underlying layers are mostly irrelevant to the description of HTTP.

Client: the user-agent

The user-agent is any tool that acts on behalf of the user. This role is primarily performed by the Web browser, but it may also be performed by programs used by engineers and Web developers to debug their applications.

The browser is always the entity initiating the request. It is never the server (though some mechanisms have been added over the years to simulate server-initiated messages).

To display a Web page, the browser sends an original request to fetch the HTML document that represents the page. It then parses this file, making additional requests corresponding to execution scripts, layout information (CSS) to display, and sub-resources contained within the page (usually images and videos). The Web browser then combines these resources to present the complete document, the Web page. Scripts executed by the browser can fetch more resources in later phases and the browser updates the Web page accordingly.

A Web page is a hypertext document. This means some parts of the displayed content are links, which can be activated (usually by a click of the mouse) to fetch a new Web page, allowing the user to direct their user-agent and navigate through the Web. The browser translates these directions into HTTP requests and further interprets the HTTP responses to present the user with a clear response.

The Web server

On the opposite side of the communication channel is the server, which serves the document as requested by the client. A server appears as only a single machine virtually; but it may actually be a collection of servers sharing the load (load balancing), or other software (such as caches, a database server, or e-commerce servers), totally or partially generating the document on demand.

A server is not necessarily a single machine, but several server software instances can be hosted on the same machine. With HTTP/1.1 and the Host header, they can share the same IP address.

Proxies

Between the Web browser and the server, numerous computers and machines relay the HTTP messages. Due to the layered structure of the Web stack, most of these operate at the transport, network or physical levels, becoming transparent at the HTTP layer and potentially having a significant impact on performance. Those operating at the application layers are generally called proxies. So, a **proxy is a server or computer system that sits between the web browser and the server, forwarding requests without alteration (transparent) or modifying requests before passing them to the server (non-transparent), potentially impacting performance**. Proxies may perform numerous functions:

- caching (the cache can be public or private, like the browser cache)
- filtering (like an antivirus scan or parental controls)
- load balancing (to allow multiple servers to serve different requests)
- authentication (to control access to different resources)
- logging (allowing the storage of historical information)

Basic aspects of HTTP

HTTP is simple

HTTP is generally designed to be simple and human-readable, even with the added complexity introduced in HTTP/2 by encapsulating HTTP messages into frames. HTTP messages can be read and understood by humans, providing easier testing for developers, and reduced complexity for newcomers.

HTTP is extensible

Introduced in HTTP/1.0, HTTP headers make this protocol easy to extend and experiment with. New functionality can even be introduced by a simple agreement between a client and a server about a new header's semantics.

HTTP is stateless, but not sessionless

HTTP is stateless: there is no link between two requests being successively carried out on the same connection. This immediately has the prospect of being problematic for users attempting to interact with certain pages coherently, for example, using e-commerce shopping baskets. But while the core of HTTP itself is stateless, HTTP cookies allow the use of stateful sessions. Using header extensibility, HTTP Cookies are added to the workflow, allowing session creation on each HTTP request to share the same context or the same state.

HTTP and connections

A connection is controlled at the transport layer, and therefore fundamentally out of scope for HTTP. HTTP doesn't require the underlying transport protocol to be connection-based; it only requires it to be reliable, or not lose messages (at minimum, presenting an error in such cases). Among the two most common transport protocols on the Internet, TCP is reliable and UDP isn't. HTTP therefore relies on the TCP standard, which is connection-based.

Before a client and server can exchange an HTTP request/response pair, they must establish a TCP connection, a process which requires several round-trips. The default behavior of HTTP/1.0 is to open a separate TCP connection for each HTTP request/response pair. This is less efficient than sharing a single TCP connection when multiple requests are sent in close succession.

In order to mitigate this flaw, HTTP/1.1 introduced pipelining (which proved difficult to implement) and persistent connections: the underlying TCP connection can be partially controlled using the Connection header. HTTP/2 went a step further by multiplexing messages over a single connection, helping keep the connection warm and more efficient.

What can be controlled by HTTP

This extensible nature of HTTP has, over time, allowed for more control and functionality of the Web. Cache and authentication methods were functions handled early in HTTP history. The ability to relax the origin constraint, by contrast, was only added in the 2010s.

Here is a list of common features controllable with HTTP:

- **Caching**: How documents are cached can be controlled by HTTP. The server can instruct proxies and clients about what to cache and for how long. The client can instruct intermediate cache proxies to ignore the stored document.

- **Relaxing the origin constraint**: To prevent snooping and other privacy invasions, Web browsers enforce strict separation between websites. Only pages from the same origin can access all the information of a Web page. Though such a constraint is a burden to the server, HTTP headers can relax this strict separation on the server side, allowing a document to become a patchwork of information sourced from different domains; there could even be security-related reasons to do so.

- **Authentication**: Some pages may be protected so that only specific users can access them. Basic authentication may be provided by HTTP, either using the WWW-Authenticate and similar headers or by setting a specific session using HTTP cookies.

- **Proxy and tunneling**: Servers or clients are often located on intranets and hide their true IP address from other computers. HTTP requests then go through proxies to cross this network barrier. Not all proxies are HTTP proxies. The SOCKS protocol, for example, operates at a lower level. Other protocols, like FTP, can be handled by these proxies.

- **Sessions**: Using HTTP cookies allows you to link requests with the state of the server. This creates sessions, despite basic HTTP being a state-less protocol. This is useful not only for e-commerce shopping baskets, but also for any site allowing user configuration of the output.

HTTP flow

When a client wants to communicate with a server, either the final server or an intermediate proxy, it performs the following steps:

1. **Open a TCP connection**: The TCP connection is used to send a request, or several, and receive an answer. The client may open a new connection, reuse an existing connection, or open several TCP connections to the servers.

2. **Send an HTTP message**: HTTP messages (before HTTP/2) are human-readable. With HTTP/2, these simple messages are encapsulated in frames, making them impossible to read directly, but the principle remains the same. For example:

```
GET / HTTP/1.1
Host: developer.mozilla.org
Accept-Language: fr
```

3. **Read the response sent by the server**, such as:

```
HTTP/1.1 200 OK
Date: Sat, 09 Oct 2010 14:28:02 GMT
Server: Apache
Last-Modified: Tue, 01 Dec 2009 20:18:22 GMT
ETag: "51142bc1-7449-479b075b2891b"
Accept-Ranges: bytes
Content-Length: 29769
Content-Type: text/html

<!DOCTYPE html>… (here come the 29769 bytes of the requested web page)
```

4. **Close or reuse the connection for further requests**.

If HTTP pipelining is activated, several requests can be sent without waiting for the first response to be fully received. HTTP pipelining has proven difficult to implement in existing networks, where old pieces of software coexist with modern versions. HTTP pipelining has been superseded in HTTP/2 with more robust multiplexing requests within a frame.

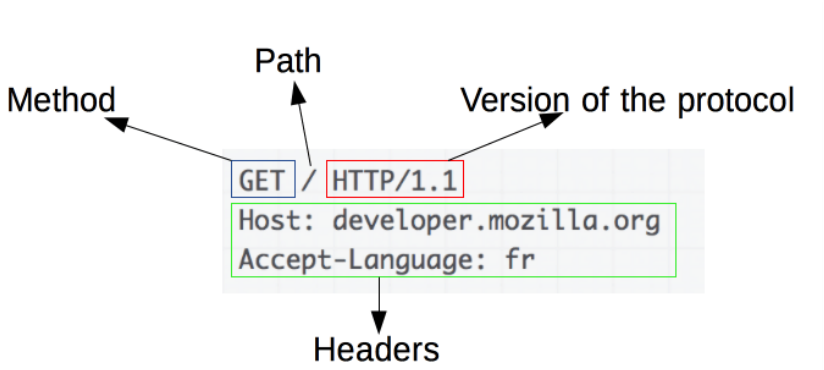
HTTP Messages

HTTP messages, as defined in HTTP/1.1 and earlier, are human-readable. In HTTP/2, these messages are embedded into a binary structure, a frame, allowing optimizations like compression of headers and multiplexing. Even if only part of the original HTTP message is sent in this version of HTTP, the semantics of each message is unchanged and the client reconstitutes (virtually) the original HTTP/1.1 request. It is therefore useful to comprehend HTTP/2 messages in the HTTP/1.1 format.

There are two types of HTTP messages, requests and responses, each with its own format.

Requests

An example HTTP request:

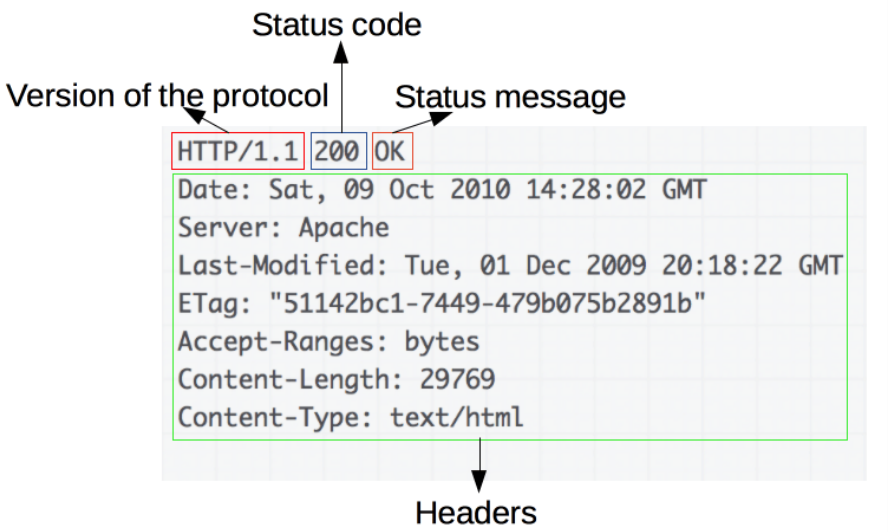


Requests consist of the following elements:

- An HTTP method, usually a verb like GET, POST, or a noun like OPTIONS or HEAD that defines the operation the client wants to perform. Typically, a client wants to fetch a resource (using GET) or post the value of an HTML form (using POST), though more operations may be needed in other cases.
- The path of the resource to fetch; the URL of the resource stripped from elements that are obvious from the context, for example without the protocol (http://), the domain (here, developer.mozilla.org), or the TCP port (here, 80).
- The version of the HTTP protocol.
- Optional headers that convey additional information for the servers.
- A body, for some methods like POST, similar to those in responses, which contain the resource sent.

Responses

An example response:



Responses consist of the following elements:

- The version of the HTTP protocol they follow.
- A status code, indicating if the request was successful or not, and why.
- A status message, a non-authoritative short description of the status code.
- HTTP headers, like those for requests.
- Optionally, a body containing the fetched resource.

APIs based on HTTP

The most commonly used API based on HTTP is the `XMLHttpRequest` API, which can be used to exchange data between a user agent and a server. The modern `Fetch API` provides the same features with a more powerful and flexible feature set.

Another API, server-sent events, is a one-way service that allows a server to send events to the client, using HTTP as a transport mechanism. Using the `EventSource` interface, the client opens a connection and establishes event handlers. The client browser automatically converts the messages that arrive on the HTTP stream into appropriate `Event` objects. Then it delivers them to

the event handlers that have been registered for the events' `type` if known, or to the `onmessage` event handler if no type-specific event handler was established.

Conclusion

HTTP is an extensible protocol that is easy to use. The client-server structure, combined with the ability to add headers, allows HTTP to advance along with the extended capabilities of the Web.

Though HTTP/2 adds some complexity by embedding HTTP messages in frames to improve performance, the basic structure of messages has stayed the same since HTTP/1.0. Session flow remains simple, allowing it to be investigated and debugged with a simple HTTP message monitor.

This lesson was adapted from MDN Web Docs: https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview

Evolution of HTTP

HTTP (HyperText Transfer Protocol) is the underlying protocol of the World Wide Web. Developed by Tim Berners-Lee and his team between 1989-1991, HTTP has gone through many changes that have helped maintain its simplicity while shaping its flexibility. Keep reading to learn how HTTP evolved from a protocol designed to exchange files in a semitrusted laboratory environment into a modern internet maze that carries images and videos in high resolution and 3D.

Invention of the World Wide Web

In 1989, while working at CERN, Tim Berners-Lee wrote a proposal to build a hypertext system over the Internet. Initially called the Mesh, it was later renamed the *World Wide Web* during its implementation in 1990. Built over the existing TCP and IP protocols, it consisted of 4 building blocks:

1. A textual format to represent hypertext documents, the HyperText Markup Language (HTML).
2. A simple protocol to exchange these documents, the HyperText Transfer Protocol (HTTP).
3. A client to display (and edit) these documents, the first web browser called the *WorldWideWeb*.
4. A server to give access to the document, an early version of httpd

These four building blocks were completed by the end of 1990, and the first servers were running outside of CERN by early 1991. On August 6, 1991, Tim Berners-Lee posted on the public *alt.hypertext* newsgroup. This is now considered to be the official start of the World Wide Web as a public project.
The HTTP protocol used in those early phases was very simple. It was later dubbed HTTP/0.9 and is sometimes called the one-line protocol.

HTTP/0.9 – The one-line protocol

The initial version of HTTP had no version number; it was later called 0.9 to differentiate it from later versions. HTTP/0.9 was extremely simple: requests consisted of a single line and started with the only possible method GET followed by the path to the resource. The full URL wasn't included as the protocol, server, and port weren't necessary once connected to the server.

```
GET /mypage.html
```

The response was extremely simple, too: it only consisted of the file itself.

```
<html>
  A very simple HTML page
</html>
```

Unlike subsequent evolutions, there were no HTTP headers. This meant that only HTML files could be transmitted. There were no status or error codes. If there was a problem, a specific HTML file was generated and included a description of the problem for human consumption.

HTTP/1.0 – Building extensibility

HTTP/0.9 was very limited, but browsers and servers quickly made it more versatile:

- Versioning information was sent within each request (HTTP/1.0 was appended to the GET line).
- A status code line was also sent at the beginning of a response. This allowed the browser itself to recognize the success or failure of a request and adapt its behavior accordingly. For example, updating or using its local cache in a specific way.
- The concept of HTTP headers was introduced for both requests and responses. Metadata could be transmitted and the protocol became extremely flexible and extensible.
- Documents other than plain HTML files could be transmitted thanks to the Content-Type header.

At this point in time, a typical request and response looked like this:

```
GET /mypage.html HTTP/1.0
User-Agent: NCSA_Mosaic/2.0 (Windows 3.1)

200 OK
Date: Tue, 15 Nov 1994 08:12:31 GMT
Server: CERN/3.0 libwww/2.17
Content-Type: text/html
<HTML>
A page with an image
  <IMG SRC="/myimage.gif">
</HTML>
```

It was followed by a second connection and a request to fetch the image (with the corresponding response):

```
GET /myimage.gif HTTP/1.0
User-Agent: NCSA_Mosaic/2.0 (Windows 3.1)

200 OK
Date: Tue, 15 Nov 1994 08:12:32 GMT
Server: CERN/3.0 libwww/2.17
Content-Type: text/gif
(image content)
```

Between 1991-1995, these were introduced with a try-and-see approach. A server and a browser would add a feature and see if it got traction. Interoperability problems were common. In an effort to solve these issues, an informational document that described the common practices was published in November 1996. This was known as RFC 1945 and defined HTTP/1.0.

HTTP/1.1 – The standardized protocol

In the meantime, proper standardization was in progress. This happened in parallel to the diverse implementations of HTTP/1.0. The first standardized version of HTTP, HTTP/1.1, was published in early 1997, only a few months after HTTP/1.0.

HTTP/1.1 clarified ambiguities and introduced numerous improvements:

- A connection could be reused, which saved time. It no longer needed to be opened multiple times to display the resources embedded in the single original document.
- Pipelining was added. This allowed a second request to be sent before the answer to the first one was fully transmitted. This lowered the latency of the communication.
- Chunked responses were also supported.
- Additional cache control mechanisms were introduced.
- Content negotiation, including language, encoding, and type, was introduced. A client and a server could now agree on which content to exchange.
- Thanks to the Host header, the ability to host different domains from the same IP address allowed server collocation.

A typical flow of requests, all through one single connection, looked like this:

```
GET /en-US/docs/Glossary/Simple_header HTTP/1.1
Host: developer.mozilla.org
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.9; rv:50.0) Gecko/20100101
            Firefox/50.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Referer: https://developer.mozilla.org/en-US/docs/Glossary/Simple_header

200 OK
Connection: Keep-Alive
Content-Encoding: gzip
Content-Type: text/html; charset=utf-8
Date: Wed, 20 Jul 2016 10:55:30 GMT
Etag: "547fa7e369ef56031dd3bff2ace9fc0832eb251a"
Keep-Alive: timeout=5, max=1000
Last-Modified: Tue, 19 Jul 2016 00:59:33 GMT
Server: Apache
Transfer-Encoding: chunked
Vary: Cookie, Accept-Encoding

(content)

GET /static/img/header-background.png HTTP/1.1
Host: developer.mozilla.org
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.9; rv:50.0) Gecko/20100101
            Firefox/50.0
Accept: */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Referer: https://developer.mozilla.org/en-US/docs/Glossary/Simple_header

200 OK
Age: 9578461
Cache-Control: public, max-age=315360000
Connection: keep-alive
Content-Length: 3077
Content-Type: image/png
Date: Thu, 31 Mar 2016 13:34:46 GMT
Last-Modified: Wed, 21 Oct 2015 18:27:50 GMT
Server: Apache

(image content of 3077 bytes)
```

HTTP/1.1 was first published as RFC 2068 in January 1997.

More than 15 years of extensions

The extensibility of HTTP made it easy to create new headers and methods. Even though the HTTP/1.1 protocol was refined over two revisions, RFC 2616 published in June 1999, and RFC 7230-RFC 7235 published in June 2014 before the release of HTTP/2, it was extremely stable for more than 15 years.

Using HTTP for secure transmissions

The largest change to HTTP was made at the end of 1994. Instead of sending HTTP over a basic TCP/IP stack, the computer services company Netscape Communications created an additional encrypted transmission layer on top of it: SSL. SSL 1.0 was never released to the public, but SSL 2.0 and its successor SSL 3.0 allowed for the creation of e-commerce websites. To do this, they encrypted and guaranteed the authenticity of the messages exchanged between the server and the client. SSL was eventually standardized and became TLS.

During the same time period, it became clear that an encrypted transport layer was needed. The web was no longer a mostly academic network and instead became a jungle where advertisers, random individuals, and criminals competed for as much private data as possible. As the applications built over HTTP became more powerful and required access to private information like address books, email, and user location, TLS became necessary outside the e-commerce use case.

Using HTTP for complex applications

Tim Berners-Lee didn't originally envision HTTP as a read-only medium. He wanted to create a web where people could add and move documents remotely—a kind of distributed file system. Around 1996, HTTP was extended to allow authoring, and a standard called WebDAV was created. It grew to include specific applications like CardDAV for handling address book entries and CalDAV for dealing with calendars. But all these *DAV extensions had a flaw: they were only usable when implemented by the servers.

In 2000, a new pattern for using HTTP was designed: representational state transfer (or REST). The API wasn't based on the new HTTP methods, but instead relied on access to specific URIs with basic HTTP/1.1 methods. This allowed any web application to let an API retrieve and modify its data without having to update the browsers or the servers. All necessary information was embedded in the files that the websites served through standard HTTP/1.1. The drawback of the REST model was that each website defined its own nonstandard RESTful API and had total control

of it. This differed from the *DAV extensions where clients and servers were interoperable. RESTful APIs became very common in the 2010s.

Since 2005, more APIs have become available to web pages. Several of these APIs create extensions to the HTTP protocol for specific purposes:

- Server-sent events, where the server can push occasional messages to the browser.
- WebSocket, a new protocol that can be set up by upgrading an existing HTTP connection.

Relaxing the security model of the web

HTTP is independent of the web security model, known as the same-origin policy. In fact, the current web security model was developed after the creation of HTTP! Over the years, it proved useful to lift some restrictions of this policy under certain constraints. The server transmitted how much and when to lift such restrictions to the client using a new set of HTTP headers. These were defined in specifications like Cross-Origin Resource Sharing (CORS) and the Content Security Policy (CSP).
In addition to these large extensions, many other headers were added, sometimes only experimentally. Notable headers are the Do Not Track (DNT) header to control privacy, X-Frame-Options, and Upgrade-Insecure-Requests but many more exist.

HTTP/2 – A protocol for greater performance

Over the years, web pages became more complex. Some of them were even applications in their own right. More visual media was displayed and the volume and size of scripts adding interactivity also increased. Much more data was transmitted over significantly more HTTP requests and this created more complexity and overhead for HTTP/1.1 connections. To account for this, Google implemented an experimental protocol SPDY in the early 2010s. This alternative way of exchanging data between client and server amassed interest from developers working on both browsers and servers. SPDY defined an increase in responsiveness and solved the problem of duplicate data transmission, serving as the foundation for the HTTP/2 protocol.

The HTTP/2 protocol differs from HTTP/1.1 in a few ways:

- It's a binary protocol rather than a text protocol. It can't be read and created manually. Despite this hurdle, it allows for the implementation of improved optimization techniques.
- It's a multiplexed protocol. Parallel requests can be made over the same connection, removing the constraints of the HTTP/1.x protocol.

- It compresses headers. As these are often similar among a set of requests, this removes the duplication and overhead of data transmitted.
- It allows a server to populate data in a client cache through a mechanism called the server push.

Officially standardized in May 2015, HTTP/2 use peaked in January 2022 at 46.9% of all websites. High-traffic websites showed the most rapid adoption in an effort to save on data transfer overhead and subsequent budgets.

This rapid adoption was likely because HTTP/2 didn't require changes to websites and applications. To use it, only an up-to-date server that communicated with a recent browser was necessary. Only a limited set of groups was needed to trigger adoption, and as legacy browser and server versions were renewed, usage naturally increased, without significant work for web developers.

Post-HTTP/2 evolution

HTTP's extensibility is still being used to add new features. Notably, we can cite new extensions of the HTTP protocol that appeared in 2016:

- Support for Alt-Svc allowed the dissociation of the identification and the location of a given resource. This meant a smarter CDN caching mechanism.
- The introduction of client hints allowed the browser or client to proactively communicate information about its requirements and hardware constraints to the server.
- The introduction of security-related prefixes in the Cookie header helped guarantee that secure cookies couldn't be altered.

HTTP/3 - HTTP over QUIC

The next major version of HTTP, HTTP/3 has the same semantics as earlier versions of HTTP but uses QUIC instead of TCP for the transport layer portion. By October 2022, 26% of all websites were using HTTP/3.

QUIC is designed to provide much lower latency for HTTP connections. Like HTTP/2, it is a multiplexed protocol, but HTTP/2 runs over a single TCP connection, so packet loss detection and retransmission handled at the TCP layer can block all streams. QUIC runs multiple streams over UDP and implements packet loss detection and retransmission independently for each stream, so that if an error occurs, only the stream with data in that packet is blocked.

Defined in RFC 9114, HTTP/3 is supported by most major browsers including Chromium (and its variants such as Chrome and Edge) and Firefox.

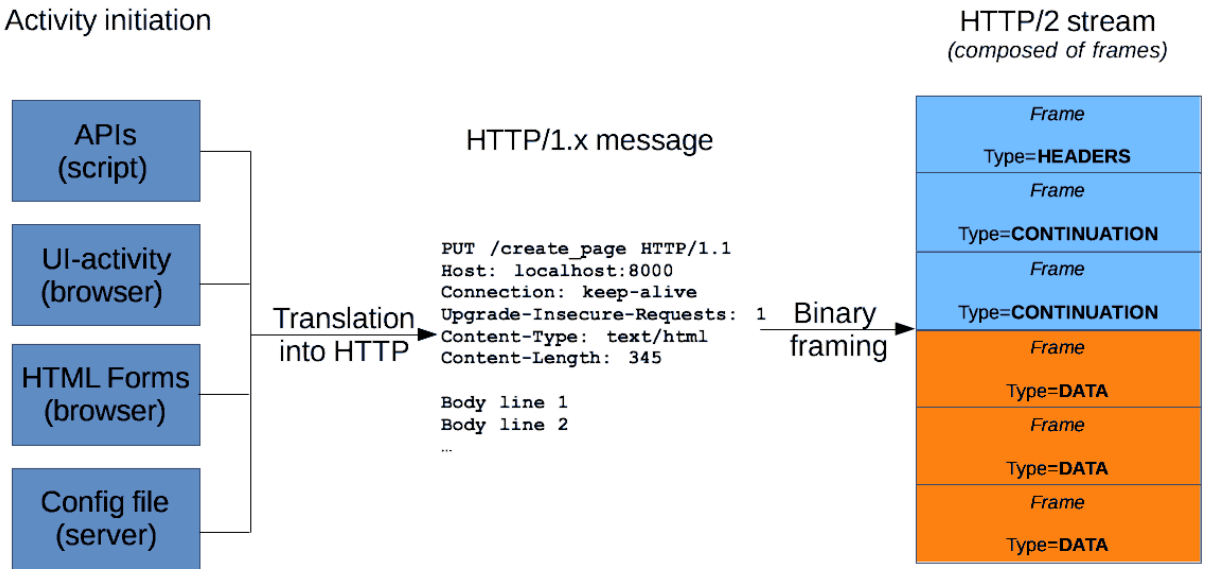
This lesson was adapted from MDN Web Docs:
https://developer.mozilla.org/en-US/docs/Web/HTTP/Basics_of_HTTP/Evolution_of_HTTP

HTTP Messages

HTTP messages are how data is exchanged between a server and a client. There are two types of messages: **requests** sent by the client to trigger an action on the server and **responses**, the answer from the server.

HTTP messages are composed of textual information encoded in ASCII and span over multiple lines. In HTTP/1.1, and earlier versions of the protocol, these messages were openly sent across the connection. In HTTP/2, the once human-readable message is now divided up into HTTP frames, providing optimization and performance improvements.

Web developers, or webmasters, rarely craft these textual HTTP messages themselves: software, a Web browser, proxy, or Web server, perform this action. They provide HTTP messages through config files (for proxies or servers), APIs (for browsers), or other interfaces.



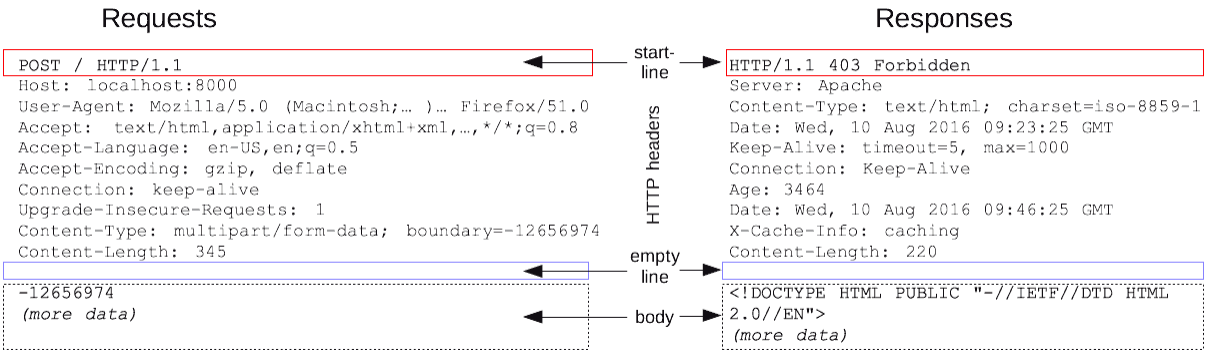
The HTTP/2 binary framing mechanism has been designed to not require any alteration of the APIs or config files applied: it is broadly transparent to the user.

HTTP requests, and responses, share similar structure and are composed of:

1. A *start-line* describing the requests to be implemented, or its status of whether successful or a failure. This start-line is always a single line.
2. An optional set of *HTTP headers* specifying the request, or describing the body included in the message.
3. A blank line indicating all meta-information for the request has been sent.

4. An optional *body* containing data associated with the request (like content of an HTML form), or the document associated with a response. The presence of the body and its size is specified by the start-line and HTTP headers.

The start-line and HTTP headers of the HTTP message are collectively known as the *head* of the requests, whereas its payload is known as the *body*.



HTTP Requests

Start line

HTTP requests are messages sent by the client to initiate an action on the server. Their *start-line* contain three elements:

1. An *HTTP method*, a verb (like `GET`, `PUT` or `POST`) or a noun (like `HEAD` or `OPTIONS`), that describes the action to be performed. For example, `GET` indicates that a resource should be fetched or `POST` means that data is pushed to the server (creating or modifying a resource, or generating a temporary document to send back).

2. The *request target*, usually a URL, or the absolute path of the protocol, port, and domain are usually characterized by the request context. The format of this request target varies between different HTTP methods. It can be

    - An absolute path, ultimately followed by a '`?`' and query string. This is the most common form, known as the *origin form*, and is used with `GET`, `POST`, `HEAD`, and `OPTIONS` methods.
        - `POST / HTTP/1.1`
        - `GET /background.png HTTP/1.0`
        - `HEAD /test.html?query=alibaba HTTP/1.1`
        - `OPTIONS /anypage.html HTTP/1.0`

- A complete URL, known as the *absolute form*, is mostly used with `GET` when connected to a proxy:
  `GET https://developer.mozilla.org/en-US/docs/Web/HTTP/Messages HTTP/1.1`

- The authority component of a URL, consisting of the domain name and optionally the port (prefixed by a ':'), is called the *authority form*. It is only used with `CONNECT` when setting up an HTTP tunnel.
  `CONNECT developer.mozilla.org:80 HTTP/1.1`

- The *asterisk form*, a simple asterisk ('*') is used with `OPTIONS`, representing the server as a whole. `OPTIONS * HTTP/1.1`

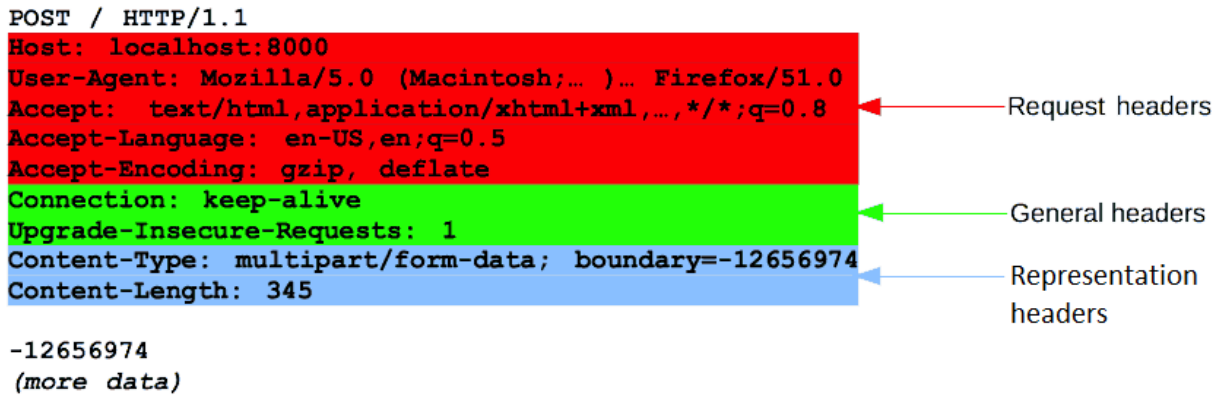
3. The *HTTP version*, which defines the structure of the remaining message, acting as an indicator of the expected version to use for the response.

Headers

HTTP headers from a request follow the same basic structure of an HTTP header: a case-insensitive string followed by a colon (' : ') and a value whose structure depends upon the header. The whole header, including the value, consists of one single line, which can be quite long.

Many different headers can appear in requests. They can be divided in several groups:

- General headers, like `Via`, apply to the message as a whole.
- Request headers, like `User-Agent` or `Accept`, modify the request by specifying it further (like `Accept-Language`), by giving context (like `Referer`), or by conditionally restricting it (like `If-None`).
- Representation headers like `Content-Type` that describe the original format of the message data and any encoding applied (only present if the message has a body).



Body

The final part of the request is its body. Not all requests have one: requests fetching resources, like `GET`, `HEAD`, `DELETE`, or `OPTIONS`, usually don't need one. Some requests send data to the server in order to update it: as often the case with POST requests (containing HTML form data).

Bodies can be broadly divided into two categories:

- Single-resource bodies, consisting of one single file, defined by the two headers: `Content-Type` and `Content-Length`.
- Multiple-resource bodies, consisting of a multipart body, each containing a different bit of information. This is typically associated with HTML Forms.

HTTP Responses

Status line

The start line of an HTTP response, called the status line, contains the following information:

1. The *protocol version*, usually `HTTP/1.1`.
2. A *status code*, indicating the success or failure of the request. Common status codes are `200`, `404`, or 302.
3. A *status text*. A brief, purely informational, textual description of the status code to help a human understand the HTTP message.

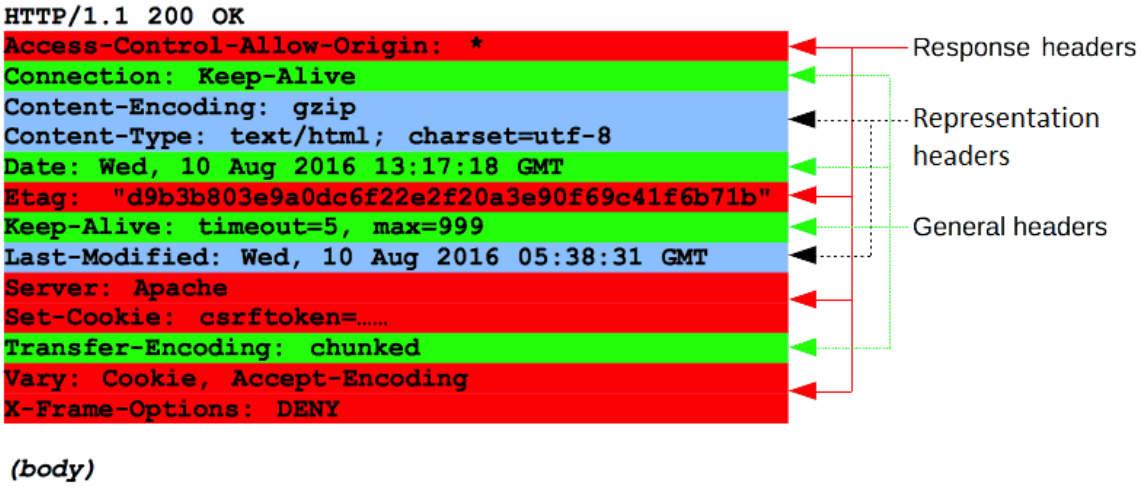
A typical status line looks like: `HTTP/1.1 404 Not Found`.

Headers

HTTP headers for responses follow the same structure as any other header: a case-insensitive string followed by a colon (' : ') and a value whose structure depends upon the type of the header. The whole header, including its value, presents as a single line.

Many different headers can appear in responses. These can be divided into several groups:

- General headers, like `Via`, apply to the whole message.
- Response headers, like `Vary` and `Accept-Ranges`, give additional information about the server which doesn't fit in the status line.
- Representation headers like `Content-Type` that describe the original format of the message data and any encoding applied (only present if the message has a body).



Body

The last part of a response is the body. Not all responses have one: responses with a status code that sufficiently answers the request without the need for corresponding payload (like `201 Created` or `204 No Content`) usually don't.

Bodies can be broadly divided into three categories:

- Single-resource bodies, consisting of a single file of known length, defined by the two headers: `Content-Type` and `Content-Length`.
- Single-resource bodies, consisting of a single file of unknown length, encoded by chunks with `Transfer-Encoding` set to `chunked`.
- Multiple-resource bodies, consisting of a multipart body, each containing a different section of information. These are relatively rare.

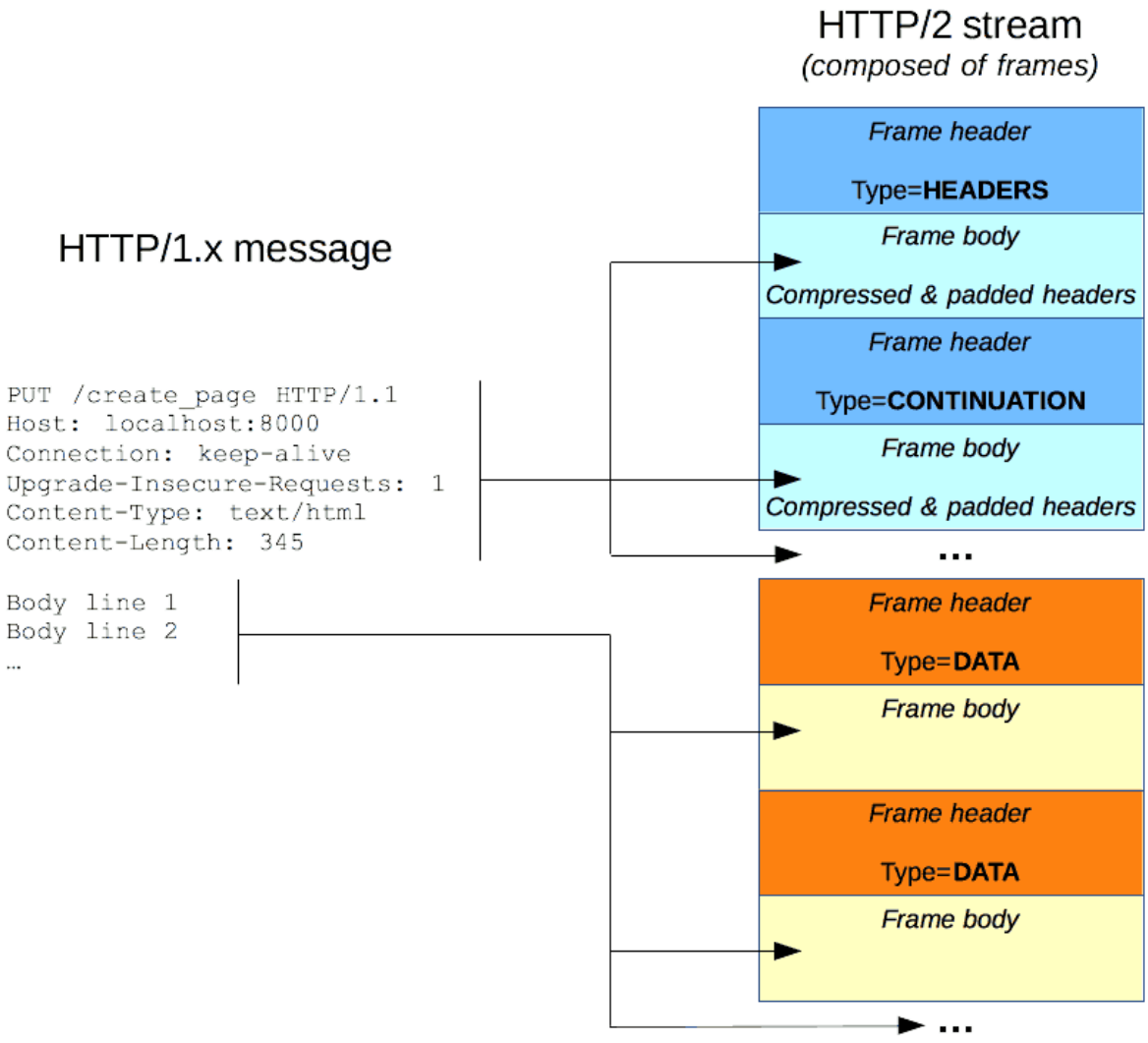
HTTP/2 Frames

HTTP/1.x messages have a few drawbacks for performance:

- Headers, unlike bodies, are uncompressed.

- Headers are often very similar from one message to the next one, yet still repeated across connections.
- No multiplexing can be done. Several connections need to open on the same server: and warm TCP connections are more efficient than cold ones.

HTTP/2 introduces an extra step: it divides HTTP/1.x messages into frames which are embedded in a stream. Data and header frames are separated, which allows header compression. Several streams can be combined together, a process called multiplexing, allowing more efficient use of underlying TCP connections.



HTTP frames are now transparent to Web developers. This is an additional step in HTTP/2, between HTTP/1.1 messages and the underlying transport protocol. No changes are needed in the APIs used by Web developers to utilize HTTP frames; when available in both the browser and the server, HTTP/2 is switched on and used.

Conclusion

HTTP messages are the key in using HTTP; their structure is simple, and they are highly extensible. The HTTP/2 framing mechanism adds a new intermediate layer between the HTTP/1.x syntax and the underlying transport protocol, without fundamentally modifying it: building upon proven mechanisms.

This lesson was adapted from MDN Web Docs: https://developer.mozilla.org/en-US/docs/Web/HTTP/Messages

A typical HTTP session

In client-server protocols, like HTTP, sessions consist of three phases:

1. The client establishes a TCP connection (or the appropriate connection if the transport layer is not TCP).

2. The client sends its request and waits for the answer.

3. The server processes the request, sending back its answer, and providing a status code and appropriate data.

As of HTTP/1.1, the connection is no longer closed after completing the third phase, and the client is now granted a further request: this means the second and third phases can now be performed any number of times.

Establishing a connection

In client-server protocols, it is the client which establishes the connection. Opening a connection in HTTP means initiating a connection in the underlying transport layer, usually this is TCP.

With TCP the default port, for an HTTP server on a computer, is port 80. Other ports can also be used, like 8000 or 8080. The URL of a page to fetch contains both the domain name, and the port number, though the latter can be omitted if it is 80.

Note: The client-server model does not allow the server to send data to the client without an explicit request for it. To work around this problem, web developers use several techniques: ping the server periodically via the `XMLHTTPRequest`, `fetch()` APIs, using the WebSockets API, or similar protocols.

Sending a client request

Once the connection is established, the user-agent can send the request (a user-agent is typically a web browser, but can be anything else, a crawler, for example). A client request consists of text directives, separated by CRLF (carriage return, followed by line feed), divided into three blocks:

1. The first line contains a request method followed by its parameters:

    - the path of the document, as an absolute URL without the protocol or domain name
    - the HTTP protocol version

2. Subsequent lines represent an HTTP header, giving the server information about what type of data is appropriate (for example, what language, what MIME types), or other data altering its behavior (for example, not sending an answer if it is already cached). These HTTP headers form a block which ends with an empty line.

3. The final block is an optional data block, which may contain further data mainly used by the POST method.

Example requests

Fetching the root page of developer.mozilla.org, (`https://developer.mozilla.org/`), and telling the server that the user-agent would prefer the page in French, if possible:

```
GET / HTTP/1.1
Host: developer.mozilla.org
Accept-Language: fr
```

Observe that final empty line, this separates the data block from the header block. As there is no Content-Length provided in an HTTP header, this data block is presented empty, marking the end of the headers, and allowing the server to process the request the moment it receives this empty line.

For example, sending the result of a form:

```
POST /contact_form.php HTTP/1.1
Host: developer.mozilla.org
Content-Length: 64
Content-Type: application/x-www-form-urlencoded

name=Joe%20User&request=Send%20me%20one%20of%20your%20catalogue
```

Request methods

HTTP defines a set of request methods indicating the desired action to be performed upon a resource. Although they can also be nouns, these request methods are sometimes referred as HTTP verbs. The most common requests are `GET` and `POST`:

- The `GET` method requests a data representation of the specified resource. Requests using `GET` should only retrieve data.
- The `POST` method sends data to a server so it may change its state. This is the method often used for HTML Forms.

Structure of a server response

After the connected agent has sent its request, the web server processes it, and ultimately returns a response. Similar to a client request, a server response is formed of text directives, separated by CRLF, though divided into three blocks:

1. The first line, the *status line*, consists of an acknowledgment of the HTTP version used, followed by a response status code (and its brief meaning in human-readable text).

2. Subsequent lines represent specific HTTP headers, giving the client information about the data sent (for example, type, data size, compression algorithm used, hints about caching). Similarly to the block of HTTP headers for a client request, these HTTP headers form a block ending with an empty line.

3. The final block is a data block, which contains the optional data.

Example responses

Successful web page response:

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 55743
Connection: keep-alive
Cache-Control: s-maxage=300, public, max-age=0
Content-Language: en-US
Date: Thu, 06 Dec 2018 17:37:18 GMT
```

```
ETag: "2e77ad1dc6ab0b53a2996dfd4653c1c3"
Server: meinheld/0.6.1
Strict-Transport-Security: max-age=63072000
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
X-XSS-Protection: 1; mode=block
Vary: Accept-Encoding,Cookie
Age: 7

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>A simple webpage</title>
</head>
<body>
  <h1>Simple HTML webpage</h1>
  <p>Hello, world!</p>
</body>
</html>
```

Notification that the requested resource has permanently moved:

```
HTTP/1.1 301 Moved Permanently
Server: Apache/2.4.37 (Red Hat)
Content-Type: text/html; charset=utf-8
Date: Thu, 06 Dec 2018 17:33:08 GMT
Location: https://developer.mozilla.org/ (this is the new link to the resource;
          it is expected that the user-agent will fetch it)
Keep-Alive: timeout=15, max=98
Accept-Ranges: bytes
Via: Moz-Cache-zlb05
Connection: Keep-Alive
Content-Length: 325 (the content contains a default page to display if the
                     user-agent is not able to follow the link)

<!DOCTYPE html>… (contains a site-customized page helping the user to find the
                  missing resource)
```

Notification that the requested resource doesn't exist:

```
HTTP/1.1 404 Not Found
Content-Type: text/html; charset=utf-8
Content-Length: 38217
Connection: keep-alive
```

```
Cache-Control: no-cache, no-store, must-revalidate, max-age=0
Content-Language: en-US
Date: Thu, 06 Dec 2018 17:35:13 GMT
Expires: Thu, 06 Dec 2018 17:35:13 GMT
Server: meinheld/0.6.1
Strict-Transport-Security: max-age=63072000
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
X-XSS-Protection: 1; mode=block
Vary: Accept-Encoding,Cookie
X-Cache: Error from cloudfront

<!DOCTYPE html>… (contains a site-customized page helping the user to find the
                  missing resource)
```

Response status codes

HTTP response status codes indicate if a specific HTTP request has been successfully completed. Responses are grouped into five classes: informational responses, successful responses, redirects, client errors, and servers errors.

1XX Informational	
100	Continue
101	Switching Protocols
102	Processing

2XX Success	
200	OK
201	Created
202	Accepted
203	Non-authoritative Information
204	No Content
205	Reset Content
206	Partial Content
207	Multi-Status
208	Already Reported
226	IM Used

3XX Redirectional	
300	Multiple Choices
301	Moved Permanently
302	Found
303	See Other
304	Not Modified
305	Use Proxy
307	Temporary Redirect
308	Permanent Redirect

4XX Client Error	
400	Bad Request
401	Unauthorized
402	Payment Required
403	Forbidden
404	Not Found
405	Method Not Allowed
406	Not Acceptable
407	Proxy Authentication Required
408	Request Timeout

4XX Client Error Continued	
409	Conflict
410	Gone
411	Length Required
412	Precondition Failed
413	Payload Too Large
414	Request-URI Too Long
415	Unsupported Media Type
416	Requested Range Not Satisfiable
417	Expectation Failed
418	I'm a teapot
421	Misdirected Request
422	Unprocessable Entity
423	Locked
424	Failed Dependency
426	Upgrade Required
428	Precondition Required
429	Too Many Requests
431	Request Header Fields Too Large
444	Connection Closed Without Response
451	Unavailable For Legal Reasons
499	Client Closed Request

5XX Server Error	
500	Internal Server Error
501	Not Implemented
502	Bad Gateway
503	Service Unavailable
504	Gateway Timeout
505	HTTP Version Not Supported
506	Variant Also Negotiates
507	Insufficient Storage
508	Loop Detected
510	Not Extended
511	Network Authentication Required
599	Network Connect Timeout Error

This lesson was adapted from MDN Web Docs: https://developer.mozilla.org/en-US/docs/Web/HTTP/Session

HTTP request methods

HTTP defines a set of **request methods** to indicate the desired action to be performed for a given resource. Although they can also be nouns, these request methods are sometimes referred to as *HTTP verbs*. Each of them implements a different semantic, but some common features are shared by a group of them: e.g. a request method can be safe, idempotent, or cacheable.

GET

The `GET` method requests a representation of the specified resource. Requests using `GET` should only retrieve data.

HEAD

The `HEAD` method asks for a response identical to a `GET` request but without the response body.

POST

The `POST` method submits an entity to the specified resource, often causing a change in state or side effects on the server.

PUT

The `PUT` method replaces all current representations of the target resource with the request payload.

DELETE

The `DELETE` method deletes the specified resource.

CONNECT

The `CONNECT` method establishes a tunnel to the server identified by the target resource.

OPTIONS

The `OPTIONS` method describes the communication options for the target resource.

TRACE

The `TRACE` method performs a message loop-back test along the path to the target resource.

PATCH

The `PATCH` method applies partial modifications to a resource.

This lesson was adapted from MDN Web Docs: https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods

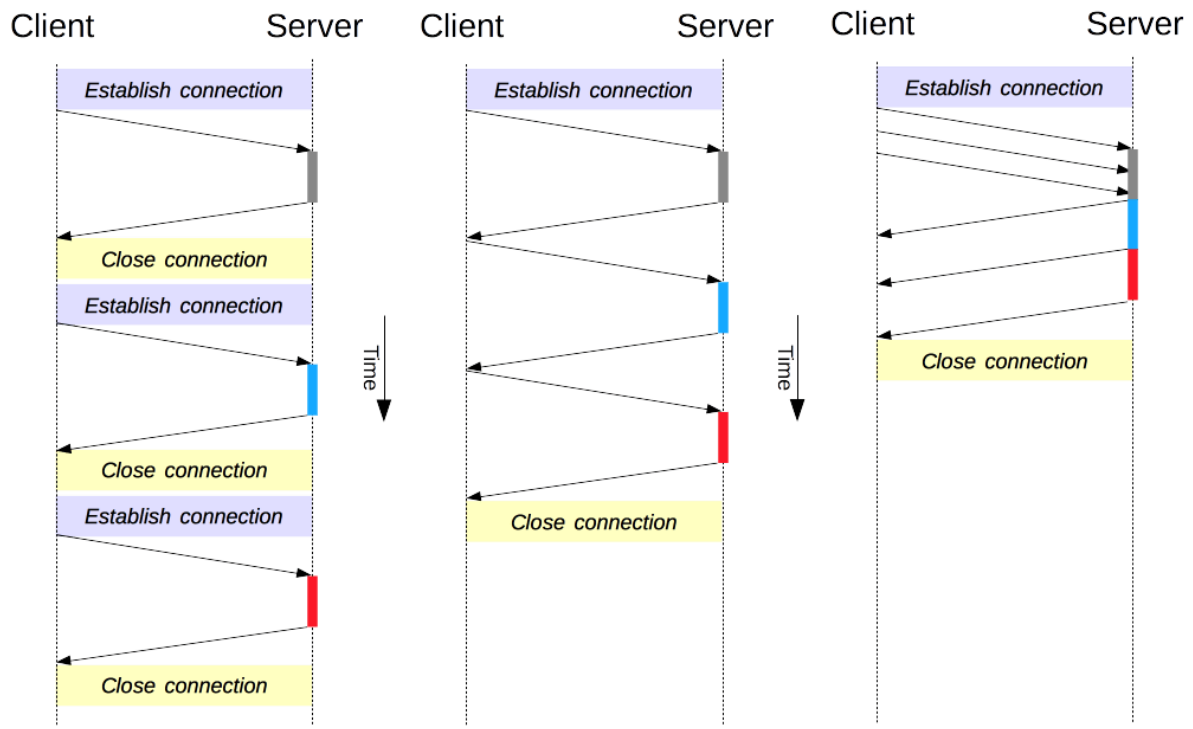
Connection management in HTTP1.x

Connection management is a key topic in HTTP: opening and maintaining connections largely impacts the performance of websites and Web applications. In HTTP/1.x, there are several models: short-lived connections, persistent connections, and HTTP pipelining.

HTTP mostly relies on TCP for its transport protocol, providing a connection between the client and the server. In its infancy, HTTP used a single model to handle such connections. These connections were short-lived: a new one created each time a request needed sending, and closed once the answer had been received.

This simple model held an innate limitation on performance: opening each TCP connection is a resource-consuming operation. Several messages must be exchanged between the client and the server. Network latency and bandwidth affect performance when a request needs sending. Modern Web pages require many requests (a dozen or more) to serve the amount of information needed, proving this earlier model inefficient.

Two newer models were created in HTTP/1.1. The persistent-connection model keeps connections opened between successive requests, reducing the time needed to open new connections. The HTTP pipelining model goes one step further, by sending several successive requests without even waiting for an answer, reducing much of the latency in the network.



Note: HTTP/2 adds additional models for connection management.

It's important to note that connection management in HTTP applies to the connection between two consecutive nodes, which is hop-by-hop and not end-to-end. The model used in connections between a client and its first proxy may differ from the model between a proxy and the destination server (or any intermediate proxies). The HTTP headers involved in defining the connection model, like `Connection` and `Keep-Alive`, are hop-by-hop headers with their values able to be changed by intermediary nodes.

A related topic is the concept of HTTP connection upgrades, wherein an HTTP/1.1 connection is upgraded to a different protocol, such as TLS/1.0, WebSocket, or even HTTP/2 in cleartext. This protocol upgrade mechanism is documented in more detail elsewhere.

Short-lived connections

The original model of HTTP, and the default one in HTTP/1.0, is short-lived connections. Each HTTP request is completed on its own connection; this means a TCP handshake happens before each HTTP request, and these are serialized.

The TCP handshake itself is time-consuming, but a TCP connection adapts to its load, becoming more efficient with more sustained (or warm) connections. Short-lived connections do not make use of this efficiency feature of TCP, and performance degrades from optimum by persisting to transmit over a new, cold connection.

This model is the default model used in HTTP/1.0 (if there is no `Connection` header, or if its value is set to `close`). In HTTP/1.1, this model is only used when the `Connection` header is sent with a value of `close`.

Note: Unless dealing with a very old system, which doesn't support a persistent connection, there is no compelling reason to use this model.

Persistent connections

Short-lived connections have two major hitches: the time taken to establish a new connection is significant, and performance of the underlying TCP connection gets better only when this connection has been in use for some time (warm connection). To ease these problems, the concept of a persistent connection has been designed, even prior to HTTP/1.1. Alternatively this may be called a keep-alive connection.

A persistent connection is one which remains open for a period of time, and can be reused for several requests, saving the need for a new TCP handshake, and utilizing TCP's performance enhancing capabilities. This connection will not stay open forever: idle connections are closed after some time (a server may use the Keep-Alive header to specify a minimum time the connection should be kept open).

Persistent connections also have drawbacks; even when idling they consume server resources, and under heavy load, DoS attacks can be conducted. In such cases, using non-persistent connections, which are closed as soon as they are idle, can provide better performance.

HTTP/1.0 connections are not persistent by default. Setting `Connection` to anything other than `close`, usually `retry-after`, will make them persistent.

In HTTP/1.1, persistence is the default, and the header is no longer needed (but it is often added as a defensive measure against cases requiring a fallback to HTTP/1.0).

HTTP pipelining

Note: HTTP pipelining is not activated by default in modern browsers:

- Buggy proxies are still common and these lead to strange and erratic behaviors that Web developers cannot foresee and diagnose easily.
- Pipelining is complex to implement correctly: the size of the resource being transferred, the effective RTT that will be used, as well as the effective bandwidth, have a direct incidence on the improvement provided by the pipeline. Without knowing these, important messages may be delayed behind unimportant ones. The notion of important even evolves during page layout! HTTP pipelining therefore brings a marginal improvement in most cases only.
- Pipelining is subject to the HOL problem.

For these reasons, pipelining has been superseded by a better algorithm, *multiplexing*, that is used by HTTP/2.

By default, HTTP requests are issued sequentially. The next request is only issued once the response to the current request has been received. As they are affected by network latencies and bandwidth limitations, this can result in significant delay before the next request is seen by the server.

Pipelining is the process to send successive requests, over the same persistent connection, without waiting for the answer. This avoids latency of the connection. Theoretically, performance could also be improved if two HTTP requests were to be packed into the same TCP message. The typical MSS (Maximum Segment Size), is big enough to contain several simple requests, although the demand in size of HTTP requests continues to grow.

Not all types of HTTP requests can be pipelined: only idempotent methods, that is `GET`, `HEAD`, `PUT` and `DELETE`, can be replayed safely. Should a failure happen, the pipeline content can be repeated.

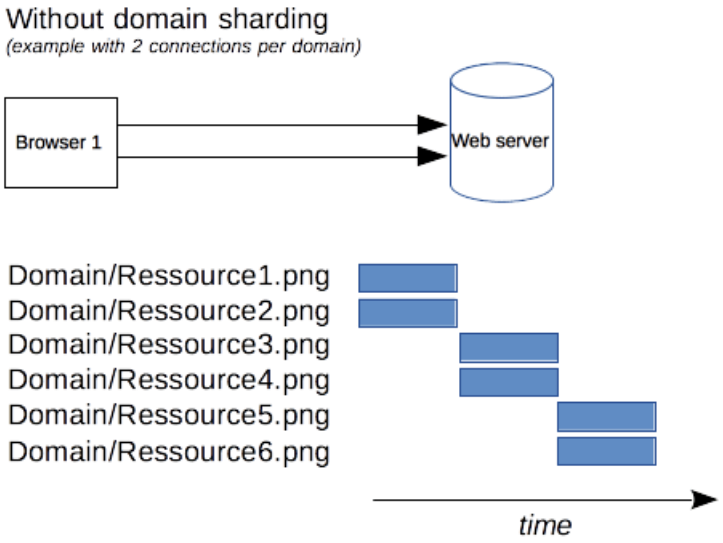
Today, every HTTP/1.1-compliant proxy and server should support pipelining, though many have limitations in practice: a significant reason no modern browser activates this feature by default.

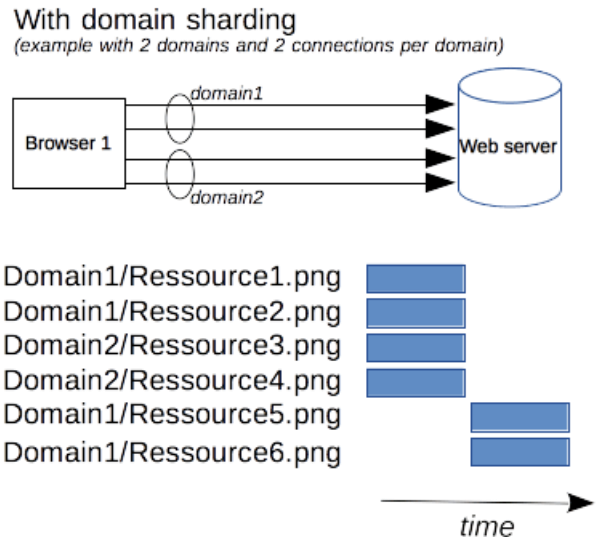
Domain sharding

Note: Unless you have a very specific immediate need, don't use this deprecated technique; switch to HTTP/2 instead. In HTTP/2, domain sharding is no longer useful: the HTTP/2 connection is able to handle parallel unprioritized requests very well. Domain sharding is even detrimental to performance. Most HTTP/2 implementations use a technique called connection coalescing to revert eventual domain sharding.

As an HTTP/1.x connection is serializing requests, even without any ordering, it can't be optimal without large enough available bandwidth. As a solution, browsers open several connections to each domain, sending parallel requests. Default was once 2 to 3 connections, but this has now increased to a more common use of 6 parallel connections. There is a risk of triggering DoS protection on the server side if attempting more than this number.

If the server wishes a faster website or application response, it is possible for the server to force the opening of more connections. For example, instead of having all resources on the same domain, say `www.example.com`, it could split over several domains, `www1.example.com`, `www2.example.com`, `www3.example.com`. Each of these domains resolves to the same server, and the Web browser will open 6 connections to each (in our example, boosting the connections to 18). This technique is called domain sharding.





Conclusion

Improved connection management allows considerable boosting of performance in HTTP. With HTTP/1.1 or HTTP/1.0, using a persistent connection – at least until it becomes idle – leads to the best performance. However, the failure of pipelining has lead to designing superior connection management models, which have been incorporated into HTTP/2.

This lesson was adapted from MDN Web Docs:
https://developer.mozilla.org/en-US/docs/Web/HTTP/Connection_management_in_HTTP_1.x

Protocol upgrade mechanism

The HTTP/1.1 protocol provides a special mechanism that can be used to upgrade an already established connection to a different protocol, using the Upgrade header field.

This mechanism is optional; it cannot be used to insist on a protocol change. Implementations can choose not to take advantage of an upgrade even if they support the new protocol, and in practice, this mechanism is used mostly to bootstrap a WebSockets connection.

Note also that HTTP/2 explicitly disallows the use of this mechanism; it is specific to HTTP/1.1.

Upgrading HTTP/1.1 Connections

The Upgrade header field is used by clients to invite the server to switch to one of the listed protocols, in descending preference order.

Because Upgrade is a hop-by-hop header, it also needs to be listed in the Connection header field. This means that a typical request that includes Upgrade would look something like this:

```
GET /index.html HTTP/1.1
Host: www.example.com
Connection: upgrade
Upgrade: example/1, foo/2
```

Other headers may be required depending on the requested protocol; for example, WebSocket upgrades allow additional headers to configure details about the WebSocket connection as well as to offer a degree of security in opening the connection.

If the server decides to upgrade the connection, it sends back a `101 Switching Protocols` response status with an Upgrade header that specifies the protocol(s) being switched to. If it does not (or cannot) upgrade the connection, it ignores the `Upgrade` header and sends back a regular response (for example, a `200 OK`).

Right after sending the 101 status code, the server can begin speaking the new protocol, performing any additional protocol-specific handshakes as necessary. Effectively, the connection becomes a two-way pipe as soon as the upgraded response is complete, and the request that initiated the upgrade can be completed over the new protocol.

Common uses for this mechanism

Here we look at the most common use cases for the Upgrade header.

Upgrading to a WebSocket connection

By far, the most common use case for upgrading an HTTP connection is to use WebSockets, which are always implemented by upgrading an HTTP or HTTPS connection. Keep in mind that if you're opening a new connection using the WebSocket API, or any library that does WebSockets, most or all of this is done for you. For example, opening a WebSocket connection is as simple as:

```
webSocket = new WebSocket("ws://destination.server.ext", "optionalProtocol");
```

The `WebSocket()` constructor does all the work of creating an initial HTTP/1.1 connection then handling the handshaking and upgrade process for you.

Note: You can also use the "`wss://`" URL scheme to open a secure WebSocket connection.

If you need to create a WebSocket connection from scratch, you'll have to handle the handshaking process yourself. After creating the initial HTTP/1.1 session, you need to request the upgrade by adding to a standard request the `Upgrade` and `Connection` headers, as follows:

```
Connection: Upgrade
Upgrade: websocket
```

WebSocket-specific headers

The following headers are involved in the WebSocket upgrade process. Other than the `Upgrade` and `Connection` headers, the rest are generally optional or handled for you by the browser and server when they're talking to each other.

`Sec-WebSocket-Extensions`

Specifies one or more protocol-level WebSocket extensions to ask the server to use. Using more than one `Sec-WebSocket-Extension` header in a request is permitted; the result is the same as if you included all of the listed extensions in one such header.

```
Sec-WebSocket-Extensions: extensions
```

`extensions`

A comma-separated list of extensions to request (or agree to support). These should be selected from the IANA WebSocket Extension Name Registry. Extensions which take parameters do so by using semicolon delineation.

For example:

```
Sec-WebSocket-Extensions: superspeed, colormode; depth=16
```

`Sec-WebSocket-Key`

Provides information to the server which is needed in order to confirm that the client is entitled to request an upgrade to WebSocket. This header can be used when insecure (HTTP) clients wish to upgrade, in order to offer some degree of protection against abuse. The value of the key is computed using an algorithm defined in the WebSocket specification, so this does not provide security. Instead, it helps to prevent non-WebSocket clients from inadvertently, or through misuse, requesting a WebSocket connection. In essence, then, this key confirms that "Yes, I really mean to open a WebSocket connection."

This header is automatically added by clients that choose to use it; it cannot be added using the `XMLHttpRequest.setRequestHeader()` method.

```
Sec-WebSocket-Key: key
```

`key`

The key for this request to upgrade. The client adds this if it wishes to do so, and the server will include in the response a key of its own, which the client will validate before delivering the upgrade response to you.

The server's response's Sec-WebSocket-Accept header will have a value computed based upon the specified key.

Sec-WebSocket-Protocol

The `Sec-WebSocket-Protocol` header specifies one or more WebSocket protocols that you wish to use, in order of preference. The first one that is supported by the server will be selected and returned by the server in a `Sec-WebSocket-Protocol` header included in the response. You can use this more than once in the header, as well; the result is the same as if you used a comma-delineated list of subprotocol identifiers in a single header.

```
Sec-WebSocket-Protocol: subprotocols
```

`Subprotocols`

A comma-separated list of subprotocol names, in the order of preference. The subprotocols may be selected from the IANA WebSocket Subprotocol Name Registry or may be a custom name jointly understood by the client and the server.

Sec-WebSocket-Version

REQUEST HEADER

Specifies the WebSocket protocol version the client wishes to use, so the server can confirm whether or not that version is supported on its end.

```
Sec-WebSocket-Version: version
```

`version`

The WebSocket protocol version the client wishes to use when communicating with the server. This number should be the most recent version possible listed in the IANA WebSocket Version Number Registry. The most recent final version of the WebSocket protocol is version 13.

RESPONSE HEADER

If the server can't communicate using the specified version of the WebSocket protocol, it will respond with an error (such as 426 Upgrade Required) that includes in its headers a `Sec-WebSocket-Version` header with a comma-separated list of the supported protocol versions. If the server does support the requested protocol version, no `Sec-WebSocket-Version` header is included in the response.

```
Sec-WebSocket-Version: supportedVersions
```

`supportedVersions`

A comma-delineated list of the WebSocket protocol versions supported by the server.

Response-only headers

The response from the server may include these.

`Sec-WebSocket-Accept`

Included in the response message from the server during the opening handshake process when the server is willing to initiate a WebSocket connection. It will appear no more than once in the response headers.

```
Sec-WebSocket-Accept: hash
```

`hash`

If a `Sec-WebSocket-Key` header was provided, the value of this header is computed by taking the value of the key, concatenating the string "258EAFA5-E914-47DA-95CA-C5AB0DC85B11" to it, taking the SHA-1 hash of that concatenated string, resulting in a 20-byte value. That value is then base64 encoded to obtain the value of this property.

This lesson was adapted from MDN Web Docs: https://developer.mozilla.org/en-US/docs/Web/HTTP/Protocol_upgrade_mechanism

Content Security Policy (CSP)

Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross-Site Scripting (XSS) and data injection attacks. These attacks are used for everything from data theft to site defacement, to malware distribution.

CSP is designed to be fully backward compatible (except CSP version 2 where there are some explicitly-mentioned inconsistencies in backward compatibility; more details here section 1.1). Browsers that don't support it still work with servers that implement it, and vice versa: browsers that don't support CSP ignore it, functioning as usual, defaulting to the standard same-origin policy for web content. If the site doesn't offer the CSP header, browsers likewise use the standard same-origin policy.

To enable CSP, you need to configure your web server to return the Content-Security-Policy HTTP header.

Alternatively, the `<meta>` element can be used to configure a policy, for example:

```
<meta
  http-equiv="Content-Security-Policy"
  content="default-src 'self'; img-src https://*; child-src 'none';" />
```

Note: Some features, such as sending CSP violation reports, are only available when using the HTTP headers.

Threats

Mitigating cross-site scripting

A primary goal of CSP is to mitigate and report XSS attacks. XSS attacks exploit the browser's trust in the content received from the server. Malicious scripts are executed by the victim's browser because the browser trusts the source of the content, even when it's not coming from where it seems to be coming from.

CSP makes it possible for server administrators to reduce or eliminate the vectors by which XSS can occur by specifying the domains that the browser should consider to be valid sources of executable scripts. A CSP compatible browser will then only execute scripts loaded in source files received from those allowed domains, ignoring all other scripts (including inline scripts and event-handling HTML attributes).

As an ultimate form of protection, sites that want to never allow scripts to be executed can opt to globally disallow script execution.

Mitigating packet sniffing attacks

In addition to restricting the domains from which content can be loaded, the server can specify which protocols are allowed to be used; for example (and ideally, from a security standpoint), a server can specify that all content must be loaded using HTTPS. A complete data transmission security strategy includes not only enforcing HTTPS for data transfer, but also marking all cookies with the `secure` attribute and providing automatic redirects from HTTP pages to their HTTPS counterparts. Sites may also use the `Strict-Transport-Security` HTTP header to ensure that browsers connect to them only over an encrypted channel.

Using CSP

Configuring Content Security Policy involves adding the Content-Security-Policy HTTP header to a web page and giving it values to control what resources the user agent is allowed to load for that page. For example, a page that uploads and displays images could allow images from anywhere, but restrict a form action to a specific endpoint. A properly designed Content Security Policy helps protect a page against a cross-site scripting attack. This article explains how to construct such headers properly, and provides examples.

Specifying your policy

You can use the Content-Security-Policy HTTP header to specify your policy, like this:

```
Content-Security-Policy: policy
```

The policy is a string containing the policy directives describing your Content Security Policy.

Writing a policy

A policy is described using a series of policy directives, each of which describes the policy for a certain resource type or policy area. Your policy should include a `default-src` policy directive, which is a fallback for other resource types when they don't have policies of their own (for a complete list, see the description of the `default-src` directive). A policy needs to include a `default-src` or `script-src` directive to prevent inline scripts from running, as well as blocking the use of `eval()`. A policy needs to include a `default-src` or `style-src` directive to restrict inline styles from being applied from a `<style>` element or a `style` attribute. There are specific directives for a wide variety of types of items, so that each type can have its own policy, including fonts, frames, images, audio and video media, scripts, and workers.

For a complete list of policy directives, see the reference page for the Content-Security-Policy header.

Examples: Common use cases

This section provides examples of some common security policy scenarios.

Example 1

A website administrator wants all content to come from the site's own origin (this excludes subdomains.)

```
Content-Security-Policy: default-src 'self'
```

Example 2

A website administrator wants to allow content from a trusted domain and all its subdomains (it doesn't have to be the same domain that the CSP is set on.)

```
Content-Security-Policy: default-src 'self' example.com *.example.com
```

Example 3

A website administrator wants to allow users of a web application to include images from any origin in their own content, but to restrict audio or video media to trusted providers, and all scripts only to a specific server that hosts trusted code.

```
Content-Security-Policy: default-src 'self'; img-src *; media-src example.org
example.net; script-src userscripts.example.com
```

Here, by default, content is only permitted from the document's origin, with the following exceptions:

- Images may load from anywhere (note the "*" wildcard).
- Media is only allowed from example.org and example.net (and not from subdomains of those sites).
- Executable script is only allowed from userscripts.example.com.

Example 4

A website administrator for an online banking site wants to ensure that all its content is loaded using TLS, in order to prevent attackers from eavesdropping on requests.

```
Content-Security-Policy: default-src https://onlinebanking.example.com
```

The server permits access only to documents being loaded specifically over HTTPS through the single origin onlinebanking.example.com.

Example 5

A website administrator of a web mail site wants to allow HTML in email, as well as images loaded from anywhere, but not JavaScript or other potentially dangerous content.

```
Content-Security-Policy: default-src 'self' *.example.com; img-src *
```

Note that this example doesn't specify a `script-src`; with the example CSP, this site uses the setting specified by the `default-src` directive, which means that scripts can be loaded only from the originating server.

Testing your policy

To ease deployment, CSP can be deployed in report-only mode. The policy is not enforced, but any violations are reported to a provided URI. Additionally, a report-only header can be used to test a future revision to a policy without actually deploying it.

You can use the `Content-Security-Policy-Report-Only` HTTP header to specify your policy, like this:

```
Content-Security-Policy-Report-Only: policy
```

If both a `Content-Security-Policy-Report-Only` header and a `Content-Security-Policy` header are present in the same response, both policies are honored. The policy specified in `Content-Security-Policy` headers is enforced while the `Content-Security-Policy-Report-Only` policy generates reports but is not enforced.

Enabling reporting

By default, violation reports aren't sent. To enable violation reporting, you need to specify the report-to policy directive, providing at least one URI to which to deliver the reports:

```
Content-Security-Policy: default-src 'self'; report-to
http://reportcollector.example.com/collector.cgi
```

Then you need to set up your server to receive the reports; it can store or process them in whatever manner you determine is appropriate.

Violation report syntax

The report JSON object is sent with an `application/csp-report Content-Type` and contains the following data:

`blocked-uri`

The URI of the resource that was blocked from loading by the Content Security Policy. If the blocked URI is from a different origin than the `document-uri`, then the blocked URI is truncated to contain just the scheme, host, and port.

`disposition`

Either "`enforce`" or "`report`" depending on whether the `Content-Security-Policy-Report-Only` header or the `Content-Security-Policy` header is used.

`document-uri`

The URI of the document in which the violation occurred.

`effective-directive`

The directive whose enforcement caused the violation. Some browsers may provide different values, such as Chrome providing `style-src-elem/style-src-attr`, even when the actually enforced directive was `style-src`.

`original-policy`

The original policy as specified by the `Content-Security-Policy` HTTP header.

`referrer`

The referrer of the document in which the violation occurred.

`script-sample`

The first 40 characters of the inline script, event handler, or style that caused the violation. Only applicable to `script-src*` and `style-src*` violations, when they contain the `'report-sample'`

`status-code`

The HTTP status code of the resource on which the global object was instantiated.

`violated-directive`

The directive whose enforcement caused the violation. The `violated-directive` is a historic name for the `effective-directive` field and contains the same value.

Sample violation report

Let's consider a page located at `http://example.com/signup.html`. It uses the following policy, disallowing everything but stylesheets from `cdn.example.com`.

```
Content-Security-Policy: default-src 'none'; style-src cdn.example.com;
report-to /_/csp-reports
```

The HTML of `signup.html` looks like this:

```
<!doctype html>
<html lang="en-US">
  <head>
    <meta charset="UTF-8" />
    <title>Sign Up</title>
    <link rel="stylesheet" href="css/style.css" />
  </head>
  <body>
    Here be content.
  </body>
</html>
```

Can you spot the mistake? Stylesheets are allowed to be loaded only from `cdn.example.com`, yet the website tries to load one from its own origin (`http://example.com`). A browser capable of enforcing CSP would send the following violation report as a POST request to `http://example.com/_/csp-reports`, when the document is visited:

```
{
  "csp-report": {
    "blocked-uri": "http://example.com/css/style.css",
```

```
      "disposition": "report",
      "document-uri": "http://example.com/signup.html",
      "effective-directive": "style-src-elem",
      "original-policy": "default-src 'none'; style-src cdn.example.com;
         report-to  /_/csp-reports",
      "referrer": "",
      "status-code": 200,
      "violated-directive": "style-src-elem"
    }
  }
```

As you can see, the report includes the full path to the violating resource in `blocked-uri`. This is not always the case. For example, if the `signup.html` attempted to load CSS from `http://anothercdn.example.com/stylesheet.css`, the browser would not include the full path, but only the origin (`http://anothercdn.example.com`). The CSP specification gives an explanation of this odd behavior. In summary, this is done to prevent leaking sensitive information about cross-origin resources.

This lesson was adapted from MDN Web Docs: https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP

Strict-Transport-Security

The HTTP **`Strict-Transport-Security`** response header (often abbreviated as HSTS) informs browsers that the site should only be accessed using HTTPS, and that any future attempts to access it using HTTP should automatically be converted to HTTPS.

Note: This is more secure than simply configuring a HTTP to HTTPS (301) redirect on your server, where the initial HTTP connection is still vulnerable to a man-in-the-middle attack.

Header type	Response header
Forbidden header name	no

Syntax

```
Strict-Transport-Security: max-age=<expire-time>
Strict-Transport-Security: max-age=<expire-time>; includeSubDomains
Strict-Transport-Security: max-age=<expire-time>; includeSubDomains; preload
```

Directives

`max-age=<expire-time>`

The time, in seconds, that the browser should remember that a site is only to be accessed using HTTPS.

`includeSubDomains Optional`

If this optional parameter is specified, this rule applies to all of the site's subdomains as well.

`preload Optional Non-standard`

See Preloading Strict Transport Security for details. When using `preload`, the `max-age` directive must be at least `31536000` (1 year), and the `includeSubDomains` directive must be present. Not part of the specification.

Description

If a website accepts a connection through HTTP and redirects to HTTPS, visitors may initially communicate with the non-encrypted version of the site before being redirected, if, for example, the visitor types `http://www.foo.com/` or even just foo.com. This creates an opportunity for a man-in-the-middle attack. The redirect could be exploited to direct visitors to a malicious site instead of the secure version of the original site.

The HTTP Strict Transport Security header informs the browser that it should never load a site using HTTP and should automatically convert all attempts to access the site using HTTP to HTTPS requests instead.

Note: The `Strict-Transport-Security` header is *ignored* by the browser when your site has only been accessed using HTTP. Once your site is accessed over HTTPS with no certificate errors, the browser knows your site is HTTPS capable and will honor the `Strict-Transport-Security` header. Browsers do this as attackers may intercept HTTP connections to the site and inject or remove the header.

An example scenario

You log into a free Wi-Fi access point at an airport and start surfing the web, visiting your online banking service to check your balance and pay a couple of bills. Unfortunately, the access point you're using is actually a hacker's laptop, and they're intercepting your original HTTP request and redirecting you to a clone of your bank's site instead of the real thing. Now your private data is exposed to the hacker. Strict Transport Security resolves this problem; as long as you've accessed your bank's website once using HTTPS, and the bank's website uses Strict Transport Security, your browser will know to automatically use only HTTPS, which prevents hackers from performing this sort of man-in-the-middle attack.

How the browser handles it

The first time your site is accessed using HTTPS and it returns the `Strict-Transport-Security` header, the browser records this information, so that future attempts to load the site using HTTP will automatically use HTTPS instead.

When the expiration time specified by the `Strict-Transport-Security` header elapses, the next attempt to load the site via HTTP will proceed as normal instead of automatically using HTTPS.
Whenever the `Strict-Transport-Security` header is delivered to the browser, it will update the expiration time for that site, so sites can refresh this information and prevent the timeout from

expiring. Should it be necessary to disable `Strict Transport Security`, setting the `max-age` to 0 (over an https connection) will immediately expire `the Strict-Transport-Security` header, allowing access via http.

Preloading Strict Transport Security

Google maintains an HSTS preload service. By following the guidelines and successfully submitting your domain, you can ensure that browsers will connect to your domain only via secure connections. While the service is hosted by Google, all browsers are using this preload list. However, it is not part of the HSTS specification and should not be treated as official.

- Information regarding the HSTS preload list in Chrome: https://www.chromium.org/hsts
- Consultation of the Firefox HSTS preload list: nsSTSPreloadList.inc

Examples

All present and future subdomains will be HTTPS for a `max-age` of 1 year. This blocks access to pages or subdomains that can only be served over HTTP.

```
Strict-Transport-Security: max-age=31536000; includeSubDomains
```

Although a `max-age` of 1 year is acceptable for a domain, two years is the recommended value as explained on https://hstspreload.org.

In the following example, max-age is set to 2 years, and is suffixed with preload, which is necessary for inclusion in all major web browsers' HSTS preload lists, like Chromium, Edge, and Firefox.

```
Strict-Transport-Security: max-age=63072000; includeSubDomains; preload
```

Specifications

HTTP Strict Transport Security (HSTS) # section-6.1

This lesson is adapted from MDN web Docs: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Strict-Transport-Security

X-Content-Type-Options

The **`X-Content-Type-Options`** response HTTP header is a marker used by the server to indicate that the MIME types advertised in the `Content-Type` headers should be followed and not be changed. The header allows you to avoid MIME type sniffing by saying that the MIME types are deliberately configured.

This header was introduced by Microsoft in IE 8 as a way for webmasters to block content sniffing that was happening and could transform non-executable MIME types into executable MIME types. Since then, other browsers have introduced it, even if their MIME sniffing algorithms were less aggressive.

Starting with Firefox 72, top-level documents also avoid MIME sniffing (if `Content-type` is provided). This can cause HTML web pages to be downloaded instead of being rendered when they are served with a MIME type other than text/html. Make sure to set both headers correctly. Site security testers usually expect this header to be set.

Note: `X-Content-Type-Options` only apply request-blocking due to `nosniff` for request destinations of `"script"` and `"style"`. However, it also enables Cross-Origin Read Blocking (CORB) protection for HTML, TXT, JSON and XML files (excluding SVG `image/svg+xml`).

Header type	Response header
Forbidden header name	no

Syntax

```
X-Content-Type-Options: nosniff
```

Directives

`nosniff`

Blocks a request if the request destination is of type style and the MIME type is not `text/css`, or of type `script` and the MIME type is not a JavaScript MIME type.

Specifications

Fetch Standard # x-content-type-options-header

This lesson is adapted from MDN web Docs: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Content-Type-Options

X-Frame-Options

The **`X-Frame-Options`** HTTP response header can be used to indicate whether or not a browser should be allowed to render a page in a `<frame>`, `<iframe>`, `<embed>` or `<object>`. Sites can use this to avoid click-jacking attacks, by ensuring that their content is not embedded into other sites.

The added security is provided only if the user accessing the document is using a browser that supports `X-Frame-Options`.

Note: The `Content-Security-Policy` HTTP header has a `frame-ancestors` directive which obsoletes this header for supporting browsers.

Header type	Response header
Forbidden header name	no

Syntax

There are two possible directives for `X-Frame-Options`:

```
X-Frame-Options: DENY
X-Frame-Options: SAMEORIGIN
```

Directives

If you specify `DENY`, not only will the browser attempt to load the page in a frame fail when loaded from other sites, attempts to do so will fail when loaded from the same site. On the other hand, if you specify `SAMEORIGIN`, you can still use the page in a frame as long as the site including it in a frame is the same as the one serving the page.

`DENY`

The page cannot be displayed in a frame, regardless of the site attempting to do so.

`SAMEORIGIN`

The page can only be displayed if all ancestor frames are same origin to the page itself.

`ALLOW-FROM` (Deprecated)

This is an obsolete directive that no longer works in modern browsers. (Using it will give the same behavior as omitting the header.) Don't use it. The Content-Security-Policy HTTP header has a frame-ancestors directive which you can use instead.

Examples

Note: Setting X-Frame-Options inside the `<meta>` element is useless! For instance, `<meta http-equiv="X-Frame-Options" content="deny">` has no effect. Do not use it! `X-Frame-Options` works only by setting through the HTTP header, as in the examples below.

Configuring Apache

To configure Apache to send the `X-Frame-Options` header for all pages, add this to your site's configuration:

```
Header always set X-Frame-Options "SAMEORIGIN"
```

To configure Apache to set the `X-Frame-Options` DENY, add this to your site's configuration:

```
Header set X-Frame-Options "DENY"
```

Configuring Nginx

To configure Nginx to send the `X-Frame-Options` header, add this either to your http, server or location configuration:

```
add_header X-Frame-Options SAMEORIGIN always;
```

Configuring IIS

To configure IIS to send the `X-Frame-Options` header, add this to your site's `Web.config` file:

```
<system.webServer>
  …
  <httpProtocol>
    <customHeaders>
      <add name="X-Frame-Options" value="SAMEORIGIN" />
    </customHeaders>
  </httpProtocol>
  …
</system.webServer>
```

Or see Microsoft support article on setting this configuration using the IIS Manager user interface.

Configuring HAProxy

To configure HAProxy to send the `X-Frame-Options` header, add this to your front-end, listen, or backend configuration:

```
rspadd X-Frame-Options:\ SAMEORIGIN
```

Alternatively, in newer versions:

```
http-response set-header X-Frame-Options SAMEORIGIN
```

Configuring Express

To configure Express to send the `X-Frame-Options` header, you can use helmet which uses frameguard to set the header. Add this to your server configuration:

```
const helmet = require("helmet");
const app = express();
app.use(helmet.frameguard({ action: "SAMEORIGIN" }));
```

Alternatively, you can use frameguard directly:

```
const frameguard = require("frameguard");
app.use(frameguard({ action: "SAMEORIGIN" }));
```

Specifications

HTML Standard
the-x-frame-options-header

This lesson is adapted from MDN web Docs: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options

X-XSS-Protection

The HTTP **`X-XSS-Protection`** response header is a feature of Internet Explorer, Chrome and Safari that stops pages from loading when they detect reflected cross-site scripting (XSS) attacks. These protections are largely unnecessary in modern browsers when sites implement a strong `Content-Security-Policy` that disables the use of inline JavaScript (`'unsafe-inline'`).

Warning: Even though this feature can protect users of older web browsers that don't yet support CSP, in some cases, **XSS protection can create XSS vulnerabilities** in otherwise safe websites. See the section below for more information.

Note:

- Chrome has removed their XSS Auditor
- Firefox has not, and will not implement X-XSS-Protection
- Edge has retired their XSS filter

This means that if you do not need to support legacy browsers, it is recommended that you use Content-Security-Policy without allowing `unsafe-inline` scripts instead.

Header type	Response header
Forbidden header name	no

Syntax

```
X-XSS-Protection: 0
X-XSS-Protection: 1
X-XSS-Protection: 1; mode=block
X-XSS-Protection: 1; report=<reporting-uri>
```

`0`

Disables XSS filtering.

`1`

Enables XSS filtering (usually default in browsers). If a cross-site scripting attack is detected, the browser will sanitize the page (remove the unsafe parts).

`1; mode=block`

Enables XSS filtering. Rather than sanitizing the page, the browser will prevent rendering of the page if an attack is detected.

`1; report=<reporting-URI>` (Chromium only)

Enables XSS filtering. If a cross-site scripting attack is detected, the browser will sanitize the page and report the violation. This uses the functionality of the CSP `report-uri` directive to send a report.

Vulnerabilities caused by XSS filtering

Consider the following excerpt of HTML code for a webpage:

```
<script>
  var productionMode = true;
</script>
<!-- [...] -->
<script>
  if (!window.productionMode) {
    // Some vulnerable debug code
  }
</script>
```

This code is completely safe if the browser doesn't perform XSS filtering. However, if it does and the search query is `?something=%3Cscript%3Evar%20productionMode%20%3D%20true%3B%3C%2Fscript%3E`, the browser might execute the scripts in the page ignoring `<script>var productionMode = true;</script>` (thinking the server included it in the response because it was in the URI), causing `window.productionMode` to be evaluated to `undefined` and executing the unsafe debug code.

Setting the `X-XSS-Protection` header to either `0` or `1`; `mode=block` prevents vulnerabilities like the one described above. The former would make the browser run all scripts and the latter would prevent the page from being processed at all (though this approach might be vulnerable to side-channel attacks if the website is embeddable in an `<iframe>`).

Examples

Block pages from loading when they detect reflected XSS attacks:

HTTP:

```
X-XSS-Protection: 1; mode=block
```

PHP:

```
header("X-XSS-Protection: 1; mode=block");
```

Apache (.htaccess):

```
<IfModule mod_headers.c>
  Header set X-XSS-Protection "1; mode=block"
</IfModule>
```

Nginx:

```
add_header "X-XSS-Protection" "1; mode=block";
```

Specifications

Not part of any specifications or drafts.

This lesson is adapted from MDN web Docs: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options

HTTP access control (CORS)

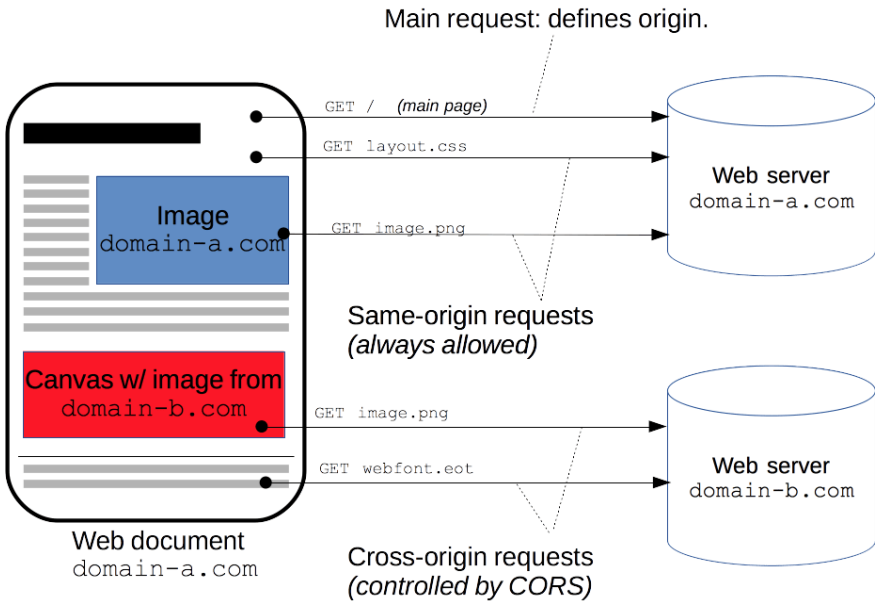
Cross-Origin Resource Sharing (CORS) is like a security rule for web browsers. It controls when a website from one place can ask another website for information.

Here's how it plays out:

1. **Asking Permission:** When one website wants to grab something (like an image or data) from another website, the second website gets to decide if that's okay. CORS helps with this decision.
2. **Testing the Waters:** Before actually fetching the data, the browser sends a little message to the second website, sort of like saying, "Hey, can I get this data from you?" This is the "preflight" request.
3. **Laying Out the Plan:** The browser also explains to the second website how it's planning to get the data. It's like saying, "I'm going to ask nicely using these methods and rules."
4. **Getting Permission:** If the second website agrees, it sends a special signal back to the browser (these are the CORS headers) saying, "Sure, you can go ahead and fetch that data."
5. **Safety First:** If the special signal isn't there, browsers are cautious and won't allow websites to grab data from other places without permission. This adds a layer of security to ensure that websites can't just take data from anywhere they want.

Picture this: If one website wants to show an image from another site, CORS ensures there's a clear understanding between the sites and the browser that everything is safe and authorized. This way, we prevent unauthorized data access and maintain a safer internet environment.

For security reasons, browsers restrict cross-origin HTTP requests initiated from scripts. For example, `XMLHttpRequest` and the Fetch API follow the same-origin policy. This means that a web application using those APIs can only request resources from the same origin the application was loaded from unless the response from other origins includes the right CORS headers.



The CORS mechanism supports secure cross-origin requests and data transfers between browsers and servers. Modern browsers use CORS in APIs such as `XMLHttpRequest` or Fetch to mitigate the risks of cross-origin HTTP requests.

What requests use CORS?

This [cross-origin sharing standard] can enable cross-origin HTTP requests for:

- Invocations of the `XMLHttpRequest` or Fetch APIs, as discussed above.
- Web Fonts (for cross-domain font usage in @font-face within CSS), so that servers can deploy TrueType fonts that can only be loaded cross-origin and used by websites that are permitted to do so.
- WebGL textures.
- Images/video frames drawn to a canvas using `drawImage()`.
- CSS Shapes from images.

This is a general article about Cross-Origin Resource Sharing and includes a discussion of the necessary HTTP headers.

Functional overview

The Cross-Origin Resource Sharing standard works by adding new HTTP headers that let servers describe which origins are permitted to read that information from a web browser. Additionally, for HTTP request methods that can cause side-effects on server data (in particular, HTTP methods other than `GET`, or `POST` with certain MIME types), the specification mandates that browsers "preflight" the request, soliciting supported methods from the server with the HTTP `OPTIONS` request method, and then, upon "approval" from the server, sending the actual request. Servers can also inform clients whether "credentials" (such as Cookies and HTTP Authentication) should be sent with requests.

CORS failures result in errors but for security reasons, specifics about the error *are not available to JavaScript*. All the code knows is that an error occurred. The only way to determine what specifically went wrong is to look at the browser's console for details.

Subsequent sections discuss scenarios, as well as provide a breakdown of the HTTP headers used.

Examples of access control scenarios

We present three scenarios that demonstrate how Cross-Origin Resource Sharing works. All these examples use `XMLHttpRequest`, which can make cross-origin requests in any supporting browser.

Simple Requests

Some requests don't need a special permission check called CORS preflight. These are called "simple requests." They are straightforward and don't need extra security checks before being sent.

In the past, web forms could send data to any website. This means servers are already prepared to handle security issues like cross-site request forgery (CSRF). So, if a request looks like a form submission, servers don't need to double-check before accepting it. However, the server still needs to allow the response to be shared with the requesting script.

Conditions for Simple Requests:

- Allowed methods: GET, HEAD, POST
- Only certain headers can be added manually (Accept, Accept-Language, Content-Language, Content-Type, Range).
- The Content-Type header must use specific types (application/x-www-form-urlencoded, multipart/form-data, text/plain).
- Certain technical conditions must be met, like not using certain types of objects or event listeners.

Example: For example, suppose web content at https://foo.example wishes to invoke content on domain https://bar.other. Code of this sort might be used in JavaScript deployed on foo.example. This is a simple request scenario:

```
const xhr = new XMLHttpRequest();
const url = "https://bar.other/resources/public-data/";

xhr.open("GET", url);
xhr.onreadystatechange = someHandler;
xhr.send();
```

Let's look at what the browser will send to the server in this case:

```
GET /resources/public-data/ HTTP/1.1
Host: bar.other
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.14; rv:71.0) Gecko/20100101
            Firefox/71.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-us,en;q=0.5
Accept-Encoding: gzip,deflate
Connection: keep-alive
Origin: https://foo.example
```

The request header of note is Origin, which shows that the invocation is coming from `https://foo.example`.

Now let's see how the server responds:

```
HTTP/1.1 200 OK
Date: Mon, 01 Dec 2008 00:23:53 GMT
Server: Apache/2
Access-Control-Allow-Origin: *
Keep-Alive: timeout=2, max=100
Connection: Keep-Alive
Transfer-Encoding: chunked
Content-Type: application/xml

[…XML Data…]
```

The browser sends an "Origin" header indicating the source (`https://foo.example`), and the server responds with an "Access-Control-Allow-Origin" header, which specifies who can access the resource. If the server allows all origins, it uses "*", but it can also specify a specific origin for more control.

Summary: Simple requests are basic requests that don't need extra security checks. They're used for getting or sending data between different websites. Servers already handle security for these requests, so they don't need extra verification, except for specifying who can access the response. Remember: If a more secure or complex request is needed, additional steps might be required, and servers can set specific rules for which origins are allowed to access their resources.

Preflighted requests

Preflighted requests are a special type of request that browsers send to a different website before making the actual request. This is done to ensure that the actual request is safe and won't cause any issues with user data.

Preflight happens when you're trying to do something a bit more complex, like sending special headers or using certain methods in your request. Browsers check with the server first to see if everything is okay.

Example Scenario: Let's say you want to send data to a different website using JavaScript. If you're doing a simple request, like just getting data using the `GET` method, there's no preflight needed. But if you're sending more specific headers or using methods like `POST`, the browser might need to do a preflight.

How Preflight Works:

1. Browser sends a special `OPTIONS` request to the server before the real request.
2. This `OPTIONS` request asks the server if it's okay to send the real request with the given headers and method.
3. The server responds, saying what's allowed and what's not.
4. If the server approves, the browser sends the actual request.

Imagine you want to send some XML data to another site using JavaScript. The browser might first ask the server if it's okay to do this. If the server says yes, the browser will send the XML data.

Preflight Request example:

```
OPTIONS /api/data HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:89.0) Gecko/20100101
            Firefox/89.0
Accept: */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Access-Control-Request-Method: POST
Access-Control-Request-Headers: X-Custom-Header
Origin: https://your-site.com
Connection: keep-alive
```

Preflight Response example:

```
HTTP/1.1 200 OK
Date: Mon, 01 Jan 2023 12:34:56 GMT
Server: Apache/2.4.39 (Unix)
Access-Control-Allow-Origin: https://your-site.com
Access-Control-Allow-Methods: POST, GET, OPTIONS
Access-Control-Allow-Headers: X-Custom-Header
Access-Control-Max-Age: 86400
Content-Length: 0
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/plain; charset=UTF-8
```

In the example above:

- The browser sends an OPTIONS request (preflight request) to `example.com` to ask if it's okay to send a `POST` request with the custom header `X-Custom-Header`.
- The server responds with a 200 OK status and includes the necessary CORS headers (`Access-Control-Allow-Origin`, `Access-Control-Allow-Methods`, `Access-Control-Allow-Headers`, and `Access-Control-Max-Age`) to indicate that the preflighted request is allowed.

This demonstrates how the preflight process works by checking with the server whether a more complex request is permitted.

Why is Preflight Important? Preflight helps keep things safe and secure. It ensures that any special requests you make won't accidentally mess up user data or cause other problems.

CORS and Redirects: Sometimes, if a preflight request is followed by a redirect, not all browsers handle it the same way. Some might show an error. To work around this, you might need to change the way your requests are structured or the way the server responds.

Remember: As a front-end developer, you don't need to be a preflight expert. Just know that browsers might send these special preflight requests when you're doing more advanced stuff with requests, and understand the basic idea of how they help ensure safe communication between websites.

Requests with Credentials

Credentialed requests are special requests that know about cookies and authentication info. They're used when you need to send sensitive information securely between different websites.

By default, browsers don't send cookies and authentication info in requests to other websites. But you can tell the browser to include them with a special flag.

Let's say you want to get some data from a different website along with your authentication details. You'll use credentialed requests to make sure your info is sent securely.

Using Credentialed Requests: In JavaScript, when you're making a request using XMLHttpRequest or Fetch, you can set the `withCredentials` flag to `true` to include cookies and authentication info. Code Example - Using `XMLHttpRequest`:

```
const xhr = new XMLHttpRequest();
const url = "https://bar.other/resources/credentialed-content/";

function callOtherDomain() {
  if (xhr) {
    xhr.open("GET", url, true);
    xhr.withCredentials = true; // Set to true to include credentials
    xhr.onreadystatechange = handler;
    xhr.send();
  }
}
```

The server you're requesting data from needs to be set up to allow credentialed requests. It should respond with the `Access-Control-Allow-Credentials: true` header.

Credentialed requests are used to send sensitive data between websites securely. You set a flag in your request code to include cookies and authentication info. This ensures that your data is treated with care and doesn't end up in the wrong hands. Keep in mind that third-party cookie policies still apply, so there are some limitations to consider.

The HTTP response headers

This section lists the HTTP response headers that servers return for access control requests as defined by the Cross-Origin Resource Sharing specification. The previous section gives an overview of these in action.

Access-Control-Allow-Origin

A returned resource may have one `Access-Control-Allow-Origin` header with the following syntax:

```
Access-Control-Allow-Origin: <origin> | *
```

`Access-Control-Allow-Origin` specifies either a single origin which tells browsers to allow that origin to access the resource, or else — for requests **without** credentials — the " * " wildcard tells browsers to allow any origin to access the resource.

For example, to allow code from the origin `https://mozilla.org` to access the resource, you can specify:

```
Access-Control-Allow-Origin: https://mozilla.org
Vary: Origin
```

If the server specifies a single origin (that may dynamically change based on the requesting origin as part of an allowlist) rather than the " * " wildcard, then the server should also include `Origin` in the `Vary` response header to indicate to clients that server responses will differ based on the value of the `Origin` request header.

Access-Control-Expose-Headers

The `Access-Control-Expose-Headers` header adds the specified headers to the allowlist that JavaScript (such as `getResponseHeader()`) in browsers is allowed to access.

```
Access-Control-Expose-Headers: <header-name>[, <header-name>]*
```

For example, the following:

```
Access-Control-Expose-Headers: X-My-Custom-Header, X-Another-Custom-Header
```

...would allow the X-My-Custom-Header and X-Another-Custom-Header headers to be exposed to the browser.

Access-Control-Max-Age

The `Access-Control-Max-Age` header indicates how long the results of a preflight request can be cached. For an example of a preflight request, see the above examples.

```
Access-Control-Max-Age: <delta-seconds>
```

The delta-seconds parameter indicates the number of seconds the results can be cached.

Access-Control-Allow-Credentials

The `Access-Control-Allow-Credentials` header indicates whether or not the response to the request can be exposed when the `credentials` flag is true. When used as part of a response to a preflight request, this indicates whether or not the actual request can be made using credentials. Note that simple `GET` requests are not preflighted, and so if a request is made for a resource with credentials, if this header is not returned with the resource, the response is ignored by the browser and not returned to web content.

```
Access-Control-Allow-Credentials: true
```

Credentialed requests are discussed above.

Access-Control-Allow-Methods

The `Access-Control-Allow-Methods` header specifies the method or methods allowed when accessing the resource. This is used in response to a preflight request. The conditions under which a request is preflighted are discussed above.

```
Access-Control-Allow-Methods: <method>[, <method>]*
```

An example of a preflight request is given above, including an example which sends this header to the browser.

Access-Control-Allow-Headers

The `Access-Control-Allow-Headers` header is used in response to a preflight request to indicate which HTTP headers can be used when making the actual request. This header is the server side response to the browser's `Access-Control-Request-Headers` header.

```
Access-Control-Allow-Headers: <header-name>[, <header-name>]*
```

The HTTP request headers

This section lists headers that clients may use when issuing HTTP requests in order to make use of the cross-origin sharing feature. Note that these headers are set for you when making invocations to servers. Developers using cross-origin `XMLHttpRequest` capability do not have to set any cross-origin sharing request headers programmatically.

Origin

The `Origin` header indicates the origin of the cross-origin access request or preflight request.

```
Origin: <origin>
```

The origin is a URL indicating the server from which the request is initiated. It does not include any path information, only the server name.

Note: The `origin` value can be `null`.

Note that in any access control request, the `Origin` header is **always** sent.

Access-Control-Request-Method

The `Access-Control-Request-Method` is used when issuing a preflight request to let the server know what HTTP method will be used when the actual request is made.

```
Access-Control-Request-Method: <method>
```

Examples of this usage can be found above in the preflighted requests lesson.

Access-Control-Request-Headers

The `Access-Control-Request-Headers` header is used when issuing a preflight request to let the server know what HTTP headers will be used when the actual request is made (such as with `setRequestHeader()`). This browser-side header will be answered by the complementary server-side header of `Access-Control-Allow-Headers`.

```
Access-Control-Request-Headers: <field-name>[, <field-name>]*
```

Examples of this usage can be found above in the preflighted requests lesson.

This lesson was adapted from MDN Web Docs: https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS

HTTP authentication

HTTP provides a general framework for access control and authentication. This page is an introduction to the HTTP framework for authentication, and shows how to restrict access to your server using the HTTP "Basic" schema.

The general HTTP authentication framework

RFC 7235 defines the HTTP authentication framework, which can be used by a server to challenge a client request, and by a client to provide authentication information.

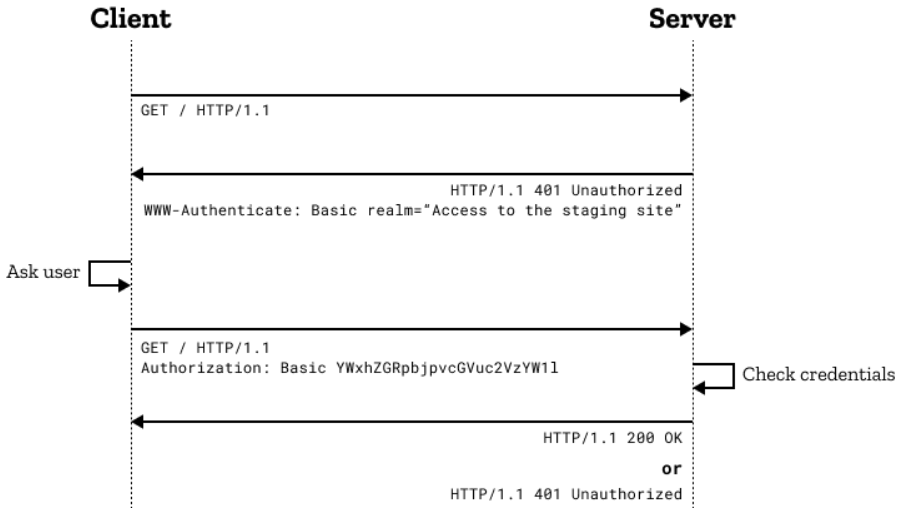
Note: in security protocols, a challenge is some data sent to the client by the server in order to generate a different response each time. Challenge-response protocols are one way to fight against replay attacks where an attacker listens to the previous messages and resends them at a later time to get the same credentials as the original message.

The challenge and response flow works like this:

1. The server responds to a client with a `401` (Unauthorized) response status and provides information on how to authorize with a `WWW-Authenticate` response header containing at least one challenge.
2. A client that wants to authenticate itself with the server can then do so by including an `Authorization` request header with the credentials.
3. Usually a client will present a password prompt to the user and will then issue the request including the correct `Authorization` header.

The general message flow below is the same for most (if not all) authentication schemes. The actual information in the headers and the way it is encoded does change!

Warning: The "Basic" authentication scheme used in the diagram below sends the credentials encoded but not encrypted. This would be completely insecure unless the exchange was over a secure connection (HTTPS/TLS).



Proxy authentication

The same challenge and response mechanism can be used for proxy authentication. As both resource authentication and proxy authentication can coexist, a different set of headers and status codes is needed. In the case of proxies, the challenging status code is `407` (Proxy Authentication Required), the `Proxy-Authenticate` response header contains at least one challenge applicable to the proxy, and the `Proxy-Authorization` request header is used for providing the credentials to the proxy server.

Access forbidden

If a (proxy) server receives invalid credentials, it should respond with a `401 Unauthorized` or with a `407 Proxy Authentication Required`, and the user may send a new request or replace the Authorization header field.

If a (proxy) server receives valid credentials that are inadequate to access a given resource, the server should respond with the `403 Forbidden` status code. Unlike `401 Unauthorized` or `407 Proxy Authentication Required`, authentication is impossible for this user, and browsers will not propose a new attempt.

In all cases, the server may prefer returning a `404 Not Found` status code, to hide the existence of the page to a user without adequate privileges or not correctly authenticated.

Authentication of cross-origin images

A potential security hole (that has since been fixed in browsers) was the authentication of cross-site images. From Firefox 59 onwards, image resources loaded from different origins to the current document are no longer able to trigger HTTP authentication dialogs (Firefox bug 1423146), preventing user credentials from being stolen if attackers were able to embed an arbitrary image into a third-party page.

Character encoding of HTTP authentication

Browsers use `utf-8` encoding for usernames and passwords.

Firefox once used `ISO-8859-1`, but changed to `utf-8` for parity with other browsers and to avoid potential problems.

WWW-Authenticate and Proxy-Authenticate headers

The `WWW-Authenticate` and `Proxy-Authenticate` response headers define the authentication method that should be used to gain access to a resource. They must specify which authentication scheme is used so that the client that wishes to authorize knows how to provide the credentials.

The syntax for these headers is the following:

```
WWW-Authenticate: <type> realm=<realm>
Proxy-Authenticate: <type> realm=<realm>
```

Here, `<type>` is the authentication scheme ("Basic" is the most common scheme and introduced below). The realm is used to describe the protected area or to indicate the scope of protection. This could be a message like "Access to the staging site" or similar, so that the user knows to which space they are trying to get access to.

Authorization and Proxy-Authorization headers

The `Authorization` and `Proxy-Authorization` request headers contain the credentials to authenticate a user agent with a (proxy) server. Here, the `<type>` is needed again followed by the

credentials, which can be encoded or encrypted depending on which authentication scheme is used.

```
Authorization: <type> <credentials>
Proxy-Authorization: <type> <credentials>
```

Authentication schemes

The general HTTP authentication framework is the base for a number of authentication schemes.

IANA maintains a list of authentication schemes, but there are other schemes offered by host services, such as Amazon AWS.

Some common authentication schemes include:

Basic
See RFC 7617, base64-encoded credentials. More information below.

Bearer
See RFC 6750, bearer tokens to access OAuth 2.0-protected resources

Digest
See RFC 7616. Firefox 93 and later support the SHA-256 algorithm. Previous versions only support MD5 hashing (not recommended).

HOBA
See RFC 7486, Section 3, **H**TTP **O**rigin-**B**ound **A**uthentication, digital-signature-based

Mutual
See RFC 8120

Negotiate / **NTLM**
See RFC4599

VAPID
See RFC 8292

SCRAM
See RFC 7804

AWS4-HMAC-SHA256
See AWS docs. This scheme is used for AWS3 server authentication.

Schemes can differ in security strength and in their availability in client or server software.

The "Basic" authentication scheme offers very poor security, but is widely supported and easy to set up. It is introduced in more detail below.

Basic authentication scheme

The "Basic" HTTP authentication scheme is defined in RFC 7617, which transmits credentials as user ID/password pairs, encoded using base64.

Security of basic authentication

As the user ID and password are passed over the network as clear text (it is base64 encoded, but base64 is a reversible encoding), the basic authentication scheme **is not secure**. HTTPS/TLS should be used with basic authentication. Without these additional security enhancements, basic authentication should not be used to protect sensitive or valuable information.

Restricting access with Apache and basic authentication

To password-protect a directory on an Apache server, you will need a `.htaccess` and a `.htpasswd` file.

The `.htaccess` file typically looks like this:

```
AuthType Basic
AuthName "Access to the staging site"
AuthUserFile /path/to/.htpasswd
Require valid-user
```

The `.htaccess` file references a `.htpasswd` file in which each line consists of a username and a password separated by a colon (:). You cannot see the actual passwords as they are hashed (using MD5-based hashing, in this case). Note that you can name your `.htpasswd` file differently if you

like, but keep in mind this file shouldn't be accessible to anyone. (Apache is usually configured to prevent access to `.ht*` files).

```
aladdin:$apr1$ZjTqBB3f$IF9gdYAGlMrs2fuINjHsz.
user2:$apr1$O04r.y2H$/vEkesPhVInBByJUkXitA/
```

Restricting access with Nginx and basic authentication

For Nginx, you will need to specify a location that you are going to protect and the `auth_basic` directive that provides the name to the password-protected area. The `auth_basic_user_file` directive then points to a `.htpasswd` file containing the encrypted user credentials, just like in the Apache example above.

```
location /status {
    auth_basic           "Access to the staging site";
    auth_basic_user_file /etc/apache2/.htpasswd;
}
```

Access using credentials in the URL

Many clients also let you avoid the login prompt by using an encoded URL containing the username and the password like this:

```
https://username:password@www.example.com/
```

The use of these URLs is deprecated. In Chrome, the `username:password@` part in URLs is even stripped out for security reasons. In Firefox, it is checked if the site actually requires authentication and if not, Firefox will warn the user with a prompt "You are about to log in to the site `www.example.com` with the username `username`, but the website does not require authentication. This may be an attempt to trick you."

Note: Authorization is not the same as authentication. Authentication is like proving your identity to a website - like entering your username and password to log in. Authorization: Once you're logged in, authorization decides what parts of the website you're allowed to access or what actions you can perform - like being given permission.

This lesson was adapted from MDN Web Docs: https://developer.mozilla.org/en-US/docs/Web/HTTP/Authentication

HTTP caching

HTTP caching refers to the mechanism by which web browsers, servers, and intermediary proxy servers store and reuse copies of resources, such as web pages, images, stylesheets, scripts, and more. This mechanism is designed to improve the efficiency and speed of web browsing by reducing the need to repeatedly download the same resources from the server.

When you visit a website, your web browser sends requests to the server for various resources that make up the webpage. These resources can include HTML files, images, CSS files, JavaScript files, and more. Instead of fetching these resources from the server every time you visit a page, HTTP caching allows the browser to store a copy of the resources locally on your device.

The HTTP cache stores a response associated with a request and reuses the stored response for subsequent requests.

There are several advantages to reusability. First, since there is no need to deliver the request to the origin server, then the closer the client and cache are, the faster the response will be. The most typical example is when the browser itself stores a cache for browser requests.

Also, when a response is reusable, the origin server does not need to process the request — so it does not need to parse and route the request, restore the session based on the cookie, query the DB for results, or render the template engine. That reduces the load on the server.

Proper operation of the cache is critical to the health of the system.

Types of caches

In the HTTP Caching spec, there are two main types of caches: **private caches** and **shared caches**.

Private caches

A private cache is a cache tied to a specific client — typically a browser cache. Since the stored response is not shared with other clients, a private cache can store a personalized response for that user.

On the other hand, if personalized contents are stored in a cache other than a private cache, then other users may be able to retrieve those contents — which may cause unintentional information leakage.

If a response contains personalized content and you want to store the response only in the private cache, you must specify a private directive.

```
Cache-Control: private
```

Personalized contents are usually controlled by cookies, but the presence of a cookie does not always indicate that it is private, and thus a cookie alone does not make the response private.

Note that if the response has an `Authorization` header, it cannot be stored in the private cache (or a shared cache, unless `public` is specified).

Shared cache

The shared cache is located between the client and the server and can store responses that can be shared among users. And shared caches can be further sub-classified into **proxy caches** and **managed caches**.

Proxy caches

In addition to the function of access control, some proxies implement caching to reduce traffic out of the network. This is usually not managed by the service developer, so it must be controlled by appropriate HTTP headers and so on. However, in the past, outdated proxy-cache implementations — such as implementations that do not properly understand the HTTP Caching standard — have often caused problems for developers.

Kitchen-sink headers like the following are used to try to work around "old and not updated proxy cache" implementations that do not understand current HTTP Caching spec directives like no-store.

```
Cache-Control: no-store, no-cache, max-age=0, must-revalidate, proxy-revalidate
```

However, in recent years, as HTTPS has become more common and client/server communication has become encrypted, proxy caches in the path can only tunnel a response and can't behave as

a cache, in many cases. So in that scenario, there is no need to worry about outdated proxy cache implementations that cannot even see the response.

On the other hand, if a TLS bridge proxy decrypts all communications in a person-in-the-middle manner by installing a certificate from a CA (certificate authority) managed by the organization on the PC, and performs access control, etc. — it is possible to see the contents of the response and cache it. However, since CT (certificate transparency) has become widespread in recent years, and some browsers only allow certificates issued with an SCT (signed certificate timestamp), this method requires the application of an enterprise policy. In such a controlled environment, there is no need to worry about the proxy cache being "out of date and not updated".

Managed caches

Managed caches are explicitly deployed by service developers to offload the origin server and to deliver content efficiently. Examples include reverse proxies, CDNs, and service workers in combination with the Cache API.

The characteristics of managed caches vary depending on the product deployed. In most cases, you can control the cache's behavior through the `Cache-Control` header and your own configuration files or dashboards.

For example, the HTTP Caching specification essentially does not define a way to explicitly delete a cache — but with a managed cache, the stored response can be deleted at any time through dashboard operations, API calls, restarts, and so on. That allows for a more proactive caching strategy.

It is also possible to ignore the standard HTTP Caching spec protocols in favor of explicit manipulation. For example, the following can be specified to opt-out of a private cache or proxy cache, while using your own strategy to cache only in a managed cache.

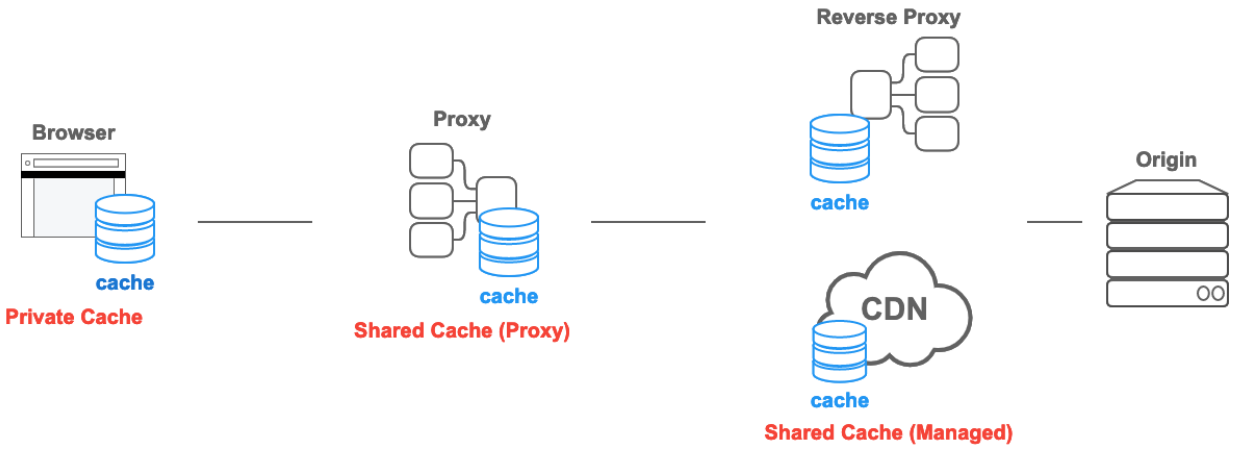
```
Cache-Control: no-store
```

For example, Varnish Cache uses VCL (Varnish Configuration Language, a type of DSL) logic to handle cache storage, while service workers in combination with the Cache API allow you to create that logic in JavaScript.

That means if a managed cache intentionally ignores a no-store directive, there is no need to perceive it as being "non-compliant" with the standard. What you should do is, avoid using kitchen-sink headers, but carefully read the documentation of whatever managed-cache

mechanism you're using, and ensure you're controlling the cache properly in the ways provided by the mechanism you've chosen to use.

Note that some CDNs provide their own headers that are effective only for that CDN (for example, Surrogate-Control). Currently, work is underway to define a CDN-Cache-Control header to standardize those.



Heuristic caching

HTTP is designed to cache as much as possible, so even if no `Cache-Control` is given, responses will get stored and reused if certain conditions are met. This is called **heuristic caching**.

For example, take the following response. This response was last updated 1 year ago.

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 1024
Date: Tue, 22 Feb 2022 22:22:22 GMT
Last-Modified: Tue, 22 Feb 2021 22:22:22 GMT

<!doctype html>
…
```

It is heuristically known that content which has not been updated for a full year will not be updated for some time after that. Therefore, the client stores this response (despite the lack of `max-age`) and reuses it for a while. How long to reuse is up to the implementation, but the specification recommends about 10% (in this case 0.1 year) of the time after storing.

Heuristic caching is a workaround that came before `Cache-Control` support became widely adopted, and basically all responses should explicitly specify a `Cache-Control` header.

Fresh and stale based on age

Stored HTTP responses have two states: **fresh** and **stale**. The *fresh* state usually indicates that the response is still valid and can be reused, while the *stale* state means that the cached response has already expired.

The criterion for determining when a response is fresh and when it is stale is **age**. In HTTP, age is the time elapsed since the response was generated. This is similar to the TTL in other caching mechanisms.

Take the following example response (604800 seconds is one week):

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 1024
Date: Tue, 22 Feb 2022 22:22:22 GMT
Cache-Control: max-age=604800

<!doctype html>
…
```

The cache that stored the example response calculates the time elapsed since the response was generated and uses the result as the response's *age*.

For the example response, the meaning of `max-age` is the following:

- If the age of the response is *less* than one week, the response is *fresh*.
- If the age of the response is *more* than one week, the response is *stale*.

As long as the stored response remains fresh, it will be used to fulfill client requests.

When a response is stored in a shared cache, it is necessary to inform the client of the response's age. Continuing with the example, if the shared cache stored the response for one day, the shared cache would send the following response to subsequent client requests.

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 1024
Date: Tue, 22 Feb 2022 22:22:22 GMT
Cache-Control: max-age=604800
Age: 86400

<!doctype html>
…
```

The client who receives that response will find it to be fresh for the remaining 518400 seconds, the difference between the response's `max-age` and `Age`.

Expires or max-age

In HTTP/1.0, freshness used to be specified by the `Expires` header.

The `Expires` header specifies the lifetime of the cache using an explicit time rather than by specifying an elapsed time.

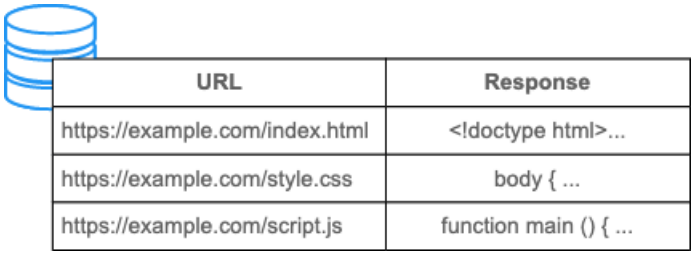
```
Expires: Tue, 28 Feb 2022 22:22:22 GMT
```

However, the time format is difficult to parse, many implementation bugs were found, and it is possible to induce problems by intentionally shifting the system clock; therefore, `max-age` — for specifying an elapsed time — was adopted for `Cache-Control` in HTTP/1.1.

If both `Expires` and `Cache-Control`: `max-age` are available, `max-age` is defined to be preferred. So it is not necessary to provide `Expires` now that HTTP/1.1 is widely used.

Vary

The way that responses are distinguished from one another is essentially based on their URLs:



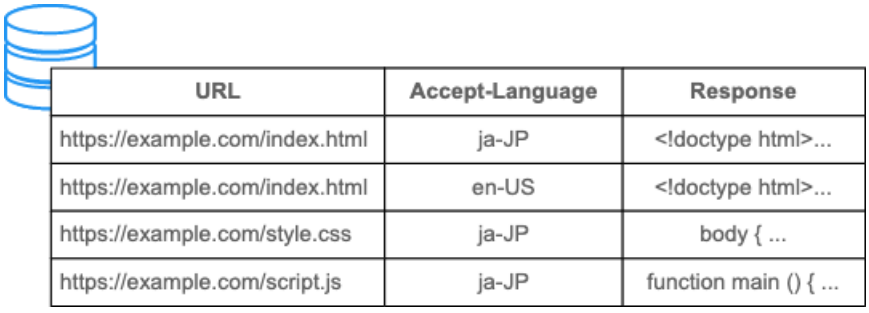
URL	Response
https://example.com/index.html	<!doctype html>...
https://example.com/style.css	body { ...
https://example.com/script.js	function main () { ...

But the contents of responses are not always the same, even if they have the same URL. Especially when content negotiation is performed, the response from the server can depend on the values of the `Accept`, `Accept-Language`, and `Accept-Encoding` request headers.

For example, for English content returned with an `Accept-Language: en` header and cached, it is undesirable to then reuse that cached response for requests that have an `Accept-Language: ja` request header. In this case, you can cause the responses to be cached separately — based on language — by adding "`Accept-Language`" to the value of the `Vary` header.

```
Vary: Accept-Language
```

That causes the cache to be keyed based on a composite of the response URL and the `Accept-Language` request header — rather than being based just on the response URL.



URL	Accept-Language	Response
https://example.com/index.html	ja-JP	<!doctype html>...
https://example.com/index.html	en-US	<!doctype html>...
https://example.com/style.css	ja-JP	body { ...
https://example.com/script.js	ja-JP	function main () { ...

Also, if you are providing content optimization (for example, for responsive design) based on the user agent, you may be tempted to include "`User-Agent`" in the value of the `Vary` header. However, the `User-Agent` request header generally has a very large number of variations, which drastically reduces the chance that the cache will be reused. So if possible, instead consider a way to vary behavior based on feature detection rather than based on the `User-Agent` request header.

For applications that employ cookies to prevent others from reusing cached personalized content, you should specify `Cache-Control: private` instead of specifying a cookie for `Vary`.

Validation

Stale responses are not immediately discarded. HTTP has a mechanism to transform a stale response into a fresh one by asking the origin server. This is called **validation**, or sometimes, **revalidation**.

Validation is done by using a **conditional request** that includes an `If-Modified-Since` or `If-None-Match` request header.

If-Modified-Since

If response was generated at 22:22:22 and has a `max-age` of 1 hour, so you know that it is fresh until 23:22:22.

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 1024
Date: Tue, 22 Feb 2022 22:22:22 GMT
Last-Modified: Tue, 22 Feb 2022 22:00:00 GMT
Cache-Control: max-age=3600

<!doctype html>
…
```

At 23:22:22, the response becomes stale and the cache cannot be reused. So the request below shows a client sending a request with an If-Modified-Since request header, to ask the server if there have been any changes made since the specified time.

```
GET /index.html HTTP/1.1
Host: example.com
Accept: text/html
If-Modified-Since: Tue, 22 Feb 2022 22:00:00 GMT
```

The server will respond with `304 Not Modified` if the content has not changed since the specified time.

Since this response only indicates "no change", there is no response body — there's just a status code — so the transfer size is extremely small.

```
HTTP/1.1 304 Not Modified
Content-Type: text/html
Date: Tue, 22 Feb 2022 23:22:22 GMT
Last-Modified: Tue, 22 Feb 2022 22:00:00 GMT
Cache-Control: max-age=3600
```

Upon receiving that response, the client reverts the stored stale response back to being fresh and can reuse it during the remaining 1 hour.

The server can obtain the modification time from the operating-system file system, which is relatively easy to do for the case of serving static files. However, there are some problems; for example, the time format is complex and difficult to parse, and distributed servers have difficulty synchronizing file-update times.

To solve such problems, the ETag response header was standardized as an alternative.

ETag/If-None-Match

The value of the `ETag` response header is an arbitrary value generated by the server. There are no restrictions on how the server must generate the value, so servers are free to set the value based on whatever means they choose — such as a hash of the body contents or a version number.

As an example, if a hash value is used for the `ETag` header and the hash value of the `index.html` resource is `33a64df5`, the response will be as follows:

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 1024
Date: Tue, 22 Feb 2022 22:22:22 GMT
ETag: "33a64df5"
Cache-Control: max-age=3600

<!doctype html>
…
```

If that response is stale, the client takes the value of the `ETag` response header for the cached response, and puts it into the `If-None-Match` request header, to ask the server if the resource has been modified:

```
GET /index.html HTTP/1.1
Host: example.com
Accept: text/html
If-None-Match: "33a64df5"
```

The server will return `304 Not Modified` if the value of the `ETag` header it determines for the requested resource is the same as the `If-None-Match` value in the request.

But if the server determines the requested resource should now have a different `ETag` value, the server will instead respond with a `200 OK` and the latest version of the resource.

Force Revalidation

If you do not want a response to be reused, but instead want to always fetch the latest content from the server, you can use the no-cache directive to force validation.

By adding `Cache-Control: no-cache` to the response along with `Last-Modified` and `ETag` — as shown below — the client will receive a `200 OK` response if the requested resource has been updated, or will otherwise receive a `304 Not Modified` response if the requested resource has not been updated.

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 1024
Date: Tue, 22 Feb 2022 22:22:22 GMT
Last-Modified: Tue, 22 Feb 2022 22:00:00 GMT
ETag: deadbeef
Cache-Control: no-cache

<!doctype html>
…
```

It is often stated that the combination of `max-age=0` and `must-revalidate` has the same meaning as `no-cache`.

```
Cache-Control: max-age=0, must-revalidate
```

`max-age=0` means that the response is immediately stale, and `must-revalidate` means that it must not be reused without revalidation once it is stale — so, in combination, the semantics seem to be the same as `no-cache`.

However, that usage of `max-age=0` is a remnant of the fact that many implementations prior to HTTP/1.1 were unable to handle the `no-cache` directive — and so to deal with that limitation, `max-age=0` was used as a workaround.

But now that HTTP/1.1-conformant servers are widely deployed, there's no reason to ever use that `max-age=0` and `must-revalidate` combination — you should instead just use `no-cache`.

Don't cache

The no-cache directive does not prevent the storing of responses but instead prevents the reuse of responses without revalidation.

If you don't want a response stored in any cache, use `no-store`.

```
Cache-Control: no-store
```

However, in general, a "do not cache" requirement in practice amounts to the following set of circumstances:

- Don't want the response stored by anyone other than the specific client, for privacy reasons.
- Want to provide up-to-date information always.
- Don't know what could happen in outdated implementations.

Under that set of circumstances, `no-store` is not always the most-appropriate directive.

Do not share with others

It would be problematic if a response with personalized content is unexpectedly visible to other users of a cache.

In such a case, using the `private` directive will cause the personalized response to only be stored with the specific client and not be leaked to any other user of the cache.

```
Cache-Control: private
```

In such a case, even if `no-store` is given, `private` must also be given.

Provide up-to-date content every time

The `no-store` directive prevents a response from being stored, but does not delete any already-stored response for the same URL.

In other words, if there is an old response already stored for a particular URL, returning `no-store` will not prevent the old response from being reused.

However, a `no-cache` directive will force the client to send a validation request before reusing any stored response.

```
Cache-Control: no-cache
```

If the server does not support conditional requests, you can force the client to access the server every time and always get the latest response with `200 OK`.

Dealing with outdated implementations

As a workaround for outdated implementations that ignore `no-store`, you may see kitchen-sink headers such as the following being used.

```
Cache-Control: no-store, no-cache, max-age=0, must-revalidate, proxy-revalidate
```

It is recommended to use `no-cache` as an alternative for dealing with such outdated implementations, and it is not a problem if `no-cache` is given from the beginning, since the server will always receive the request.

If it is the shared cache that you are concerned about, you can make sure to prevent unintended caching by also adding `private`:

```
Cache-Control: no-cache, private
```

What's lost by `no-store`

You may think adding `no-store` would be the right way to opt-out of caching.

However, it's not recommended to grant `no-store` liberally, because you lose many advantages that HTTP and browsers have, including the browser's back/forward cache.

Therefore, to get the advantages of the full feature set of the web platform, prefer the use of `no-cache` in combination with `private`.

Reload and force reload

Validation can be performed for requests as well as responses.

The **reload** and **force reload** actions are common examples of validation performed from the browser side.

Reload

For recovering from window corruption or updating to the latest version of the resource, browsers provide a reload function for users.

A simplified view of the HTTP request sent during a browser reload looks as follows:

```
GET / HTTP/1.1
Host: example.com
Cache-Control: max-age=0
If-None-Match: "deadbeef"
If-Modified-Since: Tue, 22 Feb 2022 20:20:20 GMT
```

The `max-age=0` directive in the request specifies "reuse of responses with an age of 0 or less" — so, in effect, intermediately stored responses are not reused.

As a result, a request is validated by `If-None-Match` and `If-Modified-Since`.

That behavior is also defined in the Fetch standard and can be reproduced in JavaScript by calling `fetch()` with the cache mode set to `no-cache` (note that `reload` is not the right mode for this case):

```
// Note: "reload" is not the right mode for a normal reload; "no-cache" is
fetch("/", { cache: "no-cache" });
```

Force reload

Browsers use `max-age=0` during reloads for backward-compatibility reasons — because many outdated implementations prior to HTTP/1.1 did not understand `no-cache`. But `no-cache` is fine now in this use case, and force reload is an additional way to bypass cached responses.

The HTTP Request during a browser **force reload** looks as follows:

```
GET / HTTP/1.1
Host: example.com
Pragma: no-cache
Cache-Control: no-cache
```

Since that's not a conditional request with `no-cache`, you can be sure you'll get a `200 OK` from the origin server.

That behavior is also defined in the Fetch standard and can be reproduced in JavaScript by calling `fetch()` with the cache mode set to `reload` (note that it's not `force-reload`):

```
// Note: "reload" — rather than "no-cache" — is the right mode for a "force reload"
fetch("/", { cache: "reload" });
```

Avoiding revalidation

Content that never changes should be given a long `max-age` by using cache busting — that is, by including a version number, hash value, etc., in the request URL.

However, when the user reloads, a revalidation request is sent even though the server knows that the content is immutable.

To prevent that, the `immutable` directive can be used to explicitly indicate that revalidation is not required because the content never changes.

```
Cache-Control: max-age=31536000, immutable
```

That prevents unnecessary revalidation during reloads.

Note that, instead of implementing that directive, Chrome has changed its implementation so that revalidation is not performed during reloads for subresources.

Deleting stored responses

There is basically no way to delete responses that have already been stored with a long `max-age`.

Imagine that the following response from `https://example.com/` was stored.

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 1024
Cache-Control: max-age=31536000

<!doctype html>
…
```

You may want to overwrite that response once it expired on the server, but there is nothing the server can do once the response is stored — since no more requests reach the server due to caching.

One of the methods mentioned in the specification is to send a request for the same URL with an unsafe method such as `POST`, but that is usually difficult to intentionally do for many clients.

There is also a specification for a `Clear-Site-Data: cache` header and value, but not all browsers support it — and even when it's used, it only affects browser caches and has no effect on intermediate caches.

Therefore, it should be assumed that any stored response will remain for its `max-age` period unless the user manually performs a reload, force-reload, or clear-history action.

Caching reduces access to the server, which means that the server loses control of that URL. If the server does not want to lose control of a URL — for example, in the case that a resource is frequently updated — you should add `no-cache` so that the server will always receive requests and send the intended responses.

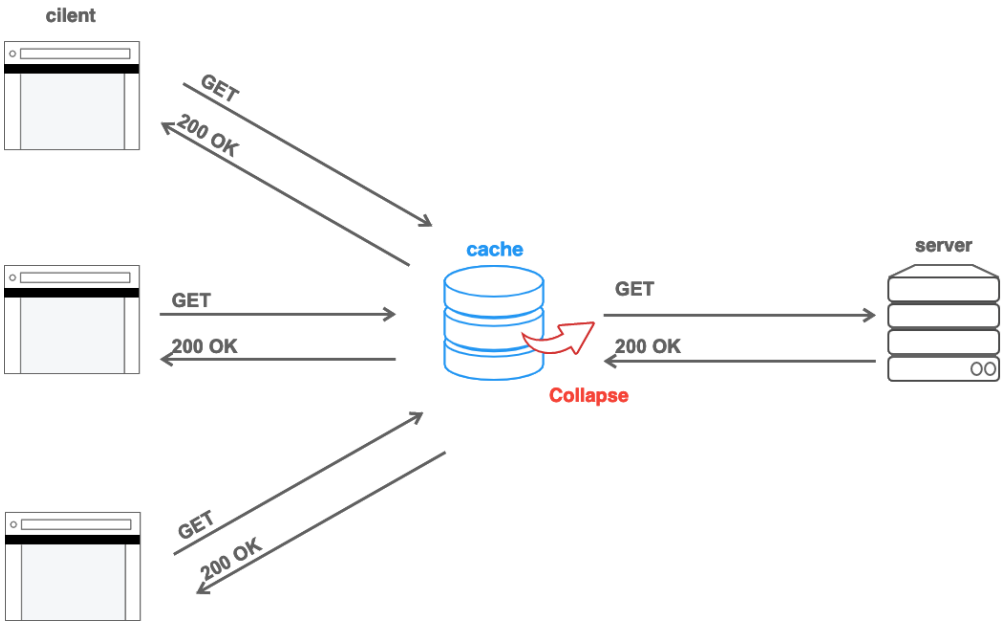
Request collapse

The shared cache is primarily located before the origin server and is intended to reduce traffic to the origin server.

Thus, if multiple identical requests arrive at a shared cache at the same time, the intermediate cache will forward a single request on behalf of itself to the origin, which can then reuse the result for all clients. This is called **request collapse**.

Request collapse occurs when requests are arriving at the same time, so even if `max-age=0` or `no-cache` is given in the response, it will be reused.

If the response is personalized to a particular user and you do not want it to be shared in collapse, you should add the `private` directive:



Common caching patterns

Caching in the context of HTTP involves strategies to store and manage cached responses for web resources like images, stylesheets, and HTML pages. There are many directives in the `Cache-Control` spec, and it may be difficult to understand all of them. Let's explore common caching patterns and techniques along with code examples:

1. **Default Settings**: By default, browsers cache resources to improve page loading speed. The browser checks the "Cache-Control" header in the response to determine how long the resource should be cached. A common value is "public, max-age=3600", which caches the resource for an hour.

```
Cache-Control: public, max-age=3600
```

2. **Cache Busting**: Cache busting involves altering the URL of a resource to force the browser to fetch the latest version, even if the URL remains the same. This can be achieved by appending a unique identifier like a timestamp or a version number to the URL.

```
<link rel="stylesheet" href="styles.css?v=12345">
```

3. **Validation with ETag**: The server can use an `ETag` (Entity Tag), a unique identifier for a resource's version, to validate whether the cached resource is still up to date. If the `ETag` matches, the server responds with a "`304 Not Modified`" status, and the browser uses the cached version.

```
ETag: "abcdef123456"
```

4. **Main Resources (Critical Rendering Path)**: To improve initial page load speed, focus on caching critical resources required for rendering the page (like HTML, CSS, and JavaScript). Use aggressive caching for these resources to ensure a faster user experience.

```
Cache-Control: public, max-age=604800
```

5. **Managed Caches in HTTP**: Managed caches, like Content Delivery Networks (CDNs), are intermediaries that store and serve cached content from locations closer to the user. They use a combination of cache settings and dynamic routing to ensure efficient content delivery.

   CDNs can be instructed to cache specific resources based on cache control headers, and they also offer cache invalidation mechanisms to remove outdated content.

```
Cache-Control: public, max-age=2592000
```

Remember that these caching patterns and techniques are meant to optimize the performance of your web applications. The choice of which pattern to use depends on factors like the nature of your content, how frequently it changes, and the user experience you want to provide. Always test and measure the impact of your caching strategies to ensure they align with your goals.

This lesson was adapted from MDN Web Docs: https://developer.mozilla.org/en-US/docs/Web/HTTP/Caching

Compression in HTTP

Compression is an important way to increase the performance of a website. For some documents, size reduction of up to 70% lowers the bandwidth capacity needs. Over the years, algorithms also got more efficient, and new ones are supported by clients and servers.

In practice, web developers don't need to implement compression mechanisms, both browsers and servers have it implemented already, but they have to be sure that the server is configured adequately. Compression happens at three different levels:

- first some file formats are compressed with specific optimized methods,
- then general encryption can happen at the HTTP level (the resource is transmitted compressed from end to end),
- and finally compression can be defined at the connection level, between two nodes of an HTTP connection.

File format compression

Each data type has some redundancy, that is *wasted space*, in it. If text can typically have as much as 60% redundancy, this rate can be much higher for some other media like audio and video. Unlike text, these other media types use a lot of space to store their data and the need to optimize storage and regain space was apparent very early. Engineers designed the optimized compression algorithm used by file formats designed for this specific purpose. Compression algorithms used for files can be grouped into two broad categories:

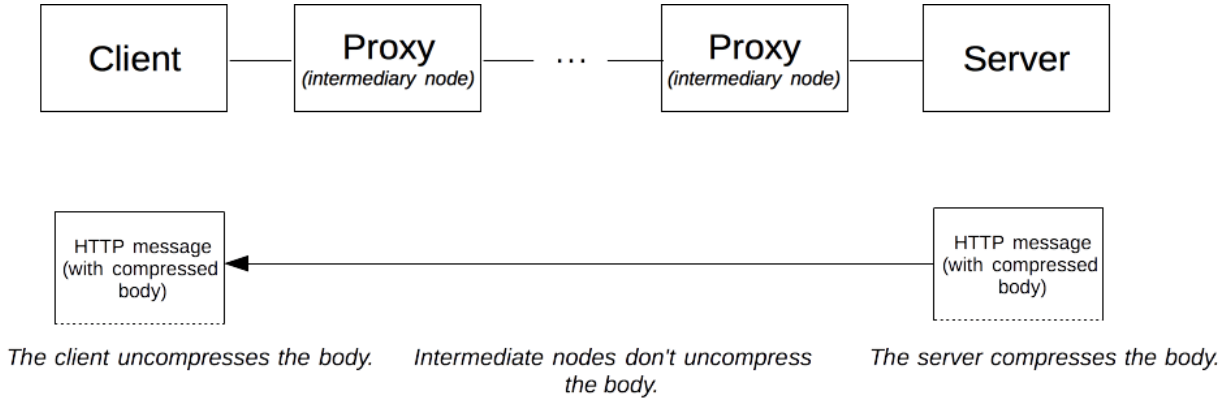
- *Loss-less compression*, where the compression-uncompression cycle doesn't alter the data that is recovered. It matches (byte to byte) with the original. For images, gif or png are using lossless compression.
- *Lossy compression*, where the cycle alters the original data in a (hopefully) imperceptible way for the user. Video formats on the Web are lossy; the jpeg image format is also lossy.

Some formats can be used for both loss-less or lossy compression, like webp, and usually lossy algorithm can be configured to compress more or less, which then of course leads to less or more quality. For better performance of a website, it is ideal to compress as much as possible, while keeping an acceptable level of quality. For images, an image generated by a tool could be not

optimized enough for the Web; it is recommended to use tools that will compress as much as possible with the required quality. There are numerous tools that are specialized for this.
Lossy compression algorithms are usually more efficient than loss-less ones.

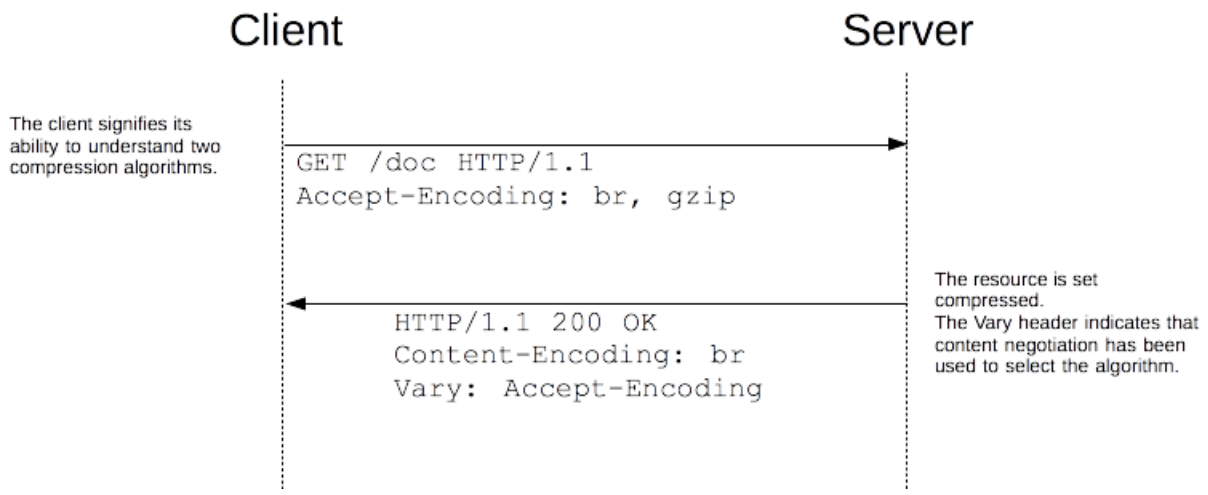
End-to-end compression

For compression, end-to-end compression is where the largest performance improvements of websites reside. End-to-end compression refers to a compression of the body of a message that is done by the server and will last unchanged until it reaches the client. Whatever the intermediate nodes are, they leave the body untouched.



All modern browsers and servers do support it and the only thing to negotiate is the compression algorithm to use. These algorithms are optimized for text. In the 1990s, compression technology was advancing at a rapid pace and numerous successive algorithms have been added to the set of possible choices. Nowadays, only two are relevant: `gzip`, the most common one, and `br` the new challenger.

To select the algorithm to use, browsers and servers use proactive content negotiation. The browser sends an Accept-Encoding header with the algorithm it supports and its order of precedence, the server picks one, uses it to compress the body of the response and uses the Content-Encoding header to tell the browser the algorithm it has chosen. As content negotiation has been used to choose a representation based on its encoding, the server must send a Vary header containing at least Accept-Encoding alongside this header in the response; that way, caches will be able to cache the different representations of the resource.

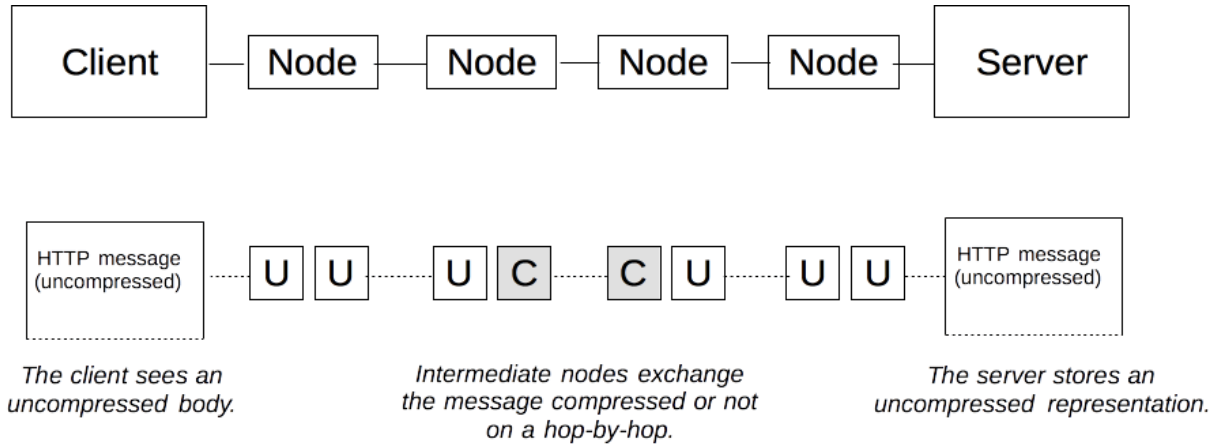


As compression brings significant performance improvements, it is recommended to activate it for all files, but already compressed ones like images, audio files and videos.

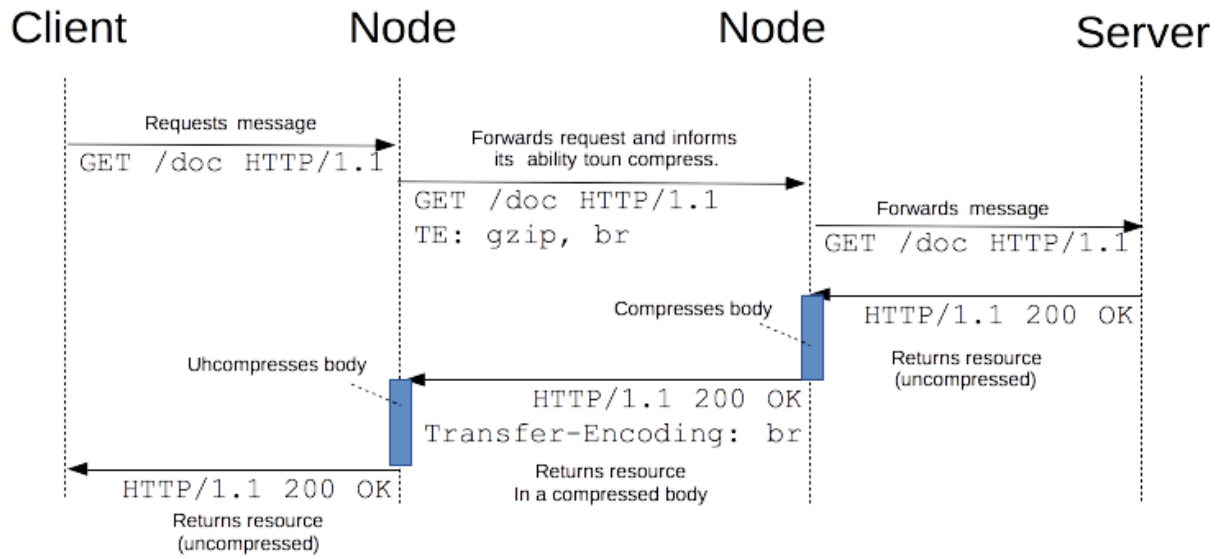
Apache supports compression and uses mod_deflate; for Nginx there is ngx_http_gzip_module; for IIS, the `<httpCompression>` element.

Hop-by-hop compression

Hop-by-hop compression, though similar to end-to-end compression, differs by one fundamental element: the compression doesn't happen on the resource in the server, creating a specific representation that is then transmitted, but on the body of the message between any two nodes on the path between the client and the server. Connections between successive intermediate nodes may apply a *different* compression.



To do this, HTTP uses a mechanism similar to the content negotiation for end-to-end compression: the node transmitting the request advertizes its will using the TE header and the other node chooses the adequate method, applies it, and indicates its choice with the Transfer-Encoding header.



In practice, hop-by-hop compression is transparent for the server and the client, and is rarely used. TE and Transfer-Encoding are mostly used to send a response by chunks, allowing to start transmitting a resource without knowing its length.

Note that using Transfer-Encoding and compression at the hop level is so rare that most servers, like Apache, Nginx, or IIS, have no easy way to configure it. Such configuration usually happens at the proxy level.

This lesson is adapted from MDN web https://developer.mozilla.org/en-US/docs/Web/HTTP/Compression

HTTP conditional requests

HTTP has a concept of conditional requests, where the result, and even the success of a request, can be changed by comparing the affected resources with the value of a validator. Such requests can be useful to validate the content of a cache, and sparing a useless control, to verify the integrity of a document, like when resuming a download, or when preventing lost updates when uploading or modifying a document on the server.

Principles

HTTP conditional requests are requests that are executed differently, depending on the value of specific headers. These headers define a precondition, and the result of the request will be different if the precondition is matched or not.

The different behaviors are defined by the method of the request used, and by the set of headers used for a precondition:

- for safe methods, like `GET`, which usually tries to fetch a document, the conditional request can be used to send back the document, if relevant only. Therefore, this spares bandwidth.
- for unsafe methods, like `PUT`, which usually uploads a document, the conditional request can be used to upload the document, only if the original it is based on is the same as that stored on the server.

Validators

All conditional headers try to check if the resource stored on the server matches a specific version. To achieve this, the conditional requests need to indicate the version of the resource. As comparing the whole resource byte to byte is impracticable, and not always what is wanted, the request transmits a value describing the version. Such values are called *validators*, and are of two kinds:

- the date of last modification of the document, the *last-modified* date.
- an opaque string, uniquely identifying each version, called the *entity tag*, or the *etag*.

Comparing versions of the same resource is a bit tricky: depending on the context, there are two kinds of *equality checks*:

- *Strong validation* is used when byte to byte identity is expected, for example when resuming a download.
- *Weak validation* is used when the user-agent only needs to determine if the two resources have the same content. This is even if they are minor differences; like different ads, or a footer with a different date.

The kind of validation is independent of the validator used. Both `Last-Modified` and `ETag` allow both types of validation, though the complexity to implement it on the server side may vary. HTTP uses strong validation by default, and it specifies when weak validation can be used.

Strong validation

Strong validation consists of guaranteeing that the resource is, byte to byte, identical to the one it is compared to. This is mandatory for some conditional headers, and the default for the others. Strong validation is very strict and may be difficult to guarantee at the server level, but it does guarantee no data loss at any time, sometimes at the expense of performance.

It is quite difficult to have a unique identifier for strong validation with `Last-Modified`. Often this is done using an `ETag` with the MD5 hash of the resource (or a derivative).

Weak validation

Weak validation differs from strong validation, as it considers two versions of the document as identical if the content is equivalent. For example, a page that would differ from another only by a different date in its footer, or different advertising, would be considered *identical* to the other with weak validation. These same two versions are considered *different* when using strong validation. Building a system of etags that creates weak validation may be complex, as it involves knowing the importance of the different elements of a page, but is very useful towards optimizing cache performance.

Conditional headers

Several HTTP headers, called conditional headers, lead to conditional requests. These are:

`If-Match`

Succeeds if the `ETag` of the distant resource is equal to one listed in this header. It performs a strong validation.

`If-None-Match`

Succeeds if the `ETag` of the distant resource is different to each listed in this header. It performs a weak validation.

`If-Modified-Since`

Succeeds if the `Last-Modified` date of the distant resource is more recent than the one given in this header.

`If-Unmodified-Since`

Succeeds if the `Last-Modified` date of the distant resource is older or the same as the one given in this header.

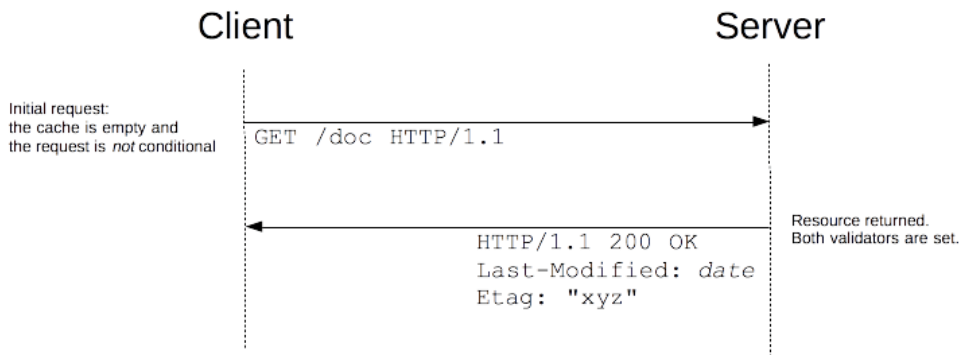
`If-Range`

Similar to `If-Match`, or `If-Unmodified-Since`, but can have only one single etag, or one date. If it fails, the range request fails, and instead of a `206 Partial Content` response, a `200 OK` is sent with the complete resource.

Use cases

Cache update

The most common use case for conditional requests is updating a cache. With an empty cache, or without a cache, the requested resource is sent back with a status of `200 OK`

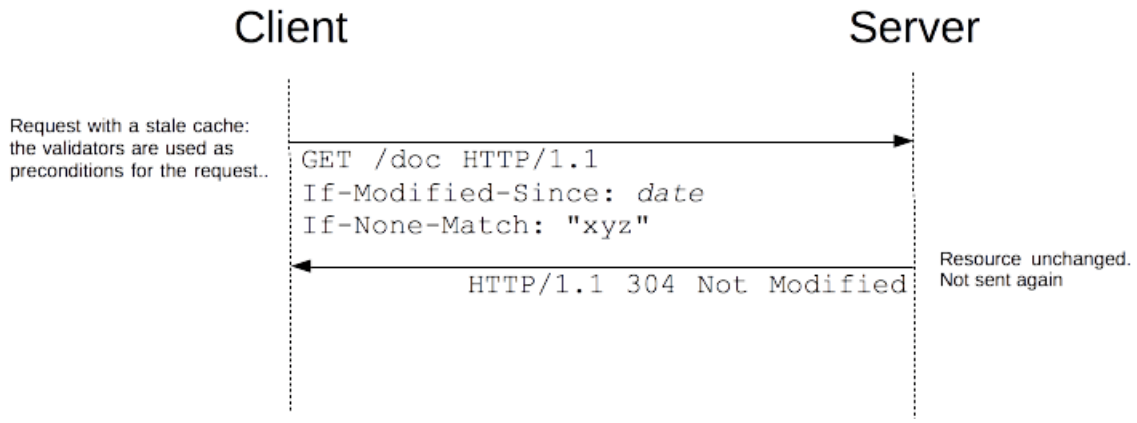


Together with the resource, the validators are sent in the headers. In this example, both `Last-Modified` and `ETag` are sent, but it could equally have been only one of them. These validators are

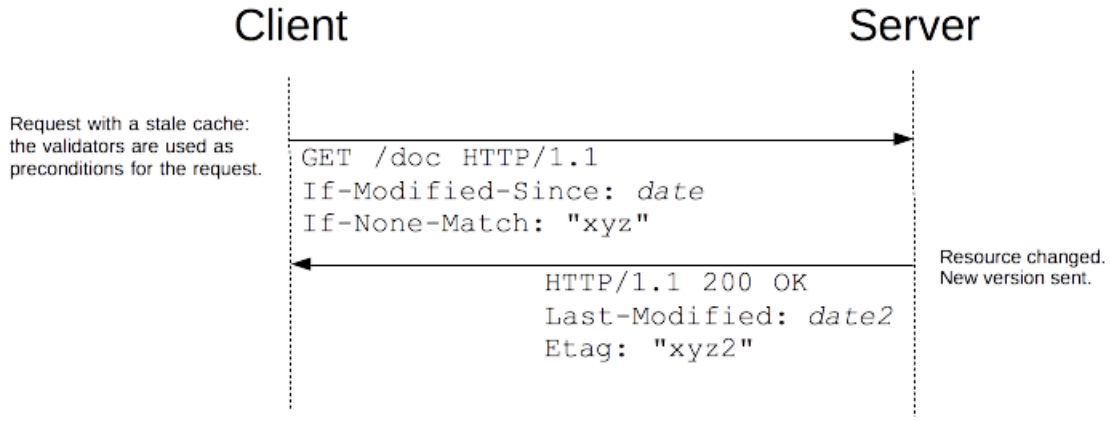
cached with the resource (like all headers) and will be used to craft conditional requests, once the cache becomes stale.

As long as the cache is not stale, no requests are issued at all. But once it has become stale, this is mostly controlled by the `Cache-Control` header, the client doesn't use the cached value directly but issues a *conditional request*. The value of the validator is used as a parameter of the `If-Modified-Since` and `If-None-Match` headers.

If the resource has not changed, the server sends back a `304 Not Modified` response. This makes the cache fresh again, and the client uses the cached resource. Although there is a response/request round-trip that consumes some resources, this is more efficient than to transmit the whole resource over the wire again.



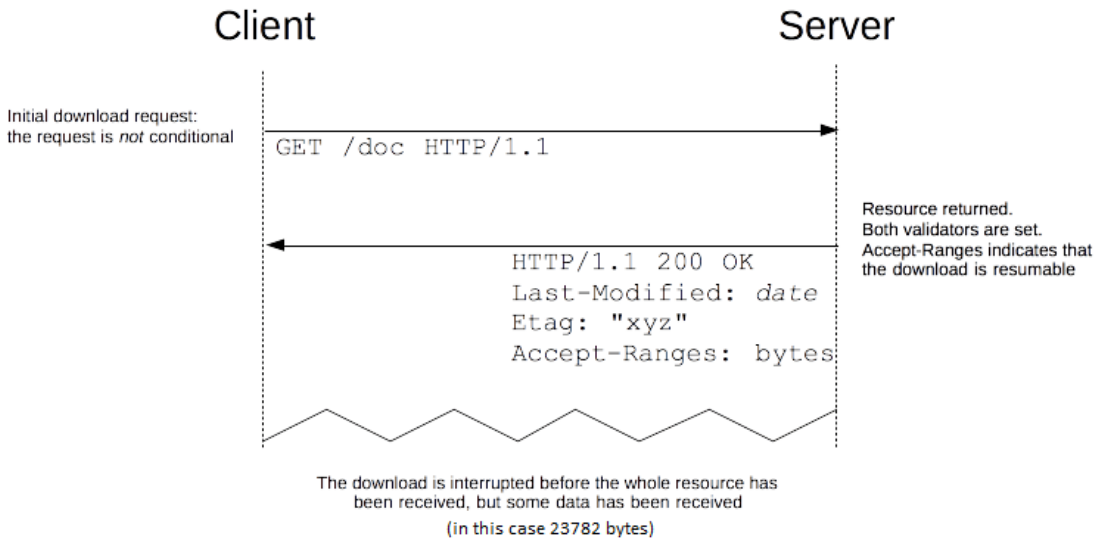
If the resource has changed, the server just sends back a `200 OK` response, with the new version of the resource (as though the request wasn't conditional). The client uses this new resource (and caches it).



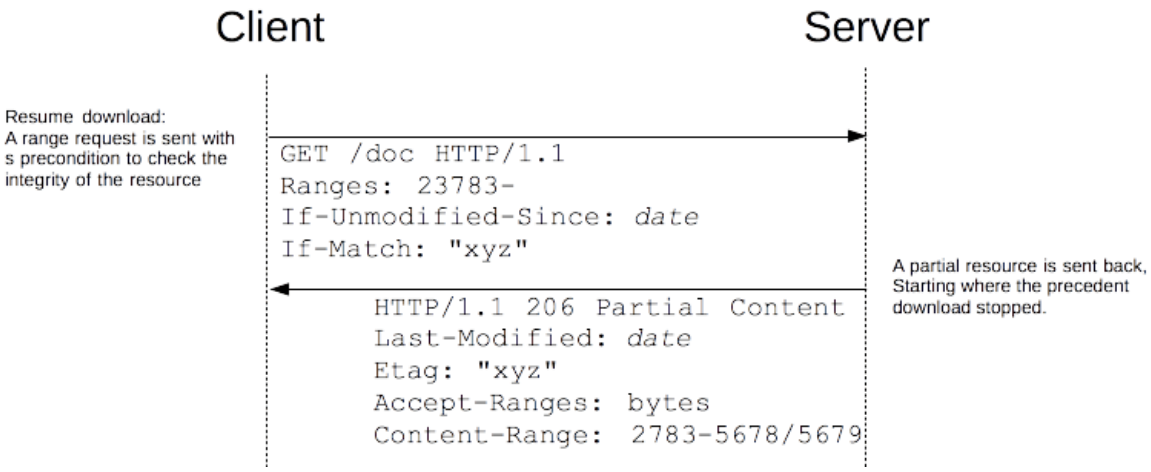
Besides the setting of the validators on the server side, this mechanism is transparent: all browsers manage a cache and send such conditional requests without any special work to be done by Web developers.

Integrity of a partial download

Partial downloading of files is a functionality of HTTP that allows resuming previous operations, saving bandwidth and time, by keeping the already obtained information:

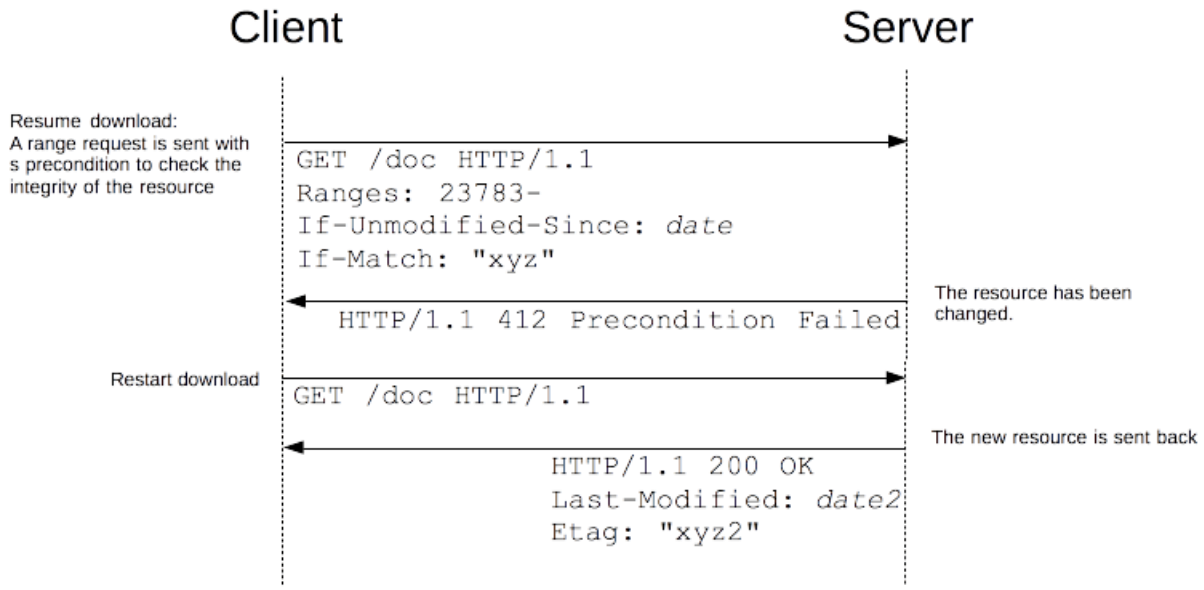


A server supporting partial downloads broadcasts this by sending the `Accept-Ranges` header. Once this happens, the client can resume a download by sending a `Ranges` header with the missing ranges:

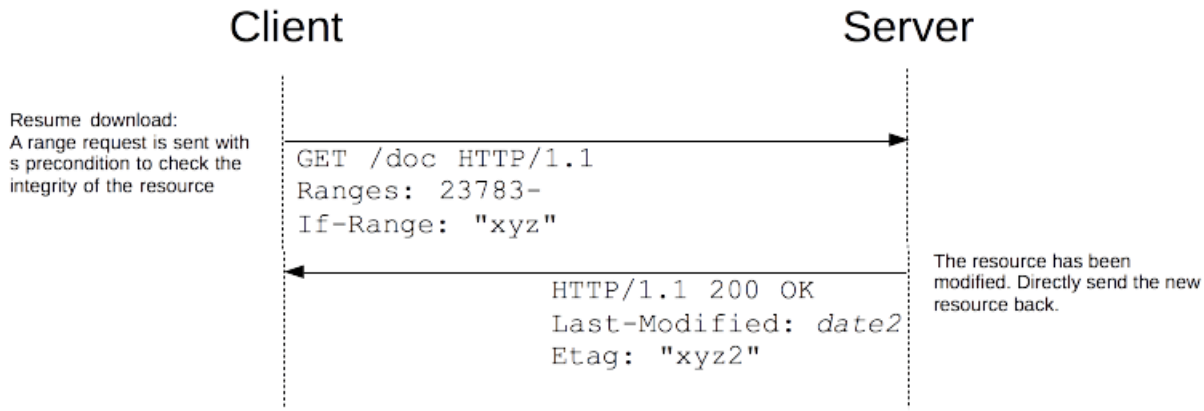


The principle is simple, but there is one potential problem: if the downloaded resource has been modified between both downloads, the obtained ranges will correspond to two different versions of the resource, and the final document will be corrupted.

To prevent this, conditional requests are used. For ranges, there are two ways of doing this. The more flexible one makes use of `If-Unmodified-Since` and `If-Match` and the server returns an error if the precondition fails; the client then restarts the download from the beginning:



Even if this method works, it adds an extra response/request exchange when the document has been changed. This impairs performance, and HTTP has a specific header to avoid this scenario: `If-Range`:

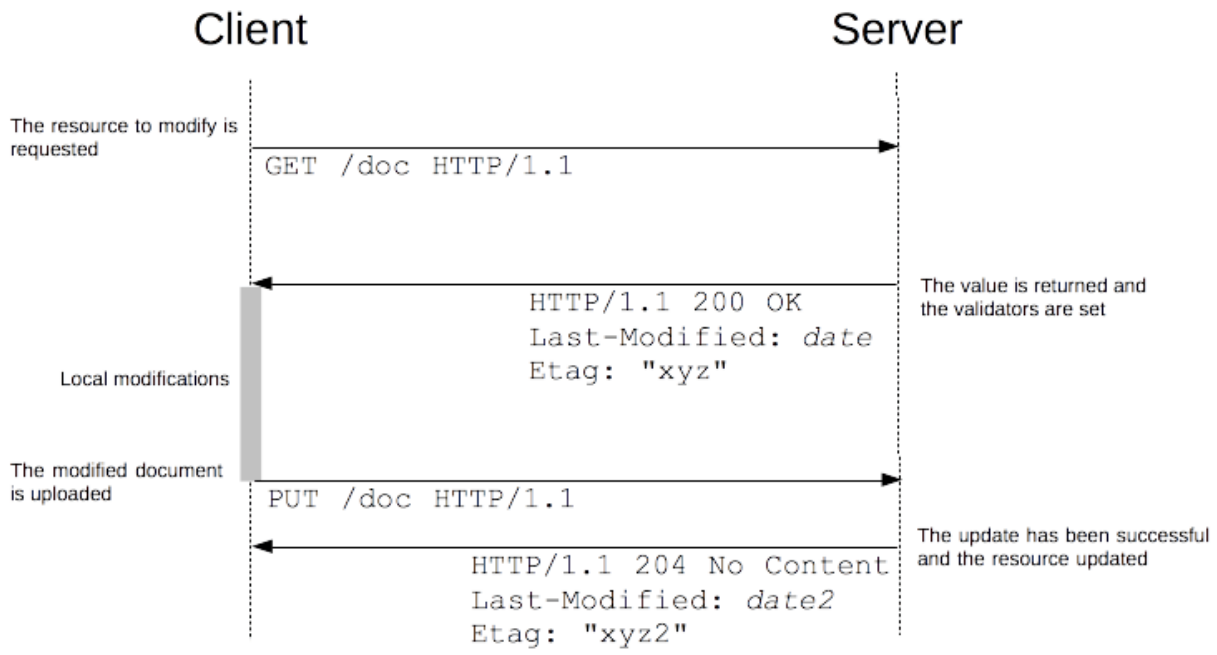


This solution is more efficient, but slightly less flexible, as only one etag can be used in the condition. Rarely is such additional flexibility needed.

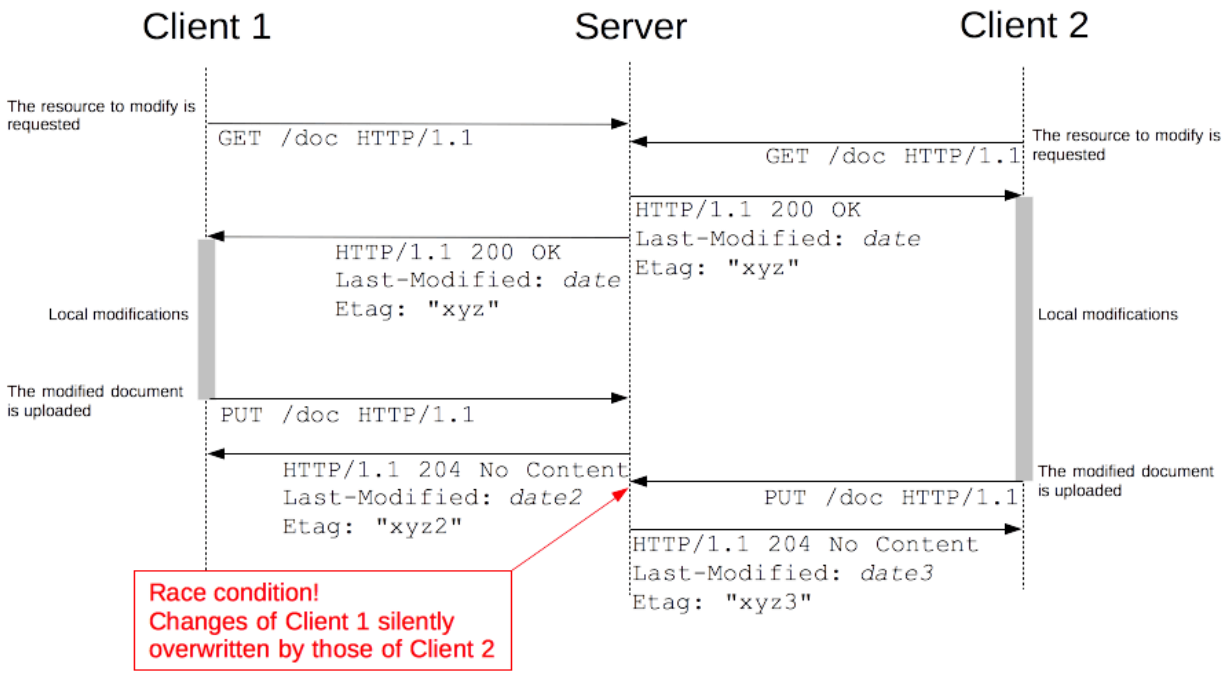
Avoiding the lost update problem with optimistic locking

A common operation in Web applications is to *update* a remote document. This is very common in any file system or source control applications, but any application that allows to store remote resources needs such a mechanism. Common websites, like wikis and other CMS, have such a need.

With the PUT method you are able to implement this. The client first reads the original files, modifies them, and finally pushes them to the server:

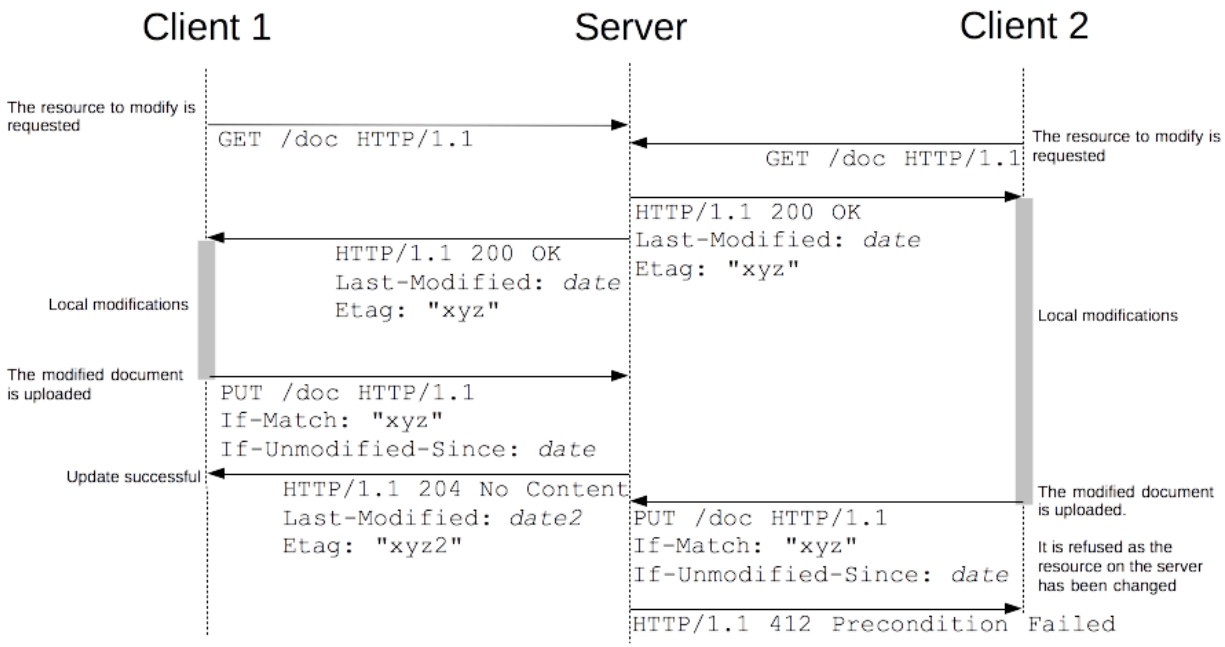


Unfortunately, things get a little inaccurate as soon as we take into account concurrency. While a client is locally modifying its new copy of the resource, a second client can fetch the same resource and do the same on its copy. What happens next is very unfortunate: when they commit back to the server, the modifications from the first client are discarded by the next client push, as this second client is unaware of the first client's changes to the resource. The decision on who wins is not communicated to the other party. Which client's changes are to be kept, will vary with the speed they commit; this depends on the performance of the clients, of the server, and even of the human editing the document at the client. The winner will change from one time to the next. This is a *race condition* and leads to problematic behaviors, which are difficult to detect and to debug:



There is no way to deal with this problem without annoying one of the two clients. However, lost updates and race conditions are to be avoided. We want predictable results, and expect that the clients are notified when their changes are rejected.

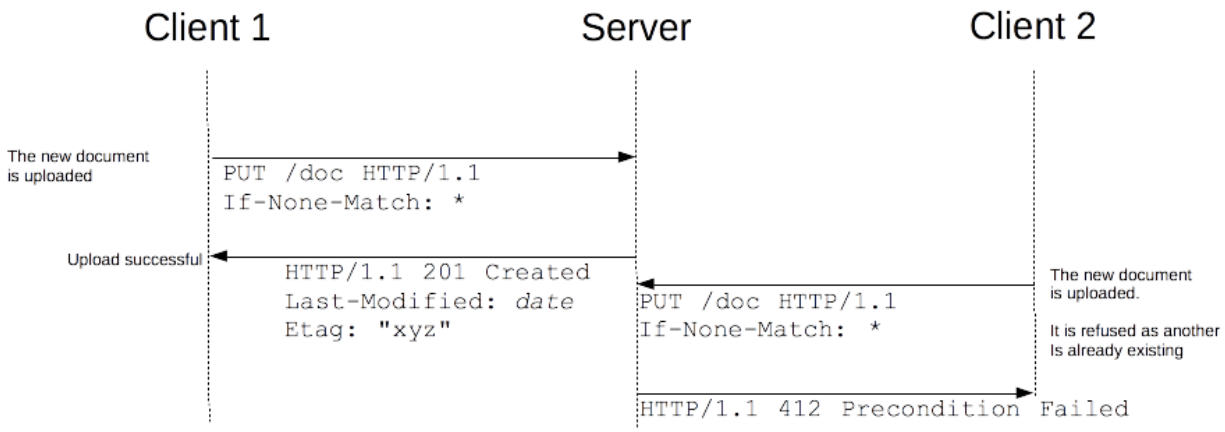
Conditional requests allow implementing the optimistic locking algorithm (used by most wikis or source control systems). The concept is to allow all clients to get copies of the resource, then let them modify it locally, controlling concurrency by successfully allowing the first client to submit an update. All subsequent updates, based on the now obsolete version of the resource, are rejected:



This is implemented using the `If-Match` or `If-Unmodified-Since` headers. If the etag doesn't match the original file, or if the file has been modified since it has been obtained, the change is rejected with a `412 Precondition Failed` error. It is then up to the client to deal with the error: either by notifying the user to start again (this time on the newest version), or by showing the user a diff of both versions, helping them decide which changes they wish to keep.

Dealing with the first upload of a resource

The first upload of a resource is an edge case of the previous. Like any update of a resource, it is subject to a race condition if two clients try to perform at similar times. To prevent this, conditional requests can be used: by adding `If-None-Match` with the special value of '*', representing any etag. The request will succeed, only if the resource didn't exist before:



`If-None-Match` will only work with HTTP/1.1 (and later) compliant servers. If unsure if the server will be compliant, you need first to issue a `HEAD` request to the resource to check this.

Conclusion

Conditional requests are a key feature of HTTP, and allow the building of efficient and complex applications. For caching or resuming downloads, the only work required for webmasters is to configure the server correctly; setting correct etags in some environments can be tricky. Once achieved, the browser will serve the expected conditional requests.

For locking mechanisms, it is the opposite: Web developers need to issue a request with the proper headers, while webmasters can mostly rely on the application to carry out the checks for them.

In both cases it's clear, conditional requests are a fundamental feature behind the Web.

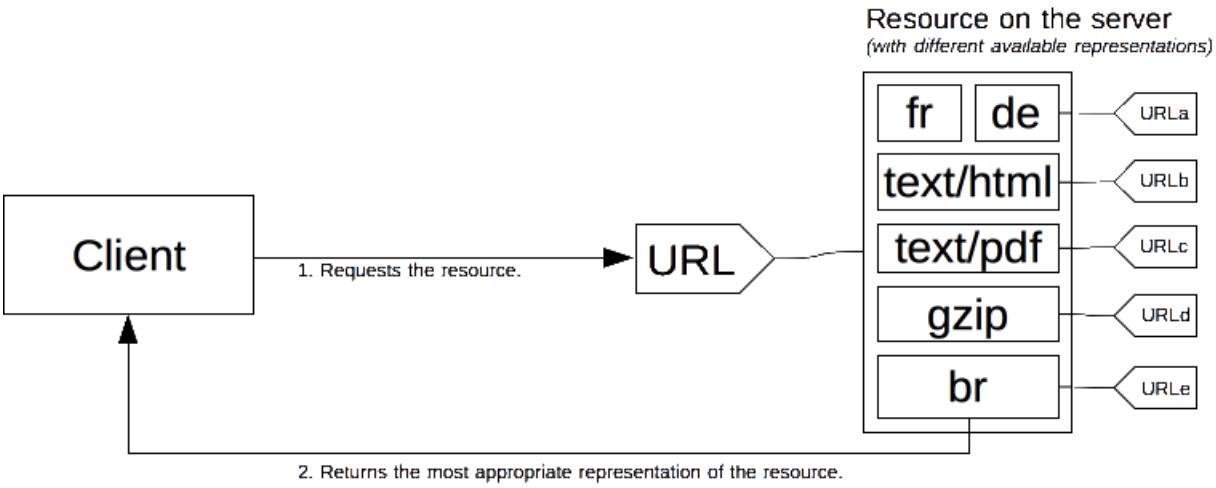
This lesson is adapted from MDN web https://developer.mozilla.org/en-US/docs/Web/HTTP/Conditional_requests

Content negotiation

In HTTP, content negotiation is the mechanism that is used for serving different representations of a resource to the same URI to help the user agent specify which representation is best suited for the user (for example, which document language, which image format, or which content encoding).

Principles of content negotiation

A specific document is called a resource. When a client wants to obtain a resource, the client requests it via a URL. The server uses this URL to choose one of the variants available–each variant is called a representation–and returns a specific representation to the client. The overall resource, as well as each of the representations, has a specific URL. Content negotiation determines how a specific representation is chosen when the resource is called. There are several ways of negotiating between the client and the server.



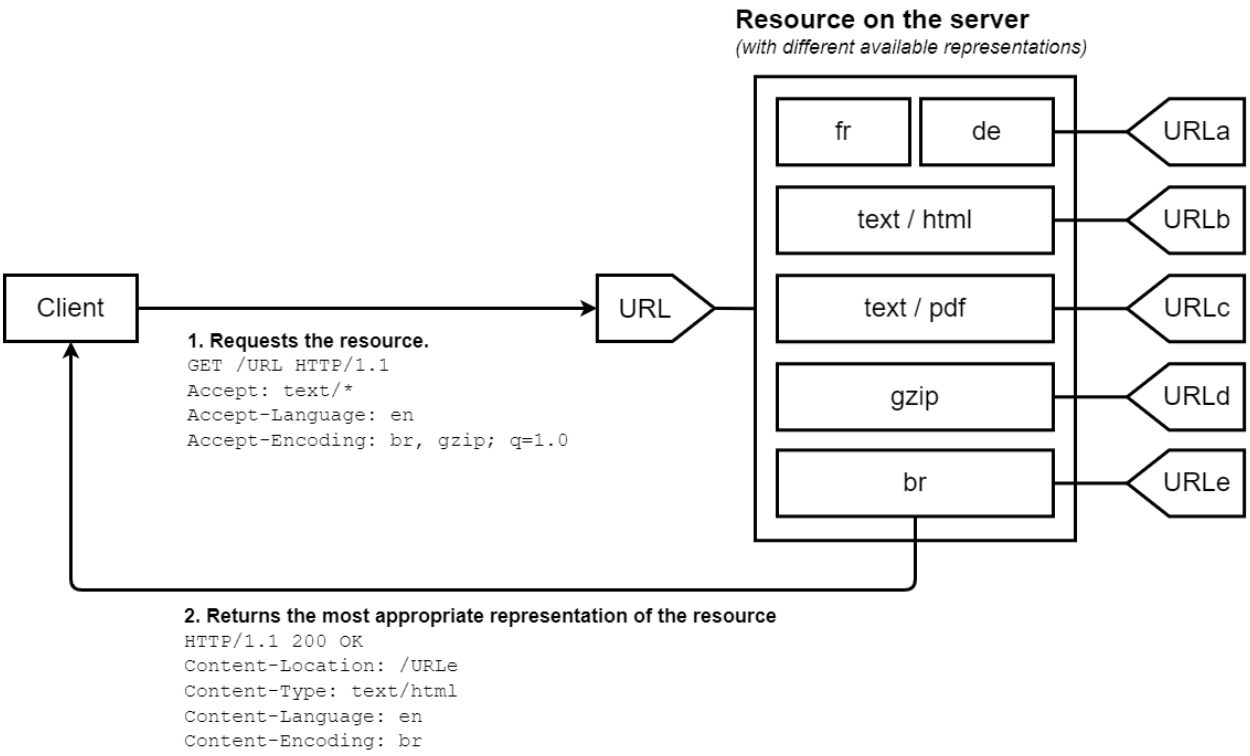
The best-suited representation is identified through one of two mechanisms:

- Specific HTTP headers by the client (*server-driven negotiation* or *proactive negotiation*), which is the standard way of negotiating a specific kind of resource.
- The `300` (Multiple Choices) or `406` (Not Acceptable), `415` (Unsupported Media Type) HTTP response codes by the server (*agent-driven negotiation* or *reactive negotiation*), that are used as fallback mechanisms.

Over the years, other content negotiation proposals, like transparent content negotiation and the `Alternates` header, have been proposed. They failed to get traction and were abandoned.

Server-driven content negotiation

In server-driven content negotiation or proactive content negotiation, the browser (or any other kind of user agent) sends several HTTP headers along with the URL. These headers describe the user's preferred choice. The server uses them as hints and an internal algorithm chooses the best content to serve to the client. If it can't provide a suitable resource, it might respond with `406` (Not Acceptable) or `415` (Unsupported Media Type) and set headers for the types of media that it does support (e.g., using the `Accept-Post` or `Accept-Patch` for POST and PATCH requests, respectively). The algorithm is server-specific and not defined in the standard. See the Apache negotiation algorithm.



The HTTP/1.1 standard defines list of the standard headers that start server-driven negotiation (such as `Accept`, `Accept-Encoding`, and `Accept-Language`). Though `User-Agent` isn't in this list, it's sometimes also used to send a specific representation of the requested resource. However, this isn't always considered a good practice. The server uses the `Vary` header to indicate which headers

it actually used for content negotiation (or more precisely, the associated request headers), so that caches can work optimally.

In addition to these, there's an experimental proposal to add more headers to the list of available headers, called *client hints*. Client hints advertise what kind of device the user agent runs on (for example, a desktop computer or a mobile device).

Even if server-driven content negotiation is the most common way to agree on a specific representation of a resource, it has several drawbacks:

- The server doesn't have total knowledge of the browser. Even with the Client Hints extension, it doesn't have a complete knowledge of the capabilities of the browser. Unlike reactive content negotiation where the client makes the choice, the server choice is always somewhat arbitrary.
- The information from the client is quite verbose (HTTP/2 header compression mitigates this problem) and a privacy risk (HTTP fingerprinting).
- As several representations of a given resource are sent, shared caches are less efficient and server implementations are more complex.

The `Accept` header

The Accept header lists the MIME types of media resources that the agent is willing to process. This is a comma-separated list of MIME types, each combined with a quality factor, a parameter that indicates the relative degree of preference between the different MIME types.
The Accept header is defined by the browser, or any other user agent, and can vary according to the context. For example, fetching an HTML page or an image, a video, or a script. It's different when fetching a document entered in the address bar or an element linked via an `<img>`, `<video>`, or `<audio>` element. Browsers are free to use the value of the header that they think is the most adequate; an exhaustive list of default values for common browsers is available.

The `Accept-CH` header

Note: This is part of an experimental technology called *Client Hints*. Initial support comes in Chrome 46 or later. The Device-Memory value is in Chrome 61 or later.

The experimental Accept-CH lists configuration data that the server can use to select an appropriate response. Valid values are:

Value	Meaning
`Device-Memory`	Indicates the approximate amount of device RAM. This value is an approximation given by rounding to the nearest power of 2 and dividing that number by 1024. For example, 512 megabytes will be reported as 0.5.
`Viewport-Width`	Indicates the layout viewport width in CSS pixels.
`Width`	Indicates the resource width in physical pixels (in other words the intrinsic size of an image).

The `Accept-CH-Lifetime` header

Note: This is part of an experimental technology called *Client Hints* and is only available in Chrome 61 or later.

The `Accept-CH-Lifetime` header is used with the Device-Memory value of the Accept-CH header and indicates the amount of time the device should opt in to sharing the device memory with the server. The value is given in milliseconds and it's optional.

The `Accept-Encoding` header

The `Accept-Encoding` header defines the acceptable content encoding (supported compressions). The value is a q-factor list (e.g., `br, gzip;q=0.8`) that indicates the priority of the encoding values. The default value `identity` is at the lowest priority (unless otherwise noted).

Compressing HTTP messages is one of the most important ways to improve the performance of a website. It shrinks the size of the data transmitted and makes better use of the available bandwidth. Browsers always send this header and the server should be configured to use compression.

The `Accept-Language` header

The `Accept-Language` header is used to indicate the language preference of the user. It's a list of values with quality factors (e.g., "`de, en;q=0.7`"). A default value is often set according to the

language of the graphical interface of the user agent, but most browsers allow different language preferences to be set.

Due to the configuration-based entropy increase, a modified value can be used to fingerprint the user. It's not recommended to change it and a website can't trust this value to reflect the actual intention of the user. It's best for site designers to avoid using language detection via this header as it can lead to a poor user experience.

- They should always provide a way to override the server-chosen language, e.g., by providing a language menu on the site. Most user agents provide a default value for the Accept-Language header that's adapted to the user interface language. End users often don't modify it because they either don't know how or aren't able to do so based on their computing environment.
- Once a user has overridden the server-chosen language, a site should no longer use language detection and should stick with the explicitly chosen language. In other words, only entry pages for a site should use this header to select the proper language.

The `User-Agent` header

Note: Though there are legitimate uses of this header for selecting content, it's considered bad practice to rely on it to define what features are supported by the user agent.

The `User-Agent` header identifies the browser sending the request. This string may contain a space-separated list of *product tokens* and *comments*.

A *product token* is a name followed by a ' / ' and a version number, like `Firefox/4.0.1`. The user agent can include as many of these as it wants. A *comment* is an optional string delimited by parentheses. The information provided in a comment isn't standardized, though several browsers add several tokens to it separated by ' ; '.

The `Vary response` header

In contrast to the previous `Accept-*` headers, which are sent by the client, the `Vary` HTTP header is sent by the web server in its response. It indicates the list of headers the server uses during the server-driven content negotiation phase. The `Vary` header is needed to inform the cache of the decision criteria so that it can reproduce it. This allows the cache to be functional while ensuring that the right content is served to the user.

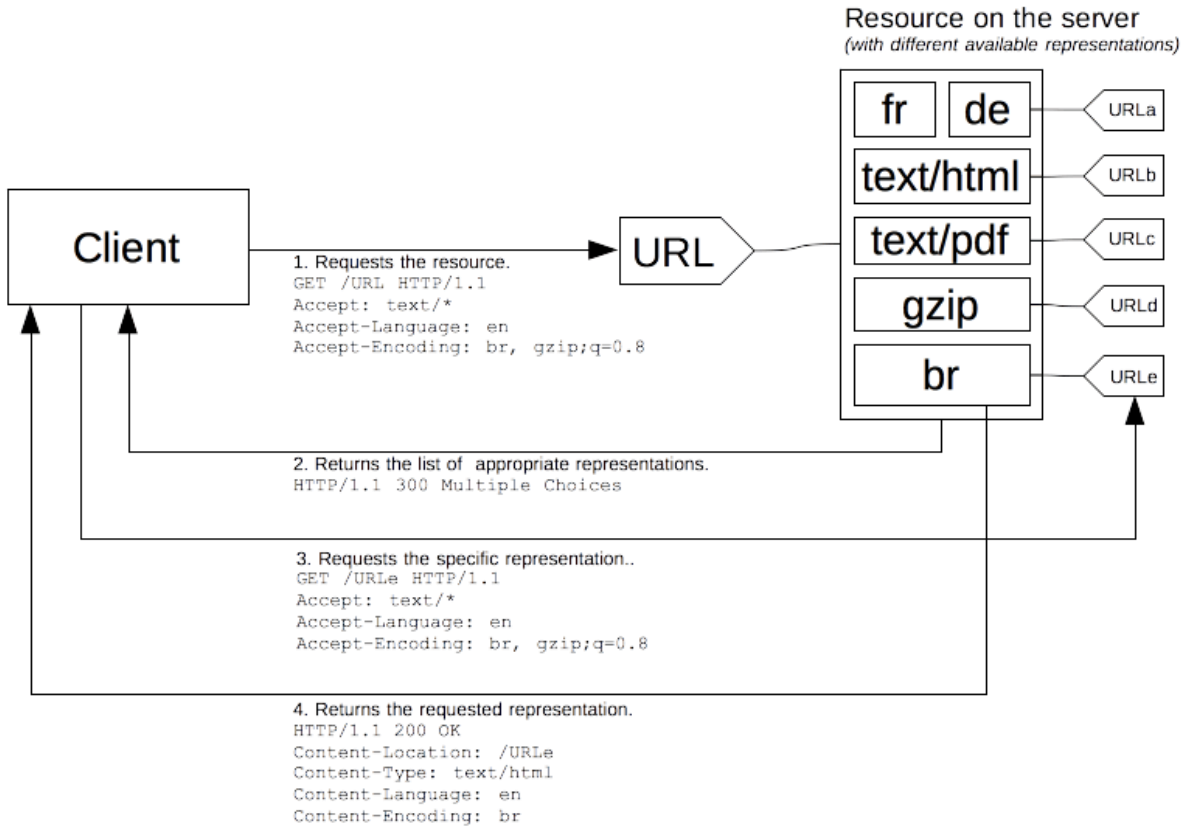
The special value ' * ' means that the server-driven content negotiation also uses information not conveyed in a header to choose the appropriate content.

The Vary header was added in version 1.1 of HTTP and allows caches to work appropriately. To work with server-driven content negotiation, a cache needs to know which criteria the server used to select the transmitted content. That way, the cache can replay the algorithm and will be able to serve acceptable content directly, without more requests to the server. Obviously, the wildcard '*' prevents caching from occurring, as the cache can't know what element is behind it. For more information, see HTTP caching > Varying responses.

Server-driven content negotiation

Server-driven negotiation's drawbacks: poor scalability - one header per feature, leading to multiple headers for different dimensions, causing increased message size and potential performance issues. Also, heightened privacy concerns due to increased HTTP fingerprinting.

HTTP allows another negotiation type: *agent-driven negotiation* or *reactive negotiation*. Server returns a page with links to available alternative resources for ambiguous requests. User selects preferred source.



Unfortunately, the HTTP standard doesn't specify the format of the page for choosing between the available resources, which prevents the process from being automated. Besides falling back to the server-driven negotiation, this method is almost always used with scripting, especially with JavaScript redirection: after having checked for the negotiation criteria, the script performs the redirection. A second problem is that one more request is needed to fetch the real resource, slowing the availability of the resource to the user.

This lesson is adapted from MDN web https://developer.mozilla.org/en-US/docs/Web/HTTP/Content_negotiation

Using HTTP cookies

An HTTP cookie (web cookie, browser cookie) is a **small piece of data that a server sends to a user's web browser**. The browser may store the cookie and send it back to the same server with later requests. Typically, an HTTP cookie is used to tell if two requests come from the same browser—keeping a user logged in, for example. It remembers stateful information for the stateless HTTP protocol.

Cookies are mainly used for three purposes:

1. Session management:
   - Logins, shopping carts, game scores, or anything else the server should remember
2. Personalization:
   - User preferences, themes, and other settings
3. Tracking:
   - Recording and analyzing user behavior

Cookies were once used for general client-side storage. While this made sense when they were the only way to store data on the client, modern storage APIs are now recommended. Cookies are sent with every request, so they can worsen performance (especially for mobile data connections). Modern APIs for client storage are the Web Storage API (`localStorage` and `sessionStorage`) and IndexedDB.

Note: To see stored cookies (and other storage that a web page can use), you can enable the Storage Inspector in Developer Tools and select Cookies from the storage tree.

Creating cookies

After receiving an HTTP request, a server can send one or more `Set-Cookie` headers with the response. The browser usually stores the cookie and sends it with requests made to the same server inside a `Cookie` HTTP header. You can specify an expiration date or time period after which the cookie shouldn't be sent. You can also set additional restrictions to a specific domain and path to limit where the cookie is sent. For details about the header attributes mentioned below, refer to the `Set-Cookie` reference article.

The `Set-Cookie` and `Cookie` headers

The `Set-Cookie` HTTP response header sends cookies from the server to the user agent. A simple cookie is set like this:

```
Set-Cookie: <cookie-name>=<cookie-value>
```

This instructs the server sending headers to tell the client to store a pair of cookies:

```
HTTP/2.0 200 OK
Content-Type: text/html
Set-Cookie: yummy_cookie=choco
Set-Cookie: tasty_cookie=strawberry

[page content]
```

Then, with every subsequent request to the server, the browser sends all previously stored cookies back to the server using the `Cookie` header.

```
GET /sample_page.html HTTP/2.0
Host: www.example.org
Cookie: yummy_cookie=choco; tasty_cookie=strawberry
```

Define the lifetime of a cookie

The lifetime of a cookie can be defined in two ways:

- Session cookies are deleted when the current session ends. The browser defines when the "current session" ends, and some browsers use session restoring when restarting. This can cause session cookies to last indefinitely.
- Permanent cookies are deleted at a date specified by the `Expires` attribute, or after a period of time specified by the `Max-Age` attribute.

For example:

```
Set-Cookie: id=a3fWa; Expires=Thu, 31 Oct 2021 07:28:00 GMT;
```

Note: When you set an Expires date and time, they're relative to the client the cookie is being set on, not the server.

If your site authenticates users, it should regenerate and resend session cookies, even ones that already exist, whenever a user authenticates. This approach helps prevent session fixation attacks, where a third party can reuse a user's session.

Restrict access to cookies

You can ensure that cookies are sent securely and aren't accessed by unintended parties or scripts in one of two ways: with the `Secure` attribute and the `HttpOnly` attribute.

A cookie with the `Secure` attribute is only sent to the server with an encrypted request over the HTTPS protocol. It's never sent with unsecured HTTP (except on localhost), which means man-in-the-middle attackers can't access it easily. Insecure sites (with `http`: in the URL) can't set cookies with the Secure attribute. However, don't assume that `Secure` prevents all access to sensitive information in cookies. For example, someone with access to the client's hard disk (or JavaScript if the `HttpOnly` attribute isn't set) can read and modify the information.

A cookie with the `HttpOnly` attribute is inaccessible to the JavaScript `Document.cookie` API; it's only sent to the server. For example, cookies that persist in server-side sessions don't need to be available to JavaScript and should have the `HttpOnly` attribute. This precaution helps mitigate cross-site scripting (XSS) attacks.

Here's an example:

```
Set-Cookie: id=a3fWa; Expires=Thu, 21 Oct 2021 07:28:00 GMT; Secure; HttpOnly
```

Define where cookies are sent

The `Domain` and `Path` attributes define the scope of a cookie: what URLs the cookies should be sent to.

Domain attribute

The `Domain` attribute specifies which hosts can receive a cookie. If the server does not specify a `Domain`, the browser defaults the domain to the same host that set the cookie, excluding

subdomains. If `Domain` is specified, then subdomains are always included. Therefore, specifying `Domain` is less restrictive than omitting it. However, it can be helpful when subdomains need to share information about a user.

For example, if you set `Domain=mozilla.org`, cookies are available on subdomains like `developer.mozilla.org`.

Path attribute

The `Path` attribute indicates a URL path that must exist in the requested URL in order to send the Cookie header. The `%x2F` ("/") character is considered a directory separator, and subdirectories match as well.

For example, if you set `Path=/docs`, these request paths match:

- `/docs`
- `/docs/`
- `/docs/Web/`
- `/docs/Web/HTTP`

But these request paths don't:

- `/`
- `/docsets`
- `/fr/docs`

SameSite attribute

The `SameSite` attribute lets servers specify whether/when cookies are sent with cross-site requests (where Site is defined by the registrable domain and the scheme: http or https). This provides some protection against cross-site request forgery attacks (CSRF). It takes three possible values: `Strict`, `Lax`, and `None`.

With `Strict`, the browser only sends the cookie with requests from the cookie's origin site. `Lax` is similar, except the browser also sends the cookie when the user navigates to the cookie's origin site (even if the user is coming from a different site). For example, by following a link from an external site. `None` specifies that cookies are sent on both originating and cross-site requests, but only in secure contexts (i.e., if `SameSite=None` then the `Secure` attribute must also be set). If no `SameSite` attribute is set, the cookie is treated as `Lax`.

Here's an example:

```
Set-Cookie: mykey=myvalue; SameSite=Strict
```

Cookie prefixes

Because of the design of the cookie mechanism, a server can't confirm that a cookie was set from a secure origin or even tell where a cookie was originally set.

A vulnerable application on a subdomain can set a cookie with the Domain attribute, which gives access to that cookie on all other subdomains. This mechanism can be abused in a session fixation attack. See session fixation for primary mitigation methods.

As a defense-in-depth measure, however, you can use cookie prefixes to assert specific facts about the cookie. Two prefixes are available:

`__Host-`

If a cookie name has this prefix, it's accepted in a `Set-Cookie` header only if it's also marked with the `Secure` attribute, was sent from a secure origin, does not include a `Domain` attribute, and has the Path attribute set to /. This way, these cookies can be seen as "domain-locked".

`__Secure-`

If a cookie name has this prefix, it's accepted in a `Set-Cookie` header only if it's marked with the `Secure` attribute and was sent from a secure origin. This is weaker than the `__Host-` prefix.

The browser will reject cookies with these prefixes that don't comply with their restrictions. Note that this ensures that subdomain-created cookies with prefixes are either confined to the subdomain or ignored completely. As the application server only checks for a specific cookie name when determining if the user is authenticated or a CSRF token is correct, this effectively acts as a defense measure against session fixation.

JavaScript access using Document.cookie

You can create new cookies via JavaScript using the `Document.cookie` property. You can access existing cookies from JavaScript as well if the `HttpOnly` flag isn't set.

```
document.cookie = "yummy_cookie=choco";
document.cookie = "tasty_cookie=strawberry";
console.log(document.cookie);
```

```
// logs "yummy_cookie=choco; tasty_cookie=strawberry"
```

Cookies created via JavaScript can't include the `HttpOnly` flag.

Please note the security issues in the Security section below. Cookies available to JavaScript can be stolen through XSS.

Security

Note: When you store information in cookies, keep in mind that all cookie values are visible to, and can be changed by, the end user. Depending on the application, you may want to use an opaque identifier that the server looks up, or investigate alternative authentication/confidentiality mechanisms such as JSON Web Tokens.

Ways to mitigate attacks involving cookies:

- Use the `HttpOnly` attribute to prevent access to cookie values via JavaScript.
- Cookies that are used for sensitive information (such as indicating authentication) should have a short lifetime, with the `SameSite` attribute set to Strict or Lax. (See SameSite attribute, above.) In browsers that support SameSite, this ensures that the authentication cookie isn't sent with cross-site requests. This would make the request effectively unauthenticated to the application server.

Creating cookies

Third-party cookies

A cookie is associated with a particular domain and scheme (such as `http` or `https`), and may also be associated with subdomains if the `Set-Cookie Domain` attribute is set. If the cookie domain and scheme match the current page, the cookie is considered to be from the same site as the page, and is referred to as a first-party cookie.

If the domain and scheme are different, the cookie is not considered to be from the same site, and is referred to as a third-party cookie. While the server hosting a web page sets first-party cookies, the page may contain images or other components stored on servers in other domains

(for example, ad banners) that may set third-party cookies. These are mainly used for advertising and tracking across the web. For example, the types of cookies used by Google.

A third-party server can create a profile of a user's browsing history and habits based on cookies sent to it by the same browser when accessing multiple sites. Firefox, by default, blocks third-party cookies that are known to contain trackers. Third-party cookies (or just tracking cookies) may also be blocked by other browser settings or extensions. Cookie blocking can cause some third-party components (such as social media widgets) not to function as intended.

There are some useful features available for developers who wish to respect user privacy, and minimize third-party tracking:

- Servers can (and should) set the cookie SameSite attribute to specify whether or not third-party cookies may be sent.
- Cookies Having Independent Partitioned State (CHIPS) enables developers to opt-in their cookies to partitioned storage, with a separate cookie jar per top-level site. This enables valid non-tracking uses of third-party cookies to continue working in browsers that do not allow cookies to be used for third-party tracking.

Cookie-related regulations

Legislation or regulations that cover the use of cookies include:

- The General Data Privacy Regulation (GDPR) in the European Union
- The ePrivacy Directive in the EU
- The California Consumer Privacy Act

These regulations have a global reach. They apply to any site on the World Wide Web that users from these jurisdictions access (the EU and California).

These regulations include requirements such as:

- Notifying users that your site uses cookies.
- Allowing users to opt out of receiving some or all cookies.
- Allowing users to use the bulk of your service without receiving cookies.

There may be other regulations that govern the use of cookies in your locality. The burden is on you to know and comply with these regulations. There are companies that offer "cookie banner" code that helps you comply with these regulations.

Other ways to store information in the browser

Another approach to storing data in the browser is the Web Storage API. The window.sessionStorage and window.localStorage properties correspond to session and permanent cookies in duration, but have larger storage limits than cookies, and are never sent to a server. More structured and larger amounts of data can be stored using the IndexedDB API, or a library built on it.

There are some techniques designed to recreate cookies after they're deleted. These are known as "zombie" cookies. These techniques violate the principles of user privacy and user control, may violate data privacy regulations, and could expose a website using them to legal liability.

This lesson was adapted from MDN Web Docs: https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies

HTTP range requests

An HTTP `Range` request asks the server to send only a portion of an HTTP message back to a client. Range requests are useful for clients like media players that support random access, data tools that know they need only part of a large file, and download managers that let the user pause and resume the download.

Checking if a server supports partial requests

If an HTTP response includes the `Accept-Ranges` header and its value is anything other than "none", then the server supports range requests. You can perform a manual check by issuing a `HEAD` request with a tool like cURL.

```
curl -I http://i.imgur.com/z4d4kWk.jpg
```

```
HTTP/1.1 200 OK
…
Accept-Ranges: bytes
Content-Length: 146515
```

In this response, `Accept-Ranges: bytes` indicates that bytes can be used as units to define a range. Here the `Content-Length` header is also useful as it indicates the full size of the image to retrieve.

If sites omit the `Accept-Ranges` header, they likely don't support partial requests. Some sites include the header but give it the explicit value "none" to indicate they lack support:

```
curl -I https://www.youtube.com/watch?v=EwTZ2xpQwpA
```

```
HTTP/1.1 200 OK
…
Accept-Ranges: none
```

A download manager might disable its pause button in that case.

Requesting a specific range from a server

If the server supports range requests, then by including the `Range` header in your HTTP request, you can specify which part or parts of the document you want the server to return.

Single part ranges

We can request a single range from a resource. Again, we can test a request by using cURL. The "`-H`" option will append a header line to the request, which in this case is the `Range` header requesting the first 1024 bytes.

```
curl http://i.imgur.com/z4d4kWk.jpg -i -H "Range: bytes=0-1023"
```

The issued request looks like this:

```
GET /z4d4kWk.jpg HTTP/1.1
Host: i.imgur.com
Range: bytes=0-1023
```

The server responses with the `206 Partial Content` status:

```
HTTP/1.1 206 Partial Content
Content-Range: bytes 0-1023/146515
Content-Length: 1024
…
(binary content)
```

The `Content-Length` header now indicates the size of the requested range (and not the full size of the image). The `Content-Range` response header indicates where in the full resource this partial message belongs.

Multipart ranges

The `Range` header also allows you to get multiple ranges at once in a multipart document. The ranges are separated by a comma.

```
curl http://www.example.com -i -H "Range: bytes=0-50, 100-150"
```

The server responses with the `206 Partial Content` status and a `Content-Type: multipart/byteranges; boundary=3d6b6a416f9b5` header, indicating that a multipart byterange follows. Each part contains its own `Content-Type` and `Content-Range` fields and the required boundary parameter specifies the boundary string used to separate each body-part.

```
HTTP/1.1 206 Partial Content
Content-Type: multipart/byteranges; boundary=3d6b6a416f9b5
Content-Length: 282

--3d6b6a416f9b5
Content-Type: text/html
Content-Range: bytes 0-50/1270

<!DOCTYPE html>
<html lang="en-US">
<head>
    <title>Example Do
--3d6b6a416f9b5
Content-Type: text/html
Content-Range: bytes 100-150/1270

eta http-equiv="Content-type" content="text/html; c
--3d6b6a416f9b5—
```

Conditional range requests

When resuming to request more parts of a resource, you need to guarantee that the stored resource has not been modified since the last fragment has been received.

The `If-Range` HTTP request header makes a range request conditional: if the condition is fulfilled, the range request will be issued and the server sends back a `206 Partial Content` answer with the appropriate body. If the condition is not fulfilled, the full resource is sent back, with

a `200 OK` status. This header can be used either with a `Last-Modified` validator, or with an `ETag`, but not with both.

```
If-Range: Wed, 21 Oct 2015 07:28:00 GMT
```

Partial request responses

There are three relevant statuses, when working with range requests:

- A successful range request elicits a `206 Partial Content` status from the server.
- A range request that is out of bounds will result in a `416 Requested Range Not Satisfiable` status, meaning that none of the range values overlap the extent of the resource. For example, the first-byte-pos of every range might be greater than the resource length.
- If range requests are not supported, an `200 OK` status is sent back and the entire response body is transmitted.

Comparison to chunked `Transfer-Encoding`

The `Transfer-Encoding` header allows chunked encoding, which is useful when larger amounts of data are sent to the client and the total size of the response is not known until the request has been fully processed. The server sends data to the client straight away without buffering the response or determining the exact length, which leads to improved latency. Range requests and chunking are compatible and can be used with or without each other.

This lesson is adapted from MDN web https://developer.mozilla.org/en-US/docs/Web/HTTP/Range_requests

Redirections in HTTP

URL redirection, also known as *URL forwarding*, is a technique to give more than one URL address to a page, a form, a whole website, or a web application. HTTP has a special kind of response, called a ***HTTP redirect***, for this operation.

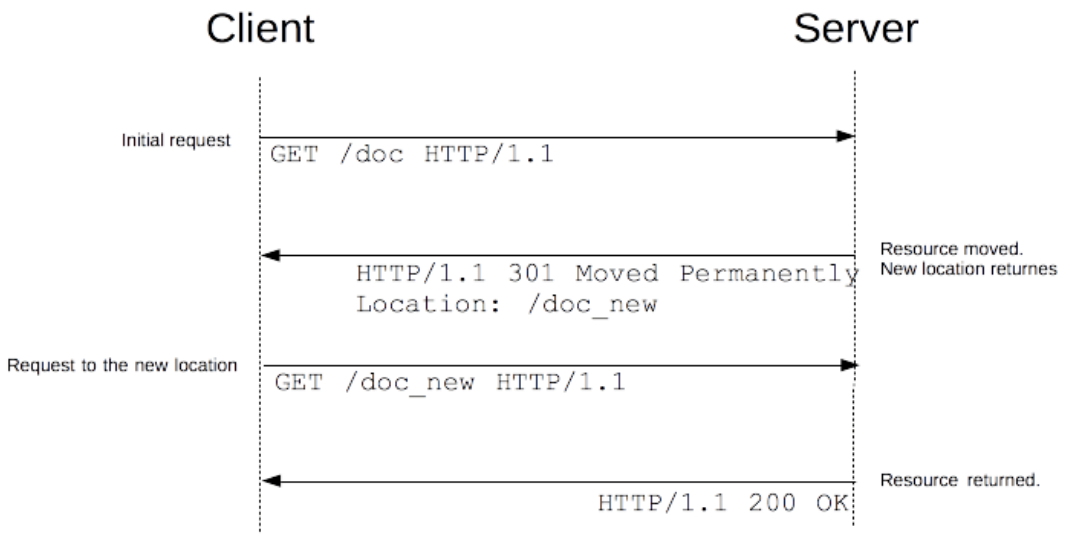
Redirects accomplish numerous goals:

- Temporary redirects during site maintenance or downtime
- Permanent redirects to preserve existing links/bookmarks after changing the site's URLs, progress pages when uploading a file, etc.

Principle

In HTTP, redirection is triggered by a server sending a special *redirect* response to a request. Redirect responses have status codes that start with `3`, and a `Location` header holding the URL to redirect to.

When browsers receive a redirect, they immediately load the new URL provided in the Location header. Besides the small performance hit of an additional round-trip, users rarely notice the redirection.



There are several types of redirects, sorted into three categories:

1. Permanent redirections
2. Temporary redirections
3. Special redirections

Permanent redirections

These redirections are meant to last forever. They imply that the original URL should no longer be used, and replaced with the new one. Search engine robots, RSS readers, and other crawlers will update the original URL for the resource.

Code	Text	Method handling	Typical use case
`301`	`Moved Permanently`	`GET` methods unchanged. Others may or may not be changed to `GET`. [1]	Reorganization of a website.
`308`	`Permanent Redirect`	Method and body not changed.	Reorganization of a website, with non-GET links/operations.

[1] The specification did not intend to allow method changes, but there are existing user agents that do change their method. `308` was created to remove the ambiguity of the behavior when using non-`GET` methods.

Temporary redirections

Sometimes the requested resource can't be accessed from its canonical location, but it can be accessed from another place. In this case, a temporary redirect can be used.

Search engine robots and other crawlers don't memorize the new, temporary URL. Temporary redirections are also used when creating, updating, or deleting resources, to show temporary progress pages.

Code	Text	Method handling	Typical use case
`302`	`Found`	`GET` methods unchanged. Others may or may not be changed to `GET`. [2]	The Web page is temporarily unavailable for unforeseen reasons.

303	See Other	GET methods unchanged. Others changed to GET (body lost).	Used to redirect after a PUT or a POST, so that refreshing the result page doesn't re-trigger the operation.
307	Temporary Redirect	Method and body not changed	The Web page is temporarily unavailable for unforeseen reasons. Better than 302 when non-GET operations are available on the site.

[2] The specification did not intend to allow method changes, but there are existing user agents that do change their method. 307 was created to remove the ambiguity of the behavior when using non-GET methods.

Special redirections

304 (Not Modified) redirects a page to the locally cached copy (that was stale), and 300 (Multiple Choice) is a manual redirection: the body, presented by the browser as a Web page, lists the possible redirections and the user clicks on one to select it.

Code	Text	Typical use case
300	Multiple Choice	Not many: the choices are listed in an HTML page in the body. Machine-readable choices are encouraged to be sent as Link headers with rel=alternate.
304	Not Modified	Sent for revalidated conditional requests. Indicates that the cached response is still fresh and can be used.

Alternative way of specifying redirections

HTTP redirects aren't the only way to define redirections. There are two others:

1. HTML redirections with the <meta> element
2. JavaScript redirections via the DOM

HTML redirections

HTTP redirects are the best way to create redirections, but sometimes you don't have control over the server. In that case, try a `<meta>` element with its `http-equiv` attribute set to Refresh in the `<head>` of the page. When displaying the page, the browser will go to the indicated URL.

```
<head>
  <meta http-equiv="Refresh" content="0; URL=https://example.com/" />
</head>
```

The `content` attribute should start with a number indicating how many seconds the browser should wait before redirecting to the given URL. Always set it to 0 for accessibility compliance.
Obviously, this method only works with HTML, and cannot be used for images or other types of content.

JavaScript redirections

Redirections in JavaScript are performed by setting a URL string to the `window.location` property, loading the new page:

```
window.location = "https://example.com/";
```

Like HTML redirections, this can't work on all resources, and obviously, this will only work on clients that execute JavaScript. On the other hand, there are more possibilities: for example, you can trigger the redirect only if some conditions are met.

Order of precedence

With three ways to trigger redirections, several ways can be used at the same time. But which is applied first?

1. HTTP redirects always execute first — they exist when there is not even a transmitted page.
2. HTML redirects (`<meta>`) execute if there weren't any HTTP redirects.
3. JavaScript redirects execute last, and only if JavaScript is enabled.

When possible, use HTTP redirects and don't add `<meta>` element redirects. If someone changes the HTTP redirects but forgets to change the HTML redirects, the redirects will no longer be identical, which could cause an infinite loop or other nightmares.

Use cases

There are numerous use cases for redirects, but as performance is impacted with every redirect, their use should be kept to a minimum.

Domain aliasing

Ideally, there is one location, and therefore one URL, for each resource. But there are reasons for alternative names for a resource:

Expanding the reach of your site

A common case is when a site resides at `www.example.com`, but accessing it from `example.com` should also work. Redirections for `example.com` to `www.example.com` are thus set up. You might also redirect from common synonyms or frequent typos of your domains.

Moving to a new domain

For example, your company was renamed, but you want existing links or bookmarks to still find you under the new name.

Forcing HTTPS

Requests to the `http://` version of your site will redirect to the `https://` version of your site.

Keeping links alive

When you restructure websites, URLs change. Even if you update your site's links to match the new URLs, you have no control over the URLs used by external resources.

You don't want to break these links, as they bring valuable users and help your SEO, so you set up redirects from the old URLs to the new ones.

Note: This technique does work for internal links, but try to avoid having internal redirects. A redirect has a significant performance cost (as an extra HTTP request occurs). If you can avoid it by correcting internal links, you should fix those links instead.

Temporary responses to unsafe requests

Unsafe requests modify the state of the server and the user shouldn't resend them unintentionally.

Typically, you don't want your users to resend `PUT`, `POST` or `DELETE` requests. If you serve the response as the result of this request, a simple press of the reload button will resend the request (possibly after a confirmation message).

In this case, the server can send back a `303` (See Other) response for a URL that will contain the right information. If the reload button is pressed, only that page is redisplayed, without replaying the unsafe requests.

Temporary responses to long requests

Some requests may need more time on the server, like DELETE requests that are scheduled for later processing. In this case, the response is a `303` (See Other) redirect that links to a page indicating that the action has been scheduled, and eventually informs about its progress, or allows to cancel it.

Configuring redirects in common servers

Apache

Redirects can be set either in the server config file or in the `.htaccess` of each directory.

The `mod_alias` module has `Redirect` and `RedirectMatch` directives that set up `302` redirects by default:

```
XMLCopy to Clipboard
<VirtualHost *:443>
  ServerName example.com
  Redirect / https://www.example.com
</VirtualHost>
```

The URL `https://example.com/` will be redirected to `https://www.example.com/`, as will any files or directories under it (`https://example.com/some-page` will be redirected to `https://www.example.com/some-page`)

`RedirectMatch` does the same, but takes a regular expression to define a collection of affected URLs:

```
RedirectMatch ^/images/(.*)$ https://images.example.com/$1
```

All documents in the `images/` directory will redirect to a different domain.

If you don't want a temporary redirect, an extra parameter (either the HTTP status code to use or the `permanent` keyword) can be used to set up a different redirect:

```
Redirect permanent / https://www.example.com
# …acts the same as:
Redirect 301 / https://www.example.com
```

The `mod_rewrite` module can also create redirects. It is more flexible, but a bit more complex.

Nginx

In Nginx, you create a specific server block for the content you want to redirect:

```
server {
  listen 80;
  server_name example.com;
  return 301 $scheme://www.example.com$request_uri;
}
```

To apply a redirect to a directory or only certain pages, use the rewrite directive:

```
rewrite ^/images/(.*)$ https://images.example.com/$1 redirect;
rewrite ^/images/(.*)$ https://images.example.com/$1 permanent;
```

IIS

In IIS, you use the `<httpRedirect>` element to configure redirections.

Configuring redirects in common servers

Redirection loops happen when additional redirections follow the one that has already been followed. In other words, there is a loop that will never be finished and no page will ever be found.

Most of the time this is a server problem, and if the server can detect it, it will send back a `500` Internal Server Error. If you encounter such an error soon after modifying a server configuration, this is likely a redirection loop.

Sometimes, the server won't detect it: a redirection loop can spread over several servers which each don't have the full picture. In this case, browsers will detect it and display an error message. Firefox displays:

> Firefox has detected that the server is redirecting the request for this address in a way that will never terminate.

...while Chrome displays:

> This Webpage has a redirect loop

In both cases, the user can't do much (unless corruption is happening on their side, like a mismatch of cache or cookies).

It is important to avoid redirection loops, as they completely break the user experience.

This lesson is adapted from MDN web https://developer.mozilla.org/en-US/docs/Web/HTTP/Redirections

HTTP resources and specifications

HTTP was first specified in the early 1990s. Designed with extensibility in mind, it has seen numerous additions over the years; this led to its specification being scattered through numerous specification documents (in the midst of experimental abandoned extensions). This page lists relevant resources about HTTP.

Specification	Title	Status
RFC 9110	HTTP Semantics	Internet Standard
RFC 9111	HTTP Caching	Internet Standard
RFC 9112	HTTP/1.1	Internet Standard
RFC 9113	HTTP/2	Proposed Standard
RFC 9114	HTTP/3	Proposed Standard
RFC 5861	HTTP Cache-Control Extensions for Stale Content	Informational
RFC 8246	HTTP Immutable Responses	Proposed Standard
RFC 6265	HTTP State Management Mechanism Defines Cookies	Proposed Standard
Draft spec	Cookie Prefixes	IETF Draft
Draft spec	Same-Site Cookies	IETF Draft
Draft spec	Deprecate modification of 'secure' cookies from non-secure origins	IETF Draft
RFC 2145	Use and Interpretation of HTTP Version Numbers	Informational
RFC 6585	Additional HTTP Status Codes	Proposed Standard
RFC 7725	An HTTP Status Code to Report Legal Obstacles	On the standard track
RFC 2397	The "data" URL scheme	Proposed Standard

RFC 3986	Uniform Resource Identifier (URI): Generic Syntax	Internet Standard
RFC 5988	Web Linking Defines the `Link` header	Proposed Standard
Experimental spec	Hypertext Transfer Protocol (HTTP) Keep-Alive Header	Informational (Expired)
Draft spec	HTTP Client Hints	IETF Draft
RFC 7578	Returning Values from Forms: multipart/form-data	Proposed Standard
RFC 6266	Use of the Content-Disposition Header Field in the Hypertext Transfer Protocol (HTTP)	Proposed Standard
RFC 2183	Communicating Presentation Information in Internet Messages: Content-Disposition Header Field Only a subset of syntax of the `Content-Disposition` header can be used in HTTP messages.	Proposed Standard
RFC 7239	Forwarded HTTP Extension	Proposed Standard
RFC 6455	The WebSocket Protocol	Proposed Standard
RFC 5246	The Transport Layer Security (TLS) Protocol Version 1.2 This specification has been modified by subsequent RFCs, but these modifications have no effect on the HTTP protocol.	Proposed Standard
RFC 8446	The Transport Layer Security (TLS) Protocol Version 1.3 Supersedes TLS 1.2.	Proposed Standard
RFC 2817	Upgrading to TLS Within HTTP/1.1	Proposed Standard
RFC 7541	HPACK: Header Compression for HTTP/2	On the standard track
RFC 7838	HTTP Alternative Services	On the standard track
RFC 7301	Transport Layer Security (TLS) Application-Layer Protocol Negotiation Extension Used to negotiate HTTP/2 at the transport to save an extra request/response round trip.	Proposed Standard

RFC 6454	The Web Origin Concept	Proposed Standard
Fetch	Cross-Origin Resource Sharing	Living Standard
RFC 7034	HTTP Header Field X-Frame-Options	Informational
RFC 6797	HTTP Strict Transport Security (HSTS)	Proposed Standard
Upgrade Insecure Requests	Upgrade Insecure Requests	Candidate Recommendation
Content Security Policy 1.0	Content Security Policy 1.0 CSP 1.1 and CSP 3.0 doesn't extend the HTTP standard	Obsolete
Microsoft document	Specifying legacy document modes* Defines X-UA-Compatible	Note
RFC 5689	HTTP Extensions for Web Distributed Authoring and Versioning (WebDAV) These extensions of the Web, as well as CardDAV and CalDAV, are out-of-scope for HTTP on the Web. Modern APIs for application are defines using the RESTful pattern nowadays.	Proposed Standard
RFC 2324	Hyper Text Coffee Pot Control Protocol (HTCPCP/1.0)	April 1st joke spec
RFC 7168	The Hyper Text Coffee Pot Control Protocol for Tea Efflux Appliances (HTCPCP-TEA)	April 1st joke spec
HTML Living Standard	HTML Defines extensions of HTTP for Server-Sent Events	Living Standard
Reporting API	`Report-To` header	Draft
Draft spec	Expect-CT Extension for HTTP	IETF Draft
RFC 7486	HTTP Origin-Bound Auth (HOBA)	Experimental

This lesson is adapted from MDN web https://developer.mozilla.org/en-US/docs/Web/HTTP/Resources_and_specifications